

Contents

第 60 章：面试中的项目表达、作品集与系统设计	7
60.1 项目表达的核心：从技术栈转向决策链	8
60.2 面试中讲项目的推荐结构	8
30 秒版：让对方知道这是什么	8
2 分钟版：展示完整工程思考	8
深入追问版：准备可展开的细节	9
60.3 作品集：不要只放代码，要放证据	9
60.4 项目不大也可以讲深	10
60.5 系统设计题：先问清楚边界	11
60.6 一个 RAG 系统设计的表达框架	11
60.7 Agent 系统设计：重点不是“会调用工具”	12
60.8 被问到不会的问题怎么办	12
60.9 简历上的项目应该怎么写	12
60.10 常见扣分表达	13
60.11 面试前的项目复盘清单	13
60.12 小结	14
第 61 章：大模型工程面试中的高频问题与现场推理	14
61.1 回答技术问题的基本结构	14
61.2 高频问题：RAG 和长上下文怎么选	15
61.3 高频问题：chunk size 怎么选	16
61.4 高频问题：向量检索、关键词检索和混合检索怎么选	16
61.5 高频问题：embedding 模型怎么评估	17
61.6 高频问题：rerank 是否一定需要	17
61.7 高频问题：如何降低大模型应用成本	18
61.8 高频问题：如何设计大模型应用评测体系	18
61.9 高频问题：微调、RAG 和 prompt engineering 怎么选	19
61.10 高频问题：Agent 为什么不稳定	20
61.11 高频问题：如何处理模型输出不稳定	20
61.12 高频问题：如何做线上监控和故障排查	21
61.13 现场推理题：不要急着给结论	22
61.14 面试回答中的“边界感”	22
61.15 小结	22
第 62 章：行为面试、面后复盘与 Offer 决策	23
62.1 行为面试考的不是“性格好不好”	23
62.2 STAR 方法要用，但不要机械套模板	23
62.3 准备行为面试：不要背答案，要准备素材库	24
62.4 如何回答“讲一次失败经历”	25
62.5 如何回答“你和别人意见不一致怎么办”	26
62.6 如何回答“为什么离开上一家公司”	26
62.7 如何回答“你的短板是什么”	27
62.8 反问环节：不要只问福利和流程	27
62.9 面试结束后必须复盘	28
62.10 如何判断一场面试的信号	28
62.11 Offer 决策不要只看总包	29
62.12 薪资沟通：专业、具体、留有余地	29
62.13 如何优雅拒绝 Offer	30
62.14 入职前要确认的事项	30
62.15 把求职当成长期系统	31
62.16 小结	31
第 63 章：入职准备、试用期与前 90 天站稳岗位	31
63.1 入职前：把不确定性提前降下来	32

63.2 不要在入职前盲目“狂补技术”	33
63.3 第一周：少评价，多理解	33
63.4 前 30 天：交付一个小而完整的闭环	34
63.5 如何和 manager 对齐预期	35
63.6 前 60 天：从执行任务到理解系统	35
63.7 前 90 天：形成可见贡献	36
63.8 新人如何避免“只会问问题”	36
63.9 不要过早推动“大重构”	37
63.10 试用期中的沟通节奏	37
63.11 如何处理试用期负反馈	38
63.12 在新团队建立可信度的底层逻辑	38
63.13 前 90 天的常见错误清单	39
63.14 试用期结束前的复盘材料	39
63.15 入职后的长期成长路线	40
63.16 小结	40
第 64 章：从初级到高级，大模型工程师的长期成长路线	41
64.1 职级不是年限，而是影响范围	41
64.2 初级大模型工程师：把明确任务做好	41
64.3 中级大模型工程师：负责一个模块或链路	42
64.4 高级大模型工程师：定义问题并做取舍	43
64.5 技术负责人：让团队持续变强	44
64.6 成长路线一：应用工程路线	44
64.7 成长路线二：模型与算法工程路线	45
64.8 成长路线三：推理与基础设施路线	46
64.9 成长路线四：评测、安全与治理路线	46
64.10 不同路线可以交叉，但要有主线	47
64.11 如何判断自己是否在成长	47
64.12 晋升材料怎么准备	48
64.13 如何避免职业成长中的常见陷阱	48
64.14 建立自己的技术资产	49
64.15 技术影响力不是写文章那么简单	49
64.16 小结	50
第 65 章：大模型工程师如何持续学习和跟进技术演进	50
65.1 不要把“知道新名词”当成学习	50
65.2 建立四层学习目标	51
65.3 技术雷达：把新技术放进坐标系	52
65.4 判断一项技术是否值得深入	52
65.5 每周学习节奏：少而稳定	53
65.6 如何读论文：先抓问题，再看方法	53
65.7 如何读源码：从运行路径开始	54
65.8 如何做小实验：控制变量	54
65.9 学习要绑定真实项目	55
65.10 面试前如何补齐知识短板	55
65.11 不同方向的学习优先级	56
65.12 如何避免信息流污染	57
65.13 把学习沉淀成可复用资产	57
65.14 学习的最终目标是提高判断力	58
65.15 小结	58
第 66 章：入职后的工程习惯与长期可信度建设	58
66.1 可信度来自稳定交付，而不是一次爆发	58
66.2 接需求时先确认问题，而不是急着给方案	59
66.3 写方案时明确范围、假设和验证方式	59
66.4 进度沟通要暴露不确定性	60

66.5 代码提交要让别人容易 review	60
66.6 日志和 trace 是工程生命线	61
66.7 评测要进入日常开发流程	61
66.8 上线前要检查失败路径	62
66.9 复盘不是追责，而是改系统	62
66.10 文档是长期协作工具	63
66.11 不要过度工程，也不要欠债不还	63
66.12 建立跨角色信任	64
66.13 个人可信度的负面清单	64
66.14 高可信工程师的日常表现	64
66.15 小结	65
第 67 章：把项目经验沉淀成作品、晋升材料和职业资本	65
67.1 项目做完不等于经验沉淀完成	65
67.2 什么是职业资产	66
67.3 每个项目都应该留下项目卡片	66
67.4 记录指标，不要只记录感受	67
67.5 失败方案也是资产	67
67.6 把项目抽象成可迁移能力	68
67.7 晋升材料要讲影响力，不只是工作量	68
67.8 简历项目要讲清边界	69
67.9 面试表达要从故事变成结构	69
67.10 内部分享是放大影响力的方式	70
67.11 开源和公开写作要注意边界	70
67.12 建立个人项目档案库	71
67.13 不同阶段的沉淀重点	71
67.14 一次项目复盘示例	72
67.15 小结	72
第 68 章：从执行者到技术负责人	73
68.1 执行者和技术负责人的区别	73
68.2 技术负责人首先要定义问题	74
68.3 技术负责人要管理技术方向，而不是追热点	74
68.4 建立方向级指标体系	75
68.5 设计架构时要考虑演进，而不是只满足当前需求	75
68.6 技术负责人要控制复杂度	76
68.7 建立评审机制，而不是靠个人经验	76
68.8 技术负责人要会做取舍	77
68.9 推动跨团队协作	77
68.10 技术负责人要培养接班人	77
68.11 如何判断自己是否准备好负责一个方向	78
68.12 常见误区	78
68.13 一个方向负责人的工作样例	79
68.14 从技术负责人到更高层级	79
68.15 小结	79
第 69 章：技术判断与方案取舍	80
69.1 技术判断不是偏好，而是有证据的选择	80
69.2 先判断问题类型，再选择技术手段	80
69.3 换模型不是万能解法	81
69.4 微调、RAG 和 prompt 的取舍	82
69.5 Agent 不是所有自动化的答案	83
69.6 长上下文和 RAG 不是简单替代关系	83
69.7 自建模型服务还是调用 API	84
69.8 开源框架还是自研	84
69.9 Demo、试点和生产系统的标准不同	85

69.10 用小实验降低决策风险	85
69.11 什么时候应该保持简单	86
69.12 什么时候应该增加复杂度	86
69.13 决策文档：让取舍可追溯	87
69.14 技术判断的常见陷阱	87
69.15 小结	88
第 70 章：技术影响力与跨团队沟通	88
70.1 技术影响力不是控制别人	88
70.2 大模型项目为什么沟通更难	89
70.3 先对齐问题，而不是先推方案	89
70.4 用共同语言连接不同角色	90
70.5 用文档建立影响力	91
70.6 方案评审要评审什么	92
70.7 把争论变成实验	92
70.8 沟通技术风险时要具体	93
70.9 面对不同意见时不要急着防守	93
70.10 向上沟通：让管理者理解取舍	94
70.11 横向沟通：让协作团队愿意参与	94
70.12 会议不是越多越好	95
70.13 建立可复用的沟通资产	95
70.14 技术影响力来自持续可信	96
70.15 技术沟通中的常见误区	96
70.16 一个跨团队推进样例	97
70.17 面试表达	97
70.18 小结	97
第 71 章：技术战略、路线图与长期规划	97
71.1 技术战略不是口号	98
71.2 战略从诊断开始	99
71.3 技术愿景和技术战略的区别	99
71.4 路线图不是任务列表	100
71.5 大模型路线图的常见维度	100
71.6 用北极星指标约束路线图	101
71.7 优先级排序：影响、紧急、依赖和可逆性	102
71.8 技术债要翻译成业务风险	103
71.9 什么时候应该平台化	103
71.10 什么时候应该停止一个方向	104
71.11 设计 3 个月、6 个月和 12 个月路线图	104
71.12 路线图要保留调整空间	105
71.13 把路线图变成团队行动	106
71.14 技术战略的面试表达	106
71.15 小结	106
第 72 章：指标、复盘与持续改进	107
72.1 没有指标，就没有方向感	107
72.2 指标不是越多越好	108
72.3 好指标要能驱动行动	109
72.4 警惕指标被优化坏	109
72.5 大模型系统的核心质量闭环	110
72.6 失败 case 分类	110
72.7 复盘不是追责	111
72.8 上线后复盘	111
72.9 事故复盘	112
72.10 指标和复盘如何进入路线图	112
72.11 建立质量节奏	113

72.12 面试中的指标表达	114
72.13 常见误区	114
72.14 小结	114
第 73 章：风险、安全与责任边界	115
73.1 安全不是上线前最后一步	115
73.2 大模型应用的风险地图	116
73.3 Prompt Injection 不是 prompt 写得更强就能解决	116
73.4 权限必须在系统层执行	117
73.5 工具调用的风险边界	117
73.6 Insecure Output Handling: 不要直接信任模型输出	118
73.7 敏感信息和数据边界	119
73.8 Excessive Agency: 不要给模型过度自主权	119
73.9 Overreliance: 不要让用户过度相信模型	120
73.10 成本和可用性攻击	120
73.11 风险分级和上线门禁	121
73.12 责任边界：模型、系统和人的分工	121
73.13 风险治理不是一次性文档	122
73.14 面试表达	122
73.15 常见误区	123
73.16 小结	123
第 74 章：成本、ROI 与商业判断	123
74.1 大模型不是免费能力	124
74.2 Token 使用量不是价值指标	124
74.3 成本要按链路拆开	125
74.4 ROI 的基本公式	126
74.5 不同阶段的 ROI 标准不同	126
74.6 质量提升要和成本一起看	127
74.7 什么时候该用强模型，什么时候该用小模型	127
74.8 RAG、微调、长上下文和 Agent 的成本取舍	128
74.9 降本不是简单砍模型	128
74.10 成本归因：谁在花钱，为什么花钱	129
74.11 成本预算和用量治理	129
74.12 自建还是 API 的商业判断	130
74.13 让工程成为业务驱动者	130
74.14 面试表达	131
74.15 常见误区	131
74.16 小结	131
第 75 章：长期成长与职业护城河	132
75.1 职业护城河不是会更多工具	132
75.2 大模型时代更需要基本功	133
75.3 从执行任务到拥有问题	133
75.4 横向影响力是高级工程师的分水岭	134
75.5 写作是放大器	134
75.6 构建可迁移能力	135
75.7 学会和 AI 工具协作	135
75.8 建立个人技术资产	136
75.9 选择能让你成长的项目	136
75.10 不要把职业押注在单一热点上	137
75.11 职业节奏：探索、聚焦、放大	137
75.12 面试表达	138
75.13 常见误区	138
75.14 小结	139

第 76 章：30-60-90 天成长计划与行动清单	139
76.1 30-60-90 天计划的目的	139
76.2 前 30 天：理解系统和建立关系	140
76.3 前 30 天的具体行动	140
76.4 前 30 天的产出	142
76.5 第 31-60 天：找到切入点并交付闭环	142
76.6 第 31-60 天的具体行动	142
76.7 第 31-60 天的产出	143
76.8 第 61-90 天：扩大影响并建立机制	143
76.9 第 61-90 天的具体行动	144
76.10 第 61-90 天的产出	145
76.11 不同角色的 30-60-90 重点	145
76.12 面试表达	146
76.13 常见误区	146
76.14 小结	146
第 77 章：面试复盘、Offer 选择与谈薪	147
77.1 面试结束不是结束	147
77.2 如何分析自己的面试表现	147
77.3 面试后如何补短板	148
77.4 反向提问是双向面试	149
77.5 大模型岗位的反向提问清单	149
77.6 Offer 选择不要只看薪资	150
77.7 如何识别高风险团队	150
77.8 谈薪的基本原则	151
77.9 职级比 title 更重要	151
77.10 入职前要确认的事项	152
77.11 面试复盘如何变成长期资产	152
77.12 面试表达	152
77.13 常见误区	153
77.14 小结	153
第 78 章：求职材料包、作品集与公开证据	153
78.1 简历是入口，不是全部	154
78.2 大模型简历的项目 bullet 模板	154
78.3 项目一页纸	155
78.4 项目复盘文档	155
78.5 GitHub 仓库如何展示	156
78.6 技术博客写什么	157
78.7 面试材料包如何组织	157
78.8 不同岗位的材料侧重点	158
78.9 公开表达要注意边界	158
78.10 面试表达	159
78.11 常见误区	159
78.12 小结	160
第 79 章：模拟面试训练与刻意练习	160
79.1 模拟面试训练什么	160
79.2 建立题库而不是临时刷题	161
79.3 项目表达训练	161
79.4 系统设计训练	162
79.5 Debug 训练	162
79.6 行为面训练	163
79.7 英语面试训练	163
79.8 一周模拟面试节奏	163
79.9 面试当天状态管理	164

79.10 小结	164
第 80 章：常见失败案例与反面教材	164
80.1 失败一：把 demo 当生产项目讲	165
80.2 失败二：简历堆满技术名词	165
80.3 失败三：没有指标	165
80.4 失败四：只会讲框架，不会讲原理	166
80.5 失败五：忽视安全和权限	166
80.6 失败六：不会讲失败	166
80.7 失败七：面试中硬编经验	167
80.8 失败八：入职后急于证明自己	167
80.9 失败九：只追热点，不沉淀资产	167
80.10 小结	168
第 81 章：完整求职作战路线图	168
81.1 第一阶段：定位目标岗位	168
81.2 第二阶段：补齐核心能力	168
81.3 第三阶段：做一个可讲项目	169
81.4 第四阶段：准备求职材料包	169
81.5 第五阶段：投递和内推	170
81.6 第六阶段：面试准备	170
81.7 第七阶段：面试复盘和迭代	170
81.8 第八阶段：Offer 选择和谈薪	171
81.9 第九阶段：入职前准备	171
81.10 小结	171
第 82 章：最终自检清单与补充篇总结	171
82.1 项目能力自检	172
82.2 技术能力自检	172
82.3 工程能力自检	172
82.4 面试表达自检	173
82.5 求职材料自检	173
82.6 职业判断自检	173
82.7 最后一版求职前检查	173
82.8 补充篇总结	174

第 60 章：面试中的项目表达、作品集与系统设计

上一章讲的是如何准备大模型工程面试：面试官真正想确认的，不只是你是否知道 RAG、Agent、微调、评测这些词，而是你是否能把一个真实问题拆开，判断约束，做出取舍，并把系统稳定地交付出来。

这一章更进一步：当你已经有项目经历、练习作品、开源仓库或课程项目时，面试中应该怎样讲，才能让对方相信你真的做过、想过、踩过坑，而不是只是在简历上堆关键词。

很多候选人并不是没有能力，而是表达方式让能力被低估。他们会说：

我做了一个基于 LangChain 的 RAG 问答系统，使用向量数据库进行检索，接入了 OpenAI API，并实现了多轮对话。

这句话听起来完整，但对面试官来说信息量很低。因为它没有回答这些更关键的问题：

- 这个系统解决了谁的问题？
- 为什么需要 RAG，而不是直接把内容塞进 prompt？
- 文档如何切分？检索效果如何判断？
- 答案错了怎么发现？怎么降低幻觉？
- 延迟、成本、权限、更新频率有什么约束？
- 你在其中负责了什么？做过哪些决策？

面试中的项目表达，本质上不是“报技术栈”，而是“证明工程判断力”。

60.1 项目表达的核心：从技术栈转向决策链

一个成熟的项目讲法，应该围绕一条决策链展开：

问题是什么 -> 约束是什么 -> 方案怎么选 -> 关键实现是什么 -> 怎么验证效果 -> 遇到什么问题 -> 如何改进 -> 最终结果如何

这条链条比任何技术名词都重要。

例如，不建议这样讲：

我用 LangChain、Chroma 和 GPT-4 做了一个知识库问答系统。

可以改成这样：

我做的是一个内部文档问答系统，目标是让新人能快速查询部署流程和故障处理手册。最初我们尝试直接把相关文档塞进 prompt，但文档数量增长后上下文长度和成本都不可控，所以改成 RAG。实现上我把文档按标题层级和段落长度混合切分，保留章节路径作为 metadata，检索时先用向量召回，再按文档类型和更新时间做过滤。评测上我整理了 80 个真实问题，关注命中率、答案可追溯性和拒答能力。后来发现很多错误来自 chunk 边界丢失上下文，所以增加了相邻 chunk 拼接和标题前缀，准确率明显提升。

这段表达有几个特点：

- 有明确业务场景，不是抽象 demo。
- 有方案演进，不是一上来就堆框架。
- 有约束意识，提到了上下文长度、成本、文档增长。
- 有实现细节，讲到了切分、metadata、召回、过滤。
- 有评测意识，不只是“效果不错”。
- 有问题和改进，说明你真的调过系统。

面试官听到这类表达，会更愿意继续追问。因为它显示你不是只会照教程搭一个链路，而是理解系统为什么这样设计。

60.2 面试中讲项目的推荐结构

一个项目可以准备三个版本：30 秒版、2 分钟版、深入追问版。

30 秒版：让对方知道这是什么

30 秒版适合简历开场，目标是快速建立上下文。

模板如下：

这个项目是为了解决【用户/业务问题】。我主要负责【你的职责】。系统采用【核心方案】，关键难点是【一个主要挑战】。最后通过【评测或结果】验证效果。

示例：

这个项目是一个面向客服团队的知识库问答系统，目的是减少人工查询产品文档的时间。我主要负责 RAG 检索链路和回答评测。系统采用文档切分、向量召回、规则过滤和引用溯源的方式，关键难点是文档版本多、问题表达不稳定。最后我们用真实客服问题集做评测，重点看答案命中率和引用正确率。

这个版本不需要讲太细，重点是让面试官知道后面可以追问什么。

2 分钟版：展示完整工程思考

2 分钟版适合面试官说“详细讲讲这个项目”。

可以按下面顺序讲：

1. 背景：为什么要做这个项目。
2. 目标：要优化什么指标或体验。
3. 约束：数据、成本、延迟、权限、准确性有什么限制。
4. 方案：为什么选择当前架构。
5. 实现：你负责的关键模块。
6. 评测：如何判断它是否有效。
7. 问题：上线或测试中遇到什么坑。
8. 改进：你如何定位和解决。

注意，不要平均用力。真正能体现水平的是“约束、取舍、评测、问题定位”。

技术栈只需要带过，除非面试官追问。

深入追问版：准备可展开的细节

深入追问版不是背一段长稿，而是提前准备几个可以展开的细节模块。例如 RAG 项目可以准备：

- 文档解析：PDF、Markdown、网页、表格如何处理。
- 切分策略：固定长度、标题层级、语义切分的取舍。
- 检索策略：向量检索、关键词检索、混合检索、rerank。
- 上下文构造：如何排序、去重、压缩、拼接。
- 幻觉控制：引用、拒答、答案约束、置信度判断。
- 评测方法：测试集来源、指标、人工标注、回归测试。
- 工程问题：缓存、并发、延迟、成本、权限隔离。

Agent 项目可以准备：

- 任务边界：什么任务适合 Agent，什么任务不适合。
- 工具设计：工具参数、返回格式、错误处理。
- 状态管理：短期记忆、长期记忆、任务上下文。
- 规划方式：单步工具调用、多步计划、反思机制。
- 安全控制：权限、确认步骤、危险操作拦截。
- 可观测性：日志、trace、中间步骤记录。
- 失败兜底：重试、降级、人工接管。

微调项目可以准备：

- 数据来源：如何清洗、去重、标注、划分训练集。
- 训练目标：风格对齐、任务能力、格式稳定性还是领域适配。
- 方法选择：prompt、RAG、LoRA、全量微调的取舍。
- 训练配置：模型、学习率、batch size、epoch、上下文长度。
- 评测方式：自动指标、人工评测、线上 A/B、错误分析。
- 风险控制：过拟合、遗忘、数据泄漏、安全输出。

提前准备这些模块的目的，是让你在被迫问时能自然展开，而不是只能重复“我用了某某框架”。

60.3 作品集：不要只放代码，要放证据

大模型方向的作品集，不一定非要很大的项目。一个小而完整的项目，只要证据链充分，也能体现能力。

作品集最常见的问题是：仓库里只有代码，没有说明。面试官打开后不知道这个项目解决什么问题，也不知道怎么运行，更看不到你的思考过程。

一个好的作品集 README 至少应该包含：

- 项目背景：这个项目解决什么问题。
- 核心功能：用户能做什么。
- 架构图或流程图：请求如何流动。
- 快速开始：如何安装、配置、运行。
- 示例输入输出：展示真实效果。
- 关键设计：为什么这样切分、检索、调用模型或设计工具。

- 评测结果：测试集、指标、错误样例。
- 已知限制：当前方案哪些地方还不够好。
- 后续改进：如果继续做，会优先改什么。

这里最重要的是“证据”。

什么是证据？

- 有真实样例，而不是只写“支持智能问答”。
- 有评测表格，而不是只写“准确率较高”。
- 有错误分析，而不是只展示成功 case。
- 有运行说明，而不是只给一堆脚本。
- 有设计取舍，而不是只列依赖包。

例如，一个 RAG 作品集可以包含这样的评测表：

问题类型	样例数量	Top-3 命中率	答案可引用率	主要错误
概念查询	30	90%	87%	相似术语混淆
操作流程	25	84%	80%	chunk 跨步骤断裂
故障排查	20	75%	70%	日志关键词缺失
权限相关	15	82%	78%	版本差异导致回答过期

这样的表格不一定要非常复杂，但它能说明你知道系统哪里强、哪里弱。

相比之下，如果 README 只写：

使用 LangChain + FAISS + Streamlit 实现本地知识库问答。

面试官很难判断你的能力边界。

60.4 项目不大也可以讲深

很多同学会担心：我没有大厂项目，没有线上千万级流量，也没有训练过百亿参数模型，是不是没什么可讲？

其实初中级大模型工程岗位更关注的是：你是否具备把问题做完整的能力。

一个小项目，只要做得闭环，也可以讲得很有价值。

例如，你做了一个“论文问答助手”。这听起来很普通，但如果你深入处理下面的问题，它就不再只是 demo：

- PDF 中标题、公式、表格、参考文献如何解析。
- 长论文如何切分，是否保留章节结构。
- 用户问“这篇论文的核心贡献是什么”时，如何聚合多个段落。
- 用户问“实验结果如何”时，如何定位表格和实验部分。
- 模型回答是否要求引用页码或段落。
- 如何评测摘要类问题和事实类问题。
- 多篇论文之间如何比较。

你不需要每一点都做到完美，但你需要知道这些问题存在，并且能说清楚你当时为什么优先解决其中几项。

面试官更愿意听到：

我这个项目规模不大，但我重点做了 PDF 解析、章节级切分和引用溯源。没有做复杂 Agent，因为任务主要是检索和总结，多步工具调用的收益不明显。后来我发现表格问答效果差，所以把表格单独抽取成 Markdown，并在 metadata 里标记 table 类型，检索时对实验类问题提高表格 chunk 的优先级。

这比泛泛地说“我做了论文助手，支持上传 PDF 和智能问答”强得多。

项目小不是问题，问题是讲不出决策。

60.5 系统设计题：先问清楚边界

大模型工程面试中，系统设计题越来越常见。题目可能是：

- 设计一个企业知识库问答系统。
- 设计一个代码助手。
- 设计一个客服 Agent。
- 设计一个多模态内容审核系统。
- 设计一个面向销售的自动线索分析工具。

很多候选人一听题目，就立刻开始画组件：前端、后端、向量数据库、LLM、缓存、消息队列。这样容易显得像在套模板。

正确做法是先问清楚边界。

以“设计一个企业知识库问答系统”为例，应该先确认：

- 用户是谁：员工、客服、研发、外部客户？
- 数据来源是什么：PDF、网页、飞书/Notion、数据库、工单？
- 数据规模多大：几百篇文档还是千万级文档？
- 更新频率如何：每天更新、实时同步还是手动上传？
- 权限要求如何：是否不同用户只能看不同文档？
- 准确性要求如何：允许概括，还是必须严格引用？
- 延迟要求如何：2 秒内、10 秒内，还是异步生成也可以？
- 成本限制如何：每次查询预算多少？
- 是否需要多轮对话、追问、任务执行？

这些问题不是拖延时间，而是在定义系统。

如果不问权限，可能设计出一个会泄露内部文档的系统。如果不问更新频率，可能索引同步方案完全不合适。如果不问准确性要求，可能不知道是否需要引用和拒答。

系统设计不是“把所有组件都放上去”，而是“在约束下做合理取舍”。

60.6 一个 RAG 系统设计的表达框架

设计 RAG 系统时，可以按六层来讲：

1. 数据接入层：文档从哪里来，如何解析、清洗、去重、权限标记。
2. 索引构建层：如何切分、生成 embedding、写入向量库和关键词索引。
3. 查询理解层：是否需要改写 query、识别意图、识别用户权限。
4. 检索排序层：向量召回、关键词召回、metadata 过滤、rerank。
5. 生成回答层：构造上下文、控制引用、拒答、格式约束。
6. 评测监控层：离线评测、线上反馈、日志、成本、延迟、错误分析。

面试时可以画成一条链路：

数据源 -> 解析清洗 -> chunk + metadata -> embedding/index
用户问题 -> query 改写 -> 权限过滤 -> hybrid retrieval -> rerank
top chunks -> prompt 构造 -> LLM 生成 -> 引用/拒答 -> 日志与评测

但只画链路还不够，必须讲关键取舍。

例如：

- 如果文档规模小，可以先用简单向量检索，不必一开始上复杂分布式架构。
- 如果术语精确匹配很重要，应考虑 BM25 + 向量混合检索。
- 如果权限严格，应在检索前或检索中做权限过滤，而不是生成后再过滤。
- 如果答案必须可审计，应要求模型基于引用回答，并在无证据时拒答。
- 如果文档频繁更新，应设计增量索引和版本管理。
- 如果成本敏感，应控制 top-k、上下文长度、模型级别和缓存策略。

这些取舍比组件名称更能体现系统设计能力。

60.7 Agent 系统设计：重点不是“会调用工具”

Agent 系统设计题经常被讲得过于玄学。很多人会说：

我会让 Agent 自主规划任务，调用多个工具，最后完成目标。

这句话风险很高。因为在真实工程里，Agent 最大的问题不是“能不能调用工具”，而是：

- 工具调用是否可靠。
- 错误是否可恢复。
- 权限是否可控。
- 中间状态是否可观察。
- 用户是否知道 Agent 做了什么。
- 危险操作是否需要确认。
- 失败时是否有兜底方案。

设计客服 Agent 时，可以这样讲：

1. 先把任务分级：只读查询、建议生成、需要修改订单的操作、需要人工审批的操作。
2. 对只读查询，可以让 Agent 自动调用知识库和订单查询工具。
3. 对修改类操作，工具必须有明确 schema，并且执行前展示操作摘要让用户确认。
4. 所有工具调用记录 trace，包括输入、输出、耗时和错误。
5. 如果模型连续两次无法完成，或用户情绪强烈，转人工。
6. 对高风险工具设置权限和白名单，不能让模型自由拼接任意 SQL 或任意 HTTP 请求。

这类表达会让面试官觉得你有生产意识。

Agent 的核心不是“自主”，而是“可控地自动化”。

60.8 被问到不会的问题怎么办

面试中被问到不会的问题很正常。关键是不要假装。

不建议这样回答：

这个我了解过，但具体细节有点忘了。

这句话太空。

更好的方式是你知道的边界说清楚，然后给出推理路径：

我没有在生产环境实现过这个方案，但我理解它要解决的问题是.....如果让我设计，我会先从.....开始验证。这里我不确定的点是.....，需要通过实验或查文档确认。

例如被问到“你如何设计一个 reranker 的训练流程”，如果你没做过，可以说：

我没有完整训练过 reranker，但我理解 reranker 的目标是对召回候选进行更精细排序。数据上需要 query、正样本文档和负样本文档，负样本可以从 BM25 或向量召回的 hard negatives 里采。训练后不能只看 pairwise accuracy，还要看它接到 RAG 后对最终答案引用命中率和延迟的影响。我不确定具体 loss 和模型选型，需要根据 cross-encoder、bi-encoder 或轻量模型的成本要求再定。

这个回答承认了边界，但仍然展示了工程推理能力。

面试官通常不要求你什么都做过，但会观察你面对未知问题时是否诚实、是否能拆解、是否知道如何验证。

60.9 简历上的项目应该怎么写

简历项目描述不要写成技术栈清单，而要写成“动作 + 方法 + 结果”。

弱表达：

基于 LangChain 和 FAISS 实现智能问答系统，支持文档上传、多轮对话和知识库检索。

更好的表达：

设计并实现企业文档 RAG 问答系统，支持 PDF/Markdown 文档解析、章节级切分、向量召回与引用溯源；构建 100 条真实问题评测集，针对 chunk 边界导致的漏召回问题引入标题前缀和相邻片段扩展，使 Top-3 证据命中率从 72% 提升到 86%。

注意，这里有几个关键词：

- 设计并实现：说明你的动作。
- PDF/Markdown、章节级切分、引用溯源：说明关键方法。
- 100 条真实问题评测集：说明评测基础。
- Top-3 证据命中率从 72% 到 86%：说明结果。

如果没有真实线上指标，也可以写离线评测结果。但不要编造数据。可以写：

基于 60 个手工整理的问题进行离线评测，记录召回失败、引用错误和拒答失败三类问题，并据此调整切分策略和 prompt 约束。

这仍然比“效果良好”强很多。

60.10 常见扣分表达

下面这些表达在面试中很容易扣分：

- “这个很简单，就是调一下 API。”
- “我用了 LangChain，所以支持 Agent。”
- “向量数据库可以解决幻觉问题。”
- “准确率挺高的，但没有具体测。”
- “这个是模型自己生成的，我也不知道为什么。”
- “我们把所有文档都塞进 prompt。”
- “权限可以在回答后再检查。”
- “失败了就重新调用一次。”
- “微调之后肯定比 prompt 好。”
- “Agent 可以自己决定调用什么工具，所以不用写太多规则。”

这些话的问题不在于完全错误，而在于暴露出工程边界感不足。

更好的表达方式是：

- API 调用只是最外层，难点在数据、评测、稳定性和成本控制。
- LangChain 是编排工具，不等于系统能力本身。
- RAG 可以提供外部证据，但不能自动消除幻觉。
- 没有评测就不能判断效果，只能说主观体验。
- 模型输出不可完全解释，所以需要日志、引用、约束和回归测试。
- 上下文窗口再大，也需要考虑成本、噪声和信息选择。
- 权限必须尽量在数据访问阶段控制，不能依赖生成后补救。
- 重试要区分错误类型，否则可能放大成本和副作用。
- 微调、RAG、prompt 是不同手段，要看问题类型和数据条件。
- Agent 越自由，越需要边界、权限和可观测性。

60.11 面试前的项目复盘清单

每个写在简历上的大模型项目，都建议按下面清单复盘一遍：

- 一句话说明这个项目解决什么问题。
- 目标用户是谁。
- 输入和输出是什么。
- 为什么需要大模型，传统方法是否可行。
- 为什么选择 RAG、Agent、微调或 prompt engineering。
- 数据从哪里来，质量如何。
- 核心模块有哪些，你负责哪些。
- 最关键的技术难点是什么。

- 做过哪些取舍，放弃过哪些方案。
- 如何评测效果，指标是什么。
- 最典型的失败 case 是什么。
- 你如何定位和改进这些失败 case。
- 系统的成本、延迟和稳定性如何。
- 是否涉及权限、安全、隐私。
- 如果重做一遍，会优先改什么。

如果这些问题你都能回答，项目就很扎实。即使项目规模不大，也能讲出深度。

如果很多问题答不上来，不一定要删掉项目，而是应该回去补证据：加评测集、补 README、记录错误样例、画架构图、整理设计取舍。

60.12 小结

大模型工程面试中，项目表达的目标不是证明你知道多少名词，而是证明你能把一个不确定的问题做成一个可运行、可评测、可维护的系统。

讲项目时，要从技术栈转向决策链：问题、约束、方案、实现、评测、问题、改进、结果。

做作品集时，不要只放代码，要放证据：README、样例、评测、错误分析、已知限制和后续改进。

做系统设计时，不要急着堆组件，要先问清楚用户、数据、权限、准确性、延迟、成本和更新频率。

面对不会的问题，不要假装会，要诚实说明边界，并展示你的拆解和验证思路。

真正能打动面试官的，不是“我用过某某框架”，而是：

我知道这个系统为什么这样设计，知道它哪里可能失败，也知道应该如何验证和改进。

这就是大模型工程师最重要的面试表达能力。

第 61 章：大模型工程面试中的高频问题与现场推理

上一章讲的是项目、作品集和系统设计的表达方式。那一章的重点是：不要只说你用了什么技术，而要说清楚你面对什么问题、做了什么取舍、如何验证结果。

这一章进入更具体的面试现场。

大模型工程面试里，很多问题看起来是在考知识点，例如：

- RAG 为什么会产生幻觉？
- chunk size 怎么选？
- embedding 模型怎么评估？
- Agent 为什么不稳定？
- 微调和 RAG 怎么选？
- 如何降低推理成本？
- 如何设计一个大模型应用的评测体系？

但真正优秀的回答，不是背定义，而是能把问题拆成工程条件、风险来源和验证方法。

面试官问“chunk size 怎么选”，不是想听一个固定数字。面试官真正想知道的是：你是否理解文档结构、检索召回、上下文噪声、生成质量、成本延迟之间的关系。

这一章会围绕高频问题展开，重点不是给出唯一标准答案，而是训练一种回答方式：先定义问题，再分析约束，再给出方案和验证路径。

61.1 回答技术问题的基本结构

大模型工程面试中的技术问题，可以用一个通用结构回答：

1. 先说明问题本质。

2. 再拆影响因素。
3. 给出常见方案。
4. 说明取舍和适用场景。
5. 最后讲如何验证。

例如面试官问：

RAG 为什么还能产生幻觉？

弱回答是：

因为大模型本身会幻觉，所以需要更好的 prompt。

这个回答太泛。

更好的回答是：

RAG 只是给模型提供外部证据，但不能保证证据一定被正确检索、正确选择和正确使用。幻觉可能来自几个环节：第一是召回失败，相关文档没有被取到；第二是召回结果有噪声，模型被无关内容干扰；第三是文档本身过期或冲突；第四是 prompt 没有限制模型必须基于证据回答；第五是用户问题超出知识库覆盖范围但系统没有拒答。改进上可以分别从 hybrid retrieval、rerank、metadata 过滤、引用约束、拒答策略和离线评测入手。验证时不能只看主观效果，应该构造问题集，分别统计证据命中率、引用正确率和无答案问题的拒答率。

这个回答的好处是：它不是一个点，而是一条链路。

面试中大部分问题都可以这样回答。

61.2 高频问题：RAG 和长上下文怎么选

常见问题：

现在模型上下文窗口越来越长，还需要 RAG 吗？

不要直接回答“需要”或“不需要”。应该先说明两者解决的问题不同。

可以这样回答：

长上下文降低了把更多材料放进 prompt 的难度，但没有完全替代 RAG。RAG 不只是为了解决上下文长度，它还解决知识选择、权限控制、数据更新、成本控制和引用溯源问题。如果文档规模小、问题集中、延迟和成本不敏感，长上下文直接塞材料可能足够简单有效。但如果知识库规模大、文档经常更新、用户权限不同、需要引用来源，还是需要检索和索引层。

可以进一步展开：

维度	长上下文	RAG
实现复杂度	较低	较高
大规模文档	成本和噪声较高	更适合
权限控制	需要额外处理	可在检索阶段过滤
数据更新	每次构造上下文	可增量索引
引用溯源	可以做但不天然	更容易
延迟成本	输入越长越贵	可控制 top-k

总结时可以说：

我会把长上下文看成一种增强能力，而不是 RAG 的替代品。真实系统里经常是组合使用：先通过检索选出候选证据，再利用较长上下文放入更多相邻片段、表格或原文内容。

这个回答体现了你没有陷入“新模型淘汰旧架构”的简单判断。

61.3 高频问题：chunk size 怎么选

这是 RAG 面试中非常常见的问题。

弱回答是：

一般设成 500 或 1000 tokens。

这个回答没有错，但没有工程含义。

更好的回答是：

chunk size 没有固定值，要看文档结构、问题类型和模型上下文预算。chunk 太小，容易丢上下文，召回到的片段不足以回答问题；chunk 太大，embedding 表达会变得混杂，检索噪声增加，也会占用更多 prompt token。实践中我会先按文档结构切，比如标题、段落、列表、表格，再设置一个 token 上限。如果文档结构弱，再用固定长度加 overlap 兜底。最后用评测集观察召回命中率、答案引用率和上下文噪声。

可以补充几个具体判断：

- FAQ、短知识条目：chunk 可以小一些，因为一个问题通常对应一个短答案。
- 操作手册：需要保留步骤完整性，chunk 不能把连续步骤切断。
- 法务、合同、政策文档：要保留条款编号、标题路径和上下文。
- 论文、技术文档：可以按章节层级切分，并把标题路径加入 chunk。
- 表格：不要简单按文本长度切，最好整体保留或转成结构化 Markdown。

如果面试官继续问 overlap，可以回答：

overlap 是为了缓解边界信息丢失，但不是越大越好。overlap 太大会增加索引规模和重复召回。更好的方式是优先利用结构切分；如果必须固定长度切分，再设置适度 overlap，并在检索后做去重或相邻片段扩展。

最后要落到验证：

我不会只凭经验定 chunk size，而是会准备一批真实问题，对不同切分策略跑离线评测，看 Top-k 证据命中、最终答案正确率、引用正确率、平均上下文长度和延迟成本。

这个回答比一个数字更有说服力。

61.4 高频问题：向量检索、关键词检索和混合检索怎么选

常见问题：

既然有 embedding，为什么还需要 BM25？

可以这样回答：

向量检索擅长语义相似，适合用户表达和文档表达不完全一致的情况；关键词检索擅长精确匹配，适合产品型号、错误码、函数名、配置项、法律条款编号这类信息。很多企业知识库里有自然语言问题，也有大量精确术语，所以只用向量检索容易漏掉关键词，只用 BM25 又可能漏掉语义改写问题。混合检索通常更稳。

接着讲实现方式：

- 简单方案：分别跑向量检索和 BM25，合并候选后去重，再按规则或分数融合排序。
- 更稳方案：召回阶段扩大候选集，再用 reranker 对候选做统一排序。
- 领域方案：根据 query 类型选择检索方式，例如包含错误码、版本号、API 名称时提高关键词检索权重。

还可以补充风险：

混合检索不是简单把两个结果拼起来。需要处理分数不可比、重复文档、权限过滤、召回数量和延迟问题。如果有严格权限要求，metadata 过滤要在召回阶段就参与，而不是等生成后再过滤。

最后讲评测：

我会分别统计不同类型下的召回表现，比如术语型、概念型、流程型、故障型。这样才能知道向量、BM25、hybrid 分别解决了什么问题，而不是只看整体平均分。

61.5 高频问题：embedding 模型怎么评估

常见问题：

怎么判断一个 embedding 模型适不适合你的业务？

不要只回答“看 MTEB 排名”。排行榜有参考价值，但不能替代业务评测。

可以这样回答：

我会先看通用 benchmark、语言支持、上下文长度、维度、成本和推理速度，但最终还是要用业务数据评测。embedding 模型的目标不是向量本身好看，而是在具体检索任务中把正确证据排到前面。

业务评测可以这样做：

1. 收集真实或模拟 query。
2. 为每个 query 标注相关文档或相关 chunk。
3. 对不同 embedding 模型构建索引。
4. 统计 Recall@k、MRR、nDCG 等检索指标。
5. 接入生成链路后，再看最终答案正确率和引用正确率。

常见判断维度包括：

- 中文、英文或中英混合效果。
- 领域术语和缩写的识别能力。
- 长文本 embedding 是否稳定。
- 对相似但不同概念的区分能力。
- 向量维度对存储和检索成本的影响。
- 在线推理延迟和批量构建索引成本。

可以补一句很重要的话：

embedding 模型不是越大越好，维度和效果也不是唯一指标。对于生产系统，还要看索引规模、召回延迟、更新成本和稳定性。

这能体现你把模型选择放在系统里考虑。

61.6 高频问题：rerank 是否一定需要

常见问题：

RAG 系统是否一定要加 reranker？

标准回答不是“一定要”。

可以这样说：

reranker 的价值是对初始召回候选做更精细的相关性排序，尤其在 top-k 里混有噪声、query 和文档需要深层匹配时有帮助。但它会增加延迟和成本，所以是否需要取决于业务要求和召回质量。

适合加 reranker 的场景：

- 文档规模大，召回候选噪声多。
- 问题和文档表达差异大。
- 答案对证据排序敏感。
- top-k 召回有相关内容，但经常排在后面。
- 混合检索后需要统一排序。

不一定需要的场景：

- 知识库很小，召回已经足够准。
- 延迟要求极高。

- 问题形式固定，可以用规则或 metadata 过滤解决大部分排序问题。
- 成本预算很紧。

评估方式：

我会先看不加 rerank 时的错误。如果正确 chunk 根本没召回，rerank 解决不了；如果正确 chunk 在候选里但排序靠后，rerank 才有明显价值。评测时要比较加 rerank 前后的 Recall@k、MRR、最终答案正确率、引用正确率、P95 延迟和单次成本。

这句话很关键：rerank 不是召回失败的万能药。

61.7 高频问题：如何降低大模型应用成本

常见问题：

一个 LLM 应用成本太高，怎么优化？

不要只说“换便宜模型”。成本优化要分层。

可以按链路拆：

1. 输入侧：减少无效上下文，压缩 prompt，控制 history 长度。
2. 检索侧：优化 top-k，减少重复 chunk，缓存检索结果。
3. 模型侧：根据任务难度做模型路由，简单问题用小模型，复杂问题用强模型。
4. 输出侧：限制最大输出长度，避免无意义长回答。
5. 缓存侧：对高频问题、稳定知识和中间结果做缓存。
6. 架构侧：批处理、异步任务、流式输出、限流和降级。

可以举一个模型路由例子：

对 FAQ 类问题，可以先用规则、检索或小模型判断是否能回答；只有需要复杂推理、总结或多步骤工具调用时才进入大模型。这样比所有请求都打到最贵模型更合理。

还要提醒风险：

成本优化不能只看平均 token。要同时看质量下降、延迟变化和失败重试带来的隐性成本。有些优化减少了单次调用成本，但增加了错误率和重试次数，整体未必更便宜。

验证指标可以包括：

- 单请求平均成本。
- P50/P95 延迟。
- 输出 token 分布。
- 缓存命中率。
- 不同模型路由比例。
- 答案正确率和用户满意度。
- 重试率和失败率。

成本优化的成熟回答，一定要把质量一起考虑。

61.8 高频问题：如何设计大模型应用评测体系

这是非常重要的问题。

弱回答是：

用人工打分，或者让 GPT 当裁判。

更好的回答是先拆评测对象：

大模型应用评测不能只评最终回答。要按链路拆开看：检索是否拿到证据，prompt 是否约束清楚，模型回答是否正确，引用是否准确，无答案时是否拒答，工具调用是否成功，延迟和成本是否达标。

对于 RAG，可以设计这些指标：

- 检索层：Recall@k、MRR、相关 chunk 命中率。
- 生成层：答案正确性、完整性、引用正确率、格式遵循率。
- 安全层：无答案拒答率、越权问题拦截率、敏感信息泄露率。
- 体验层：延迟、稳定性、用户反馈。
- 成本层：平均 token、模型调用次数、单请求成本。

对于 Agent，可以设计这些指标：

- 任务成功率。
- 工具调用准确率。
- 参数填写正确率。
- 多步任务完成率。
- 错误恢复率。
- 危险操作拦截率。
- 人工接管率。

评测集来源可以包括：

- 历史用户问题。
- 客服工单或搜索日志。
- 产品文档中的典型任务。
- 人工构造的边界 case。
- 上线后的失败样例回流。

还要说明自动评测的边界：

LLM-as-judge 可以提高评测效率，但不能完全替代人工标注。尤其是事实正确性、引用准确性、权限安全这类问题，要有可校验标准或人工抽检。更稳的做法是规则评测、标准答案、LLM judge 和人工复核结合。

最后讲回归：

每次改 embedding、chunk、prompt、模型或 reranker，都应该跑同一套回归集，避免某类问题提升但另一类问题退化。

这类回答会让面试官看到你有持续迭代意识。

61.9 高频问题：微调、RAG 和 prompt engineering 怎么选

常见问题：

一个业务效果不好，你会选择 prompt、RAG 还是微调？

可以这样回答：

我会先判断失败原因。prompt、RAG 和微调解决的问题不同，不能把它们当成同一种优化手段。

可以按问题类型区分：

问题类型	优先方案
指令不清、格式不稳	prompt engineering
缺少外部知识或知识更新频繁	RAG
需要特定风格、格式、领域表达习惯	微调
需要调用工具或执行动作	Agent / tool use
模型基础能力不足	换模型或微调
数据权限和可追溯性要求高	RAG + 权限控制 + 引用

可以举例：

如果客服问答答错是因为产品文档没有进入上下文，微调不是第一选择，应该先做 RAG。如果答案有证据但格式总是不符合要求，可以先优化 prompt 或做轻量格式微调。如果模型需要稳定输出某种业务 JSON，并且有大量高质量样本，微调可能更合适。

还要说明微调的风险：

- 数据质量差会把错误模式学进去。
- 微调不适合记忆频繁变化的知识。
- 微调后仍需要评测和安全控制。
- 微调可能带来过拟合、遗忘和维护成本。

一个成熟的总结是：

我会先用最简单可验证的方式解决问题。prompt 成本最低，RAG 适合外部知识，微调适合稳定行为和风格。最终选择要看错误来源、数据条件、更新频率、可追溯性和成本。

61.10 高频问题：Agent 为什么不稳定

常见问题：

Agent 系统为什么经常不稳定？怎么改进？

可以先说明 Agent 的不稳定来自多个环节：

- 任务目标不清，模型不知道何时结束。
- 工具描述不清，模型选错工具。
- 工具 schema 设计差，参数容易填错。
- 工具返回信息太长或不结构化，模型难以继续推理。
- 多步链路中前一步错误会传播。
- 没有权限边界，模型可能尝试危险操作。
- 缺少 trace，失败后无法定位。
- 没有兜底策略，模型会在错误路径上反复尝试。

改进方法也要对应链路：

- 明确任务边界，把复杂任务拆成可控步骤。
- 工具设计使用清晰 schema，参数命名和描述要具体。
- 工具返回结构化结果，避免返回大段无关文本。
- 对高风险操作加入用户确认或人工审批。
- 设置最大步数、超时、重试策略和失败退出条件。
- 记录每一步 tool call、输入、输出、错误和耗时。
- 对常见任务使用固定 workflow，而不是完全自由规划。

可以补充一句：

生产里的 Agent 不应该追求无限自主，而应该追求在明确边界内可靠完成任务。越高风险的动作，越需要规则、权限和确认机制。

这句话通常很加分。

61.11 高频问题：如何处理模型输出不稳定

常见问题：

同一个问题模型每次回答不一样，怎么办？

可以先区分“不稳定”的类型：

- 内容事实不一致。
- 输出格式不一致。
- 引用来源不一致。
- 工具调用路径不一致。
- 长度和风格不一致。

不同问题对应不同处理方式。

如果是事实不一致：

- 增强检索证据。
- 要求基于引用回答。
- 降低生成自由度。
- 对无证据问题拒答。

如果是格式不一致：

- 使用更明确的 schema。
- 使用结构化输出或 constrained decoding。
- 增加格式校验和重试。
- 对稳定格式任务考虑微调。

如果是工具调用不一致：

- 收紧工具选择空间。
- 优化工具描述和参数 schema。
- 对固定流程使用 workflow。
- 增加中间状态检查。

还可以说：

temperature 降低可以减少随机性，但它不是解决所有不稳定的办法。如果上下文噪声大、证据冲突、prompt 模糊，即使 temperature 很低也可能输出错误。

这能避免把问题简化成一个采样参数。

61.12 高频问题：如何做线上监控和故障排查

常见问题：

一个 LLM 应用上线后，你会监控什么？

可以按四类指标回答：

1. 质量指标：答案正确率、用户反馈、拒答率、引用正确率、任务成功率。
2. 性能指标：TTFT、总延迟、P95/P99、吞吐、排队时间。
3. 成本指标：输入 token、输出 token、模型调用次数、缓存命中率、单请求成本。
4. 稳定性指标：错误率、超时率、重试率、工具失败率、检索失败率。

对 RAG，还要记录：

- query。
- 检索到的 chunk id。
- chunk 分数。
- rerank 结果。
- 最终进入 prompt 的上下文。
- 答案引用。

对 Agent，还要记录：

- 每一步工具调用。
- 工具参数。
- 工具返回。
- 中间 reasoning 摘要或 trace。
- 失败位置。
- 是否人工接管。

故障排查时可以按链路定位：

如果回答错了，我会先看正确证据有没有被召回；如果没召回，是索引、切分、embedding、query 改写或权限过滤问题；如果召回了但没进入上下文，是排序或上下文构造问题；如果进入上下文但回答错，是 prompt、模型能力或证据冲突问题。

这条定位链非常实用，也很适合面试表达。

61.13 现场推理题：不要急着给结论

有些面试官会给一个开放题：

我们的知识库问答系统最近用户反馈准确率下降，你怎么排查？

这类题目考的是现场推理。不要一上来就说“换 embedding”或“加 rerank”。

可以按下面方式回答：

1. 先确认现象：是所有问题下降，还是某类问题下降。
2. 确认时间点：是否和模型、prompt、索引、文档、权限、流量变化有关。
3. 分层排查：检索、排序、上下文、生成、权限、评测口径。
4. 对比样例：拿成功和失败 case 做差异分析。
5. 临时止血：必要时回滚配置、降级模型或加人工兜底。
6. 长期修复：补评测集、监控和回归流程。

可以给出一个完整回答：

我会先看准确率下降是否集中在某些文档类型、某些用户群或某个时间段。如果是最近开始下降，会检查是否有索引更新、embedding 模型切换、prompt 修改、文档批量更新或权限规则变化。然后抽样失败 case，看正确证据是否被召回。如果没有召回，排查文档解析、chunk、embedding、query 改写和过滤；如果召回了但没排到前面，排查 rerank 或排序融合；如果证据进入 prompt 但模型答错，检查 prompt 约束、证据冲突和模型版本。线上先通过回滚最近变更或降级到稳定链路止血，同时把失败 case 加入回归集。

这类回答的关键是：先定位，再优化。

现场推理题最忌讳的是直接开药方。

61.14 面试回答中的“边界感”

大模型工程面试很看重边界感。

所谓边界感，就是你知道一个技术能解决什么，不能解决什么。

例如：

- RAG 能提供外部证据，但不能保证模型必然正确使用证据。
- 微调能改变模型行为模式，但不适合承载频繁变化的事实知识。
- Agent 能自动化多步任务，但越自由越需要权限和可观测性。
- 长上下文能容纳更多信息，但不等于信息选择问题消失。
- rerank 能改善排序，但不能召回根本没有进入候选集的证据。
- prompt 能快速修复一些行为问题，但不能替代数据质量和系统设计。
- LLM-as-judge 能提高评测效率，但不能完全替代可验证标准和人工抽检。

面试中可以多使用这种句式：

这个方法适合.....但不适合.....如果约束是.....我会优先.....如果约束变成.....我会改成.....

这类表达比绝对化判断更成熟。

61.15 小结

大模型工程面试中的高频问题，很少只有一个标准答案。

面试官真正想看的，是你能不能把一个技术问题放回系统链路里分析：问题在哪里，影响因素是什么，方案有什么取舍，如何验证效果，失败后如何定位。

回答问题时，不要只背概念。更好的结构是：

- 先说明问题本质。
- 再拆影响因素。
- 给出可选方案。
- 说明适用场景和风险。
- 最后落到评测和验证。

如果你能用这种方式回答 RAG、Agent、微调、评测、成本、监控和系统设计问题，面试官会更容易相信你具备真实工程能力。

大模型工程师不是只会调用模型的人，而是能让模型在真实约束下稳定产生价值的人。

第 62 章：行为面试、面后复盘与 Offer 决策

前两章分别讲了项目表达、系统设计和技术问答。它们解决的是“你会不会做”和“你能不能讲清楚”。

但真正的面试流程不只包含技术题。

大模型工程师面试中，行为面试、团队协作问题、反问环节、面后复盘、薪资沟通和 offer 决策同样重要。很多候选人技术能力不差，但最后输在这些环节：

- 项目讲得很强，但协作故事很弱。
- 技术问题能答，但一问冲突处理就空泛。
- 面试结束没有反问，显得对岗位不够认真。
- 面后不复盘，下一轮继续犯同样错误。
- 拿到 offer 后只看薪资数字，没有判断团队、成长和 risk。

这一章不教套路话术，而是把求职后半段当成一个工程问题：收集信号、降低不确定性、做出更好的职业决策。

62.1 行为面试考的不是“性格好不好”

行为面试常见问题包括：

- 讲一次你解决复杂问题的经历。
- 讲一次你和同事意见不一致的经历。
- 讲一次你失败或踩坑的经历。
- 讲一次你在时间很紧的情况下完成任务的经历。
- 讲一次你推动别人接受你方案的经历。
- 讲一次你处理模糊需求的经历。

很多候选人会误以为行为面试只是考沟通能力、性格或价值观。其实对工程岗位来说，它主要考察四件事：

1. 你是否真的经历过复杂项目。
2. 你遇到问题时如何判断优先级。
3. 你如何和产品、算法、平台、业务团队协作。
4. 你是否能从失败中复盘并改变做法。

对大模型工程师尤其如此。

大模型项目的不确定性比传统工程项目更强。需求可能模糊，模型输出可能不稳定，评测口径可能不清楚，业务方可能只说“效果不好”，基础设施可能受限，成本可能突然超预算。

因此，行为面试不是技术面试的附属品，而是在判断你能否在真实组织中把事情推进下去。

62.2 STAR 方法要用，但不要机械套模板

行为面试常用 STAR 方法：

- Situation：当时是什么背景。
- Task：你的目标或职责是什么。
- Action：你具体做了什么。
- Result：结果是什么。

这个结构有用，因为它能避免回答散乱。

但机械套 STAR 也会很假。例如：

Situation 是我们要做一个 RAG 系统，Task 是我要提升准确率，Action 是我优化了 chunk 和 prompt，Result 是效果提升了。

这听起来像在填表，不像真实经历。

更好的回答要在 STAR 基础上补两个东西：冲突和证据。

冲突是指当时的困难在哪里：

- 数据质量差。
- 需求变化快。
- 指标不清楚。
- 线上延迟不达标。
- 算法效果和工程成本冲突。
- 业务方想要“全能 Agent”，但风险不可控。

证据是指你做了什么可以被验证：

- 构建了多少条评测样例。
- 召回指标从多少到多少。
- P95 延迟下降了多少。
- 成本降低了多少。
- 失败 case 被归因成哪些类型。
- 上线后监控到了什么变化。

一个更扎实的回答可以是：

当时我们做企业知识库问答，业务方反馈“答案不准”，但没有明确指标。我负责排查效果问题。第一步我没有直接改 prompt，而是从线上日志里抽了 80 个失败问题，按召回失败、证据排序错误、引用错误、知识库缺失和无答案未拒答五类标注。结果发现最大问题不是生成，而是 PDF 解析后标题层级丢失，导致切出来的 chunk 缺少上下文。后来我把章节标题路径写入 metadata 和正文前缀，并对连续条款做相邻片段扩展。离线评测里 Top-3 证据命中率从 68% 提升到 82%，引用错误也明显下降。这个经历让我意识到，大模型应用的问题不能只靠调 prompt，要先把失败链路拆开。

这个回答有背景、有任务、有动作、有结果，也有工程判断。

62.3 准备行为面试：不要背答案，要准备素材库

行为面试不能靠临场编。临场编出来的故事通常有三个问题：

- 细节经不起追问。
- 结果没有证据。
- 和岗位要求不匹配。

更好的准备方式是建立一个“素材库”。每个素材不是一篇完整作文，而是一段真实经历，可以适配多个问题。

建议至少准备下面八类素材：

素材类型	可覆盖的问题
复杂技术问题	解决难题、技术深度、排障能力
项目从 0 到 1	主动性、系统设计、落地能力
线上事故或失败	复盘、责任感、稳定性意识
跨团队协作	沟通、推动、冲突处理

素材类型	可覆盖的问题
指标提升	结果导向、实验能力、数据意识
需求模糊	产品理解、澄清问题、定义边界
时间压力	优先级、取舍、交付能力
学习新领域	成长性、自驱力、研究能力

对大模型岗位，素材最好来自真实项目链路。例如：

- 从日志中定位 RAG 失败原因。
- 把一个 demo 改造成可上线服务。
- 给 Agent 工具调用加权限、trace 和确认机制。
- 为模型应用建立离线评测集。
- 在成本超预算时设计模型路由。
- 与业务方一起定义“准确率”到底是什么。

每个素材建议提前写清楚六个点：

1. 背景是什么。
2. 当时最难的约束是什么。
3. 你负责哪一部分。
4. 你做了哪些关键动作。
5. 结果如何验证。
6. 你学到了什么，后来如何改变做法。

注意最后一点。

很多候选人讲完结果就结束了。但面试官常常更关心：你从这件事中形成了什么更通用的方法。

例如你可以说：

后来我做大模型应用时，会先建立失败样例表和最小评测集，再开始调 prompt 或换模型。否则很容易陷入主观试错。

这句话能把一次经历上升为可迁移能力。

62.4 如何回答“讲一次失败经历”

“讲一次失败经历”是行为面试高频题。

不建议回答：

我最大的失败是太追求完美，经常把事情做得太细。

这类回答像变相夸自己，可信度很低。

也不建议讲一个完全失控、没有复盘、没有改进的失败。

好的失败经历应该满足三个条件：

1. 失败真实存在。
2. 后果可控，不涉及严重诚信或责任问题。
3. 你能讲清楚后来如何改进。

大模型工程里可以讲的失败包括：

- 一开始只看主观 demo 效果，没有建立评测集，导致上线后某类问题退化。
- Agent 工具返回没有结构化，导致模型多步任务中间状态混乱。
- 只优化平均延迟，没有关注 P95，导致高峰期用户体验差。
- RAG 切分策略没有保留标题路径，导致政策类问答引用错误。
- 没有提前和业务方定义拒答边界，导致模型对知识库外问题胡答。

回答结构可以是：

当时我做了什么判断，这个判断为什么看起来合理；后来出现了什么问题；我怎么定位原因；我采取了什么补救；这件事改变了我后续做项目的方法。

示例：

我之前做过一个内部文档问答 demo，早期主要靠人工试问来判断效果。demo 阶段看起来不错，但接入更多文档后，用户反馈政策类问题经常答得不完整。复盘后发现，我们没有固定评测集，也没有区分召回错误和生成错误，所以每次改 prompt 都只是凭感觉。我后来补了 120 条问题，按文档类型和问题类型分组，记录证据命中、引用正确和拒答情况。这个过程发现主要问题是 chunk 切分把条款和标题分开了，不是模型本身不会答。修复后我也形成了一个习惯：只要是大模型应用，哪怕早期 demo，也要尽快建立小规模但稳定的回归集。

这个回答承认失败，但重点是复盘和方法升级。

62.5 如何回答“你和别人意见不一致怎么办”

这类问题不是考你是否“听话”，也不是考你是否“强势”。

它考的是你能否在不确定问题上推动理性决策。

大模型项目里经常有意见分歧：

- 产品希望答案更“像人”，工程希望答案更可控。
- 业务希望 Agent 自动执行操作，安全团队要求人工确认。
- 算法希望上更强模型，平台团队担心成本和延迟。
- 有人希望微调，有人认为 RAG 更合适。
- 有人想快速上线，有人认为评测还不够。

回答时不要把自己包装成“永远正确的人”。更好的表达是：

1. 先确认目标是否一致。
2. 把分歧转成可比较的维度。
3. 用实验或数据缩小争论。
4. 对不可验证的风险设置边界和回滚方案。
5. 最后尊重决策，但保留复盘机制。

示例：

有一次业务方希望客服 Agent 能自动修改订单，因为这样体验最好。但我担心模型误操作会带来风险。我没有直接否定，而是把任务分成只读查询、建议生成、低风险修改和高风险修改四类。只读查询可以自动执行；低风险修改需要展示摘要让用户确认；高风险修改必须人工审批。我们用这个分级方案既保留了自动化体验，也避免了模型直接执行危险操作。后来线上 trace 显示，大部分请求其实停留在只读和建议层，真正需要修改订单的比例不高，所以这个方案成本也可接受。

这个回答体现的是合作，而不是争输赢。

62.6 如何回答“为什么离开上一家公司”

这个问题要谨慎。

不要抱怨前公司、前老板或同事。即使你的不满是真实的，面试中也不适合展开负面情绪。

好的回答应该满足三点：

1. 不攻击任何人。
2. 解释职业目标变化。
3. 连接到当前岗位。

例如：

我在上一段经历中主要负责传统后端和一部分模型应用接入，这段经历让我补齐了工程交付能力。但我发现自己更想深入做大模型应用的完整链路，包括评测、RAG、Agent、推理成本和线上稳定性。当前这个岗位的方向更接近我接下来想长期积累的能力，所以我希望切换到更聚焦的大模型工程团队。

如果是因为成长空间有限，可以说：

上一段经历给了我很多从 0 到 1 的机会，但后续项目规模和技术复杂度相对稳定。我希望进入一个对模型效果、系统可靠性和业务规模要求更高的环境，继续提升自己在大模型工程化方面的能力。

如果是因为组织调整，可以说：

公司方向调整后，我负责的项目和我长期想做的大模型工程方向不再完全一致。所以我开始寻找更匹配的机会。

核心原则是：把原因讲成职业匹配问题，而不是情绪问题。

62.7 如何回答“你的短板是什么”

“短板”问题不能回答得太虚，也不能暴露岗位核心风险。

不建议说：

- 我太追求完美。
- 我有时候工作太投入。
- 我没有什么明显短板。

也不建议说：

- 我不太会写代码。
- 我不喜欢和人沟通。
- 我做事不太细。

对大模型工程岗位，比较合适的短板应该是可改进、非致命、且有行动计划的。

例如：

我之前更偏应用工程，对底层推理框架的 CUDA kernel 级优化经验还不深。最近我在补 vLLM、PagedAttention、continuous batching 和 KV cache 管理相关内容，也会通过压测实验理解不同参数对吞吐和延迟的影响。短期内我不把自己包装成底层内核专家，但我能和 infra 同学有效协作，并且能从应用侧提出明确的性能需求和验证指标。

这个回答有边界，也有成长路径。

再如：

我过去有一段时间比较依赖人工体验来判断模型效果，后来意识到这在大模型应用里不够可靠。现在我会尽量把失败 case 沉淀成评测集，用固定指标做回归。这是我已经在改的地方。

短板问题的重点不是“包装缺点”，而是证明你有自我校准能力。

62.8 反问环节：不要只问福利和流程

面试最后通常会问：

你有什么问题想问我？

这个环节非常重要。

它不只是礼貌流程，而是你收集岗位信息、判断团队质量、展示思考深度的机会。

不建议只问：

- 加班多吗？
- 薪资范围是多少？
- 什么时候给结果？
- 有没有年终奖？

这些问题不是不能问，但更适合 HR 环节或 offer 阶段。

技术面和 hiring manager 面更适合问这些问题：

1. 这个岗位当前最重要的业务目标是什么？
2. 团队现在的大模型应用主要卡在哪些问题上？
3. 你们如何评估 RAG、Agent 或模型应用的效果？
4. 线上系统目前更关注质量、成本、延迟还是规模？
5. 团队里算法、工程、产品、平台之间如何分工？
6. 新人入职前三个月通常会负责什么类型的任务？
7. 如果我加入，怎样算前三个月做得不错？
8. 这个方向接下来半年最大的技术风险是什么？

这些问题有两个好处：

- 你能判断岗位是不是只做简单 API 包装。
- 面试官能感受到你关心真实落地，而不是只关心头衔。

反问也可以根据前面面试内容延伸。例如面试官刚问了 RAG 评测，你可以问：

刚才我们聊到 RAG 评测，我也了解一下团队现在的评测集主要来自线上日志、人工构造，还是业务方标注？目前更大的挑战是标注成本，还是指标和用户体验之间的一致性？

这类反问会显得自然，而且能继续展示你的专业判断。

62.9 面试结束后必须复盘

很多人面完一场就等结果，这是浪费。

面试结束后的半小时，是复盘质量最高的时候。因为你还记得：

- 面试官问了什么。
- 哪些问题答得卡。
- 哪些追问暴露了短板。
- 哪些项目表达不够清楚。
- 哪些题其实可以整理成模板。

建议每场面试后记录下面内容：

复盘项	记录内容
公司和岗位	面试轮次、面试官角色、方向
技术问题	原题、你的回答、追问点
项目问题	问了哪个项目、卡在哪里
行为问题	问了什么经历、回答是否具体
系统设计	题目、约束、你漏问了什么
反问信息	团队目标、技术栈、风险信号
待补内容	需要补的知识点或表达
下次改进	下一场要调整的回答

复盘不是写日记，而是为了形成下一轮改进。

例如你发现三家公司都追问“评测集怎么构造”，那说明这不是偶然问题，而是你的项目表达需要补强。

如果面试官总是追问“这个指标怎么来的”，说明你的简历和项目讲稿里结果证据不足。

如果你总在 Agent 权限问题上答得泛，说明需要回去补工具调用、审批、trace、幂等和回滚。

把面试看成迭代过程，求职效率会高很多。

62.10 如何判断一场面试的信号

面试结果不完全可控，但面试过程会释放很多信号。

好的信号包括：

- 面试官愿意深入追问你的项目细节。
- 面试官开始和你讨论真实团队问题。
- 你们在某个技术点上有来回推演。
- 面试官主动介绍岗位挑战和团队现状。
- 面试时间明显延长，且不是因为你的回答混乱。

风险信号包括：

- 面试官只问八股，不关心项目细节。
- 岗位描述很大，但具体工作说不清。
- 团队对评测、上线、监控没有清晰说法。
- 只强调“我们要做最强 Agent”，但讲不出用户、数据和指标。
- 对工作强度、职责边界、汇报关系含糊其辞。
- 多轮面试中不同人对岗位目标说法明显冲突。

当然，单个信号不能决定一切。面试官个人风格不同，公司流程也不同。

但如果多个风险信号同时出现，就要谨慎。

求职不是单向被选择。你也在选择一个系统：团队目标、管理方式、工程文化、业务节奏和成长空间。

62.11 Offer 决策不要只看总包

拿到 offer 后，最容易犯的错误是只看总包数字。

薪资当然重要，但它不是唯一变量。

大模型工程岗位的 offer 决策至少要看八个维度：

维度	要问的问题
方向匹配	是否真的做大模型核心链路，还是外围包装
团队质量	直属 manager 和同事是否专业、稳定
数据和场景	是否有真实用户、真实数据和可验证目标
工程成熟度	是否有评测、监控、上线和回滚机制
成长空间	你能否负责更完整的问题，而不是只做螺丝钉
业务确定性	项目是否有持续投入，还是短期试水
工作强度	节奏是否可持续，是否长期透支
薪资结构	base、bonus、股票、签字费、试用期、兑现条件

对早中期工程师来说，一个能让你快速成长的岗位，可能比短期多一点薪资更重要。

但也不能用“成长”无限抵消薪资和风险。如果团队方向不清、项目不稳定、管理混乱，再多概念也很难沉淀能力。

一个实用方法是给每个 offer 写一页纸：

1. 我加入后前三个月会做什么。
2. 半年后我能沉淀什么能力。
3. 这个岗位最大的风险是什么。
4. 如果一年后离开，我简历上能多出什么有分量的项目。
5. 这个机会是否让我更接近长期目标。

如果这些问题答不出来，就不要只被头衔或总包吸引。

62.12 薪资沟通：专业、具体、留有余地

薪资沟通不是对抗，而是信息交换和价值确认。

基本原则：

- 不要在技术面早期过度纠缠薪资。
- 不要虚构已有 offer 或薪资。
- 不要用情绪化语言施压。
- 不要只说“能不能再高一点”，要给出理由。
- 关注整体 package，而不只是 base。

可以这样表达：

我对这个岗位方向和团队都很感兴趣，尤其是你们在 RAG 评测和线上稳定性上的工作，和我接下来想深耕的方向比较匹配。关于 package，我目前也在推进其他机会。结合我的项目经验、岗位职责和市场水平，如果整体能到 X 到 Y 的区间，我会更容易做出决定。当然我也愿意了解公司在 base、bonus、股票或签字费上的结构安排。

这个表达有几个特点：

- 先确认兴趣。
- 给出理由。
- 给区间而不是绝对命令。
- 讨论整体结构。

如果对方暂时不能提高现金，可以问：

- 是否有签字费。
- 是否能调整股票或 bonus。
- 是否有试用期后 review。
- 是否能提高职级或明确晋升评估。
- 是否能延长决策时间以便完整比较。

但要注意，职级不是越高越好。如果高职级对应更高预期，而你实际支持不足，入职后压力会很大。

62.13 如何优雅拒绝 Offer

拒绝 offer 也要专业。

行业很小，尤其是大模型圈子。你今天拒绝的团队，未来可能成为合作方、同事或面试官。

拒绝时不需要长篇解释，也不要过度批评。

可以这样写：

非常感谢团队在整个面试过程中的时间和认可。我认真考虑后，决定接受另一个与我当前阶段目标更匹配的机会，所以这次不能继续推进。和团队交流过程中我收获很多，也很认可你们在这个方向上的工作。希望未来还有机会合作。

如果是因为薪资或方向不匹配，也可以保持简洁：

经过综合考虑，我认为当前机会和我的阶段性目标在方向/条件上还不是最匹配，所以这次先不接受 offer。感谢团队的时间和信任。

不要失联，不要拖到最后一刻才回复，也不要用一个 offer 去反复压另一个团队。

专业的求职者，不只在面试中专业，在拒绝时也专业。

62.14 入职前要确认的事项

接受 offer 前，建议确认这些事项：

- 职位名称和职级。
- 汇报对象。
- base、bonus、股票、签字费和发放规则。
- 试用期长度和试用期薪资。
- 工作地点、远程政策和出差要求。
- 入职后所属团队和主要方向。

- 是否有竞业、保密或知识产权条款。
- 背调范围和入职材料要求。
- offer 有效期和预计入职时间。

对大模型岗位，还建议确认：

- 是否能接触真实模型应用链路。
- 是否有 GPU、推理服务、数据平台等基础设施支持。
- 是否有明确评测指标和上线流程。
- 是否主要做 demo、售前、平台、业务应用，还是底层 infra。
- 团队对安全、权限、数据合规的要求是什么。

这些问题不是挑剔，而是降低双方预期偏差。

很多入职后的失望，来自 offer 前没有问清楚。

62.15 把求职当成长期系统

求职不是一次性考试，而是一个长期系统。

短期看，你需要拿到 offer。

长期看，你需要持续积累可迁移能力：

- 能做真实项目。
- 能讲清楚项目。
- 能复盘失败。
- 能判断团队。
- 能选择机会。
- 能在新环境中继续成长。

大模型行业变化很快。某个框架、模型或协议可能很快过时，但下面这些能力不会过时：

- 把模糊问题定义清楚。
- 用实验和评测减少争论。
- 在质量、成本、延迟和风险之间做取舍。
- 和不同角色协作推进复杂项目。
- 面对未知问题诚实拆解并持续学习。

行为面试、反问、复盘和 offer 决策，本质上都在考这些能力。

62.16 小结

大模型工程师求职的后半段，不只是“等结果”和“谈薪资”。

行为面试要讲真实经历，不要背空泛模板。STAR 方法有用，但更重要的是冲突、动作、证据和复盘。

反问环节要用来收集岗位信息，判断团队是否真的在解决有价值的问题。

面后复盘要沉淀题目、追问、短板和下一轮改进，把面试当成迭代过程。

offer 决策要综合方向、团队、场景、工程成熟度、成长空间、业务风险、工作强度和薪资结构，而不是只看总包。

薪资沟通要专业、具体、留有余地；拒绝 offer 也要保持尊重。

最终，你要争取的不只是一个岗位，而是一段能让你持续变强的职业路径。

第 63 章：入职准备、试用期与前 90 天站稳岗位

拿到 offer 不是结束。

对大模型工程师来说，真正的考验往往从入职第一天才开始。

面试阶段你展示的是过往能力和潜力；入职之后，团队关心的是你能否在真实约束下持续交付：理解业务、融入协作、读懂代码、定位问题、稳定上线、推动改进。

很多候选人面试表现很好，但入职后前 90 天没有站稳，常见原因不是“不够聪明”，而是没有把新环境当成一个需要系统适应的工程问题：

- 没有快速建立业务上下文。
- 过早评价历史系统，却不了解历史约束。
- 只埋头写代码，不主动对齐优先级。
- 做了很多事，但没有让 manager 和团队感知到进展。
- 对模型效果、线上风险和业务目标之间的关系理解太慢。
- 遇到问题不及时暴露，最后拖成试用期风险。

这一章讲入职前准备、前 30 天、60 天和 90 天分别该做什么，以及如何在新团队中建立可信度。

63.1 入职前：把不确定性提前降下来

接受 offer 后到正式入职前，通常会有几天到几周时间。

这段时间不需要过度焦虑，也不需要把所有技术重新学一遍。更重要的是把“岗位上下文”补起来。

建议确认四类信息。

第一类是组织信息：

- 直属 manager 是谁。
- 所在团队负责哪个业务方向。
- 团队规模大概多大。
- 团队里算法、工程、产品、平台如何分工。
- 入职后是否有 mentor 或 buddy。

第二类是技术信息：

- 主要技术栈是什么。
- 使用哪些模型、推理服务、向量库、评测平台和监控系统。
- 是否有内部文档可以提前看。
- 是否需要提前准备开发环境或权限申请。

第三类是目标信息：

- 入职前三个月大致会负责什么。
- 当前团队最急的问题是什么。
- 试用期评估更看重交付、学习速度、协作还是独立负责能力。

第四类是手续信息：

- 入职材料。
- 背调进度。
- 设备领取。
- 办公地点。
- 入职时间。

这些信息不一定都能提前拿到，但你可以礼貌询问。

例如：

为了更快进入状态，我想提前了解一下入职后前几周大概会接触哪些业务或技术栈。如果有适合提前阅读的公开资料或内部文档目录，也可以先发我，我会提前熟悉。

这类沟通不会显得冒进，反而会让团队感觉你有准备意识。

63.2 不要在入职前盲目“狂补技术”

很多人拿到大模型岗位 offer 后，会开始焦虑：

- 要不要把所有 Transformer 论文再读一遍？
- 要不要重学 CUDA？
- 要不要把 LangChain、LlamaIndex、vLLM、Ray、Kubernetes 全部补完？
- 要不要提前做一个完整 Agent 项目？

这种焦虑可以理解，但效率不一定高。

入职前最值得补的，不是“大而全”的知识，而是和岗位最相关的短板。

如果你加入的是 RAG 应用团队，优先补：

- 文档解析。
- chunk 策略。
- embedding 与 rerank。
- 召回评测。
- 引用与拒答。
- 线上日志分析。

如果你加入的是 Agent 平台团队，优先补：

- function calling。
- tool schema。
- 权限和审批。
- trace 与可观测性。
- 幂等与重试。
- 工具执行沙箱。

如果你加入的是推理服务团队，优先补：

- batching。
- KV cache。
- PagedAttention。
- quantization。
- speculative decoding。
- 延迟和吞吐压测。

如果你加入的是评测团队，优先补：

- 离线评测集构造。
- LLM-as-a-judge。
- 人工标注一致性。
- 回归测试。
- A/B 实验。
- 指标与用户体验偏差。

入职前的学习目标不是变成全能专家，而是让自己在第一周听得懂团队讨论，在第一个任务中少踩低级坑。

63.3 第一周：少评价，多理解

入职第一周最重要的不是立刻产出惊人成果，而是建立上下文。

你需要理解：

- 团队正在解决什么业务问题。
- 当前系统为什么长成这样。
- 哪些是历史债务，哪些是有意取舍。
- 当前最重要的指标是什么。
- 哪些问题已经有人尝试过但失败了。
- 哪些系统不能随便动。

新同学常犯一个错误：刚看几天代码，就开始评价系统设计不合理。

有些评价可能是对的，但时机不对。你还不知道历史约束、业务压力、人力限制、上线事故、合规要求和组织边界。

更稳妥的方式是先问问题：

- 这个模块当初为什么这样拆？
- 这里是因为性能原因，还是因为历史兼容？
- 这个配置目前由谁维护？
- 这个指标线上有没有监控？
- 这块之前有没有尝试过其他方案？

问问题不是示弱，而是在建立系统模型。

第一周建议完成一份自己的“团队地图”：

维度	需要记录的内容
业务目标	团队服务的用户、场景、核心指标
系统链路	请求从入口到模型再到结果返回的路径
数据链路	数据来源、清洗、标注、评测和日志回流
模型链路	使用哪些模型、如何路由、如何升级
上线链路	开发、测试、灰度、回滚和监控流程
协作关系	产品、算法、平台、业务方和安全团队接口人
风险区域	不能轻易改的模块、历史事故、合规要求

这份地图不用正式提交，但它会极大提升你后续做事的准确性。

63.4 前 30 天：交付一个小而完整的闭环

入职前 30 天，最好的目标不是“证明自己很强”，而是交付一个小而完整的闭环。

什么叫小而完整？

不是只改一行配置，也不是接一个巨大项目。

它应该满足几个条件：

1. 范围可控。
2. 能真实解决一个问题。
3. 有明确验收标准。
4. 涉及基本开发、测试、上线或文档流程。
5. 能让你熟悉团队协作方式。

例如：

- 给 RAG 系统补一个失败 case 分析脚本。
- 修复一个线上引用格式错误。
- 为某类知识库增加标题路径 metadata。
- 给 Agent 工具调用增加 trace 字段。
- 为推理服务补一个延迟压测报告。
- 把一个人工排查流程整理成可复用 runbook。
- 给评测集增加一类高频业务问题。

这类任务不一定惊艳，但非常适合新人建立可信度。

它证明你能：

- 看懂现有系统。
- 尊重团队流程。
- 和相关人对齐需求。
- 写出可合并的代码或文档。

- 交付后能说明效果。

前 30 天不要贪多。

新人最重要的是建立“这个人靠谱”的第一印象。靠谱的含义很具体：说清楚、做得到、有反馈、遇到问题不隐瞒。

63.5 如何和 manager 对齐预期

试用期风险很多时候不是能力问题，而是预期错位。

你以为自己在认真学习，manager 可能觉得你产出太慢。

你以为自己做了很多优化，manager 可能觉得这些不是当前优先级。

你以为某个问题很重要，团队可能已经决定暂时不投入。

因此，入职后要主动和 manager 对齐。

建议在第一周或第二周约一次 1:1，问清楚：

- 试用期最重要的评估标准是什么？
- 前 30 天希望我完成什么？
- 哪些事情我应该优先熟悉？
- 哪些项目是当前团队最高优先级？
- 我应该多久同步一次进展？
- 如果遇到阻塞，应该找谁？

可以直接说：

我希望前三个月尽快进入状态，也避免方向跑偏。想和你对齐一下，前 30 天你最希望我达成的结果是什么？哪些事情如果我做到了，会说明我已经比较好地融入团队？

这不是讨好，而是降低不确定性。

对齐后，最好把结论写成简短文字发给 manager：

我理解下来，接下来两周优先做三件事：熟悉现有 RAG 链路，完成 XX 失败 case 的分析，补齐 YY 评测脚本。过程中我每周五同步一次进展，如果遇到数据权限或上线流程问题，会及时找 A 或 B 确认。

这样的同步能避免很多误会。

63.6 前 60 天：从执行任务到理解系统

前 30 天你主要是在适应环境和交付小闭环。到 60 天左右，你需要开始从“执行任务”升级为“理解系统”。

这意味着你不仅知道怎么改代码，还要知道为什么这样改。

对 RAG 系统，你要能说清楚：

- 文档从哪里来。
- 如何解析和切分。
- embedding 和 rerank 如何选型。
- 检索失败通常发生在哪些环节。
- 哪些问题应该拒答。
- 如何判断改动是否真的提升效果。

对 Agent 系统，你要能说清楚：

- 工具有哪些权限边界。
- 工具 schema 如何设计。
- 多步调用如何记录状态。
- 失败时如何重试或回滚。
- 哪些操作需要用户确认。
- trace 如何帮助排查问题。

对推理服务，你要能说清楚：

- 请求如何排队和 batching。
- KV cache 如何影响显存。
- 不同模型和 batch size 下延迟如何变化。
- P50、P95 和吞吐之间如何权衡。
- 降级和限流策略是什么。

对评测系统，你要能说清楚：

- 样例从哪里来。
- 标注口径是什么。
- 指标是否和用户体验一致。
- 评测结果如何进入发布流程。
- 如何避免只对评测集过拟合。

60 天左右，一个合格的新同学应该已经能独立负责中小任务，并能解释任务所在链路的上下游影响。

63.7 前 90 天：形成可见贡献

试用期结束前，你需要形成可见贡献。

可见贡献不是指天天加班，也不是写了多少行代码，而是团队能明确说出：你解决了什么问题，带来了什么变化。

可见贡献可以是：

- 一个上线功能。
- 一个稳定性改进。
- 一个评测体系补齐。
- 一个成本优化。
- 一个排障工具。
- 一个关键线上问题修复。
- 一套文档或 runbook。
- 一个跨团队流程改善。

关键是要有证据。

例如：

- 某类问题的证据命中率提升了多少。
- 某个接口 P95 延迟下降了多少。
- 某个排查流程从 2 小时缩短到 20 分钟。
- 某类线上错误告警减少了多少。
- 某个评测集覆盖了多少高频问题。
- 某个工具让多少个业务接入方复用。

如果你的工作很难量化，也要尽量留下可验证产物：文档、设计稿、代码合并记录、评测报告、上线记录、事故复盘、需求验收。

90 天时，manager 不应该只听你说“我学到了很多”，而应该能看到你已经让团队某个局部变好了。

63.8 新人如何避免“只会问问题”

新人当然需要问问题。

但如果所有问题都直接丢给别人，会消耗团队信任。

更好的方式是带着上下文提问。

低质量问题是：

这个为什么报错？

高质量问题是：

我在本地跑 RAG 评测脚本时，`retriever_config` 为空导致初始化失败。我查了最近两次配置变更，发现测试环境现在从配置中心读取，但本地文档里还是使用 `yaml`。想确认一下，本地开发推荐是手动导出配置，还是走 `mock` 配置？

低质量问题是：

这个模块是干什么的？

高质量问题是：

我看请求从 `gateway` 进入后，会先经过 `query rewrite`，再进入 `hybrid retrieval`，最后 `rerank`。这里的 `query rewrite` 是主要为了解决口语化问题，还是为了补齐业务实体？有没有评测看它对召回的影响？

提问前建议先做三件事：

1. 查文档。
2. 看代码或日志。
3. 写下自己的初步判断。

然后再问：

我查到的信息是 A，看到的现象是 B，我猜可能原因是 C。想确认这个判断是否正确。

这种提问方式会让别人愿意帮你，因为你不是把思考外包给对方。

63.9 不要过早推动“大重构”

很多工程师入职后看到历史代码，会本能想重构。

有时这确实必要，但新人阶段不适合轻易推动大重构。

原因很简单：你还没有足够上下文，也没有足够信任额度。

历史系统看起来不优雅，可能背后有这些原因：

- 兼容老客户。
- 迁移成本太高。
- 上线窗口有限。
- 依赖其他团队接口。
- 某些逻辑是事故后加的保护。
- 业务规则本身就复杂。

新人更适合做渐进式改进：

- 先补测试。
- 先补日志。
- 先补类型和配置校验。
- 先消除一个明确重复逻辑。
- 先把风险最高的路径加监控。
- 先把重构收益和风险写清楚。

如果确实认为需要重构，可以这样表达：

我看到这里有三类逻辑混在一起：召回配置、权限过滤和结果格式化。短期我不建议直接大改，因为这条链路影响线上问答。我的建议是先补一组回归样例和日志字段，再把结果格式化抽出来，作为第一步低风险改动。等这一步稳定后，再评估是否拆权限过滤。

这种表达比“这里代码太乱了，我们重写吧”更容易被接受。

63.10 试用期中的沟通节奏

试用期要避免两个极端。

一个极端是过度同步，每做一点小事都打扰别人。

另一个极端是长时间沉默，直到 deadline 前才暴露问题。

比较合适的节奏是：

- 每周给 manager 同步一次重点进展。
- 对关键任务，在开始前确认目标和验收标准。
- 遇到阻塞超过半天到一天，要及时暴露。
- 对跨团队依赖，要提前约时间，不要等最后。
- 重要结论尽量文字化，避免口头信息丢失。

周报可以很短：

本周进展：

1. 完成 RAG 失败 case 标注 80 条，初步归因为召回缺失、排序错误、知识库缺失三类。
2. 修复引用展示中的段落编号丢失问题，已合并到测试环境。
3. 梳理了当前评测脚本运行流程，补充了本地运行文档。

风险/阻塞：

1. 线上日志权限还未开通，影响进一步分析真实用户问题，已联系 A 处理。

下周计划：

1. 补齐 120 条回归评测样例。
2. 对比 metadata 前缀对 Top-3 证据命中的影响。

这样的同步清晰、简短、可跟踪。

63.11 如何处理试用期负反馈

如果试用期中收到负反馈，不要本能防御，也不要立刻崩溃。

负反馈可能来自三个方面：

- 产出不够。
- 沟通不够。
- 方向不对。

你要做的是把反馈具体化。

不要只问：

我哪里做得不好？

要问：

你提到我进展偏慢，能否帮我具体看一下，是任务拆解慢、代码实现慢、还是遇到阻塞没有及时同步？
如果接下来两周要改善，你最希望看到哪两个变化？

然后把改进动作写下来：

- 每两天同步一次当前任务状态。
- 每个任务开始前先写验收标准。
- 遇到阻塞超过半天就暴露。
- 请 mentor 帮忙 review 设计方向。
- 把大任务拆成可交付小任务。

负反馈不可怕。可怕的是反馈很模糊，而你却没有把它变成可执行计划。

63.12 在新团队建立可信度的底层逻辑

可信度不是靠一句“我很负责”建立的。

它来自连续的小证据：

- 答应的事情按时完成。

- 做不到时提前说。
- 不懂的问题愿意查清楚。
- 线上风险不隐瞒。
- 代码改动有测试和回滚意识。
- 技术判断有证据，而不是只凭感觉。
- 争论时围绕目标和数据，而不是面子。

对大模型工程师来说，还有几条特别重要：

- 不把所有问题都归因于“模型不行”。
- 不用 demo 效果替代系统评测。
- 不忽视安全、权限、成本和延迟。
- 不在没有证据时承诺模型效果。
- 不把 prompt 当成唯一解决方案。

如果团队发现你每次判断都能讲清依据，每次改动都考虑上下游影响，每次遇到问题都能复盘，你的可信度会增长很快。

63.13 前 90 天的常见错误清单

下面这些错误很常见，最好提前避开。

第一，急于证明自己。

新人越急于证明自己，越容易跳过上下文，做出不合适的判断。

第二，只学习不交付。

学习很重要，但试用期最终要看你能否产生贡献。

第三，只交付不沟通。

你做了什么、为什么做、进展如何、风险在哪里，都需要被关键人知道。

第四，不敢暴露问题。

问题越晚暴露，解决成本越高。

第五，过度依赖 mentor。

mentor 可以帮你，但不能替你承担全部思考。

第六，忽略业务目标。

技术改动必须服务业务目标。否则再优雅也可能不是优先级。

第七，不记录。

新环境信息密度很高，不记录就会反复问同样问题，也难以形成系统理解。

第八，轻视线上风险。

大模型应用的线上风险不只是不报错，还包括胡答、越权、成本暴涨、延迟抖动和用户误用。

63.14 试用期结束前的复盘材料

试用期结束前，建议主动准备一份简短复盘。

内容不需要华丽，重点是清楚。

可以包含：

- 入职后主要完成了哪些工作。
- 每项工作的背景、动作和结果。
- 哪些能力已经补齐。
- 对团队系统和业务的理解。

- 当前还存在的短板。
- 接下来 1 到 2 个季度希望负责的方向。

示例结构：

一、前三个月主要工作

1. RAG 失败 case 分析与评测集补齐
2. 引用格式错误修复与测试环境上线
3. 本地评测脚本文档化

二、结果

1. 标注 160 条真实问题，归因 5 类失败原因
2. 修复段落编号丢失问题，减少引用不完整 case
3. 新人本地启动评测流程从半天缩短到 30 分钟以内

三、我的理解

当前系统主要瓶颈不是生成模型，而是文档解析和 metadata 缺失导致的召回质量问题。

四、后续计划

1. 推进标题路径 metadata 的灰度实验
2. 建立每周失败 case 回流机制
3. 补齐拒答类评测样例

这份材料既帮助 manager 评估你，也帮助你从“新人”切换到“稳定贡献者”。

63.15 入职后的长期成长路线

前 90 天站稳之后，不代表可以停止思考。

接下来你要逐步扩大影响范围。

第一阶段：能独立完成明确任务。

你能根据需求完成开发、测试、上线和文档。

第二阶段：能独立负责一个模块。

你不仅能改代码，还能维护质量、处理问题、规划小版本。

第三阶段：能定义问题。

你能从线上反馈、业务目标和系统指标中发现真正值得做的事。

第四阶段：能推动跨团队协作。

你能协调产品、算法、平台、安全和业务方，让复杂项目落地。

第五阶段：能形成方法论。

你能把一次项目经验沉淀成评测流程、工程规范、平台能力或团队共识。

大模型工程师的成长，不是简单从“会调用模型”变成“会调用更多模型”。真正的成长是你越来越能处理复杂约束：效果、成本、延迟、安全、业务价值、团队协作和长期可维护性。

63.16 小结

拿到 offer 只是职业转换的开始。

入职前要降低信息不确定性，聚焦补齐和岗位直接相关的短板。

第一周少评价、多理解，建立业务、系统、数据、模型、上线和协作地图。

前 30 天要交付一个小而完整的闭环，建立“靠谱”的第一印象。

前 60 天要从执行任务升级为理解系统，能讲清楚上下游链路和关键取舍。

前 90 天要形成可见贡献，用结果、数据、文档或上线记录证明你让团队某个局部变好了。

试用期中最重要的不是表现得无所不能，而是持续建立可信度：目标清楚、进展透明、问题及时暴露、判断有证据、交付可验证。

真正站稳岗位之后，你的目标不只是“通过试用期”，而是在新团队中逐步成长为能定义问题、推动协作、沉淀方法的大模型工程师。

第 64 章：从初级到高级，大模型工程师的长期成长路线

前面几章讲了如何准备面试、拿到 offer、入职后前 90 天如何站稳岗位。

但职业成长不会停在“通过试用期”。

大模型工程师真正的长期竞争力，不是会用某个框架、会调某个 prompt、会背某些模型原理，而是能在越来越复杂的问题中持续产生影响：从完成任务，到负责模块；从解决局部问题，到定义系统方向；从个人交付，到推动团队形成方法论。

这一章讨论一个更长期的问题：大模型工程师如何从初级成长为高级，甚至成长为技术负责人。

64.1 职级不是年限，而是影响范围

很多人理解职业成长时，会把它和工作年限绑定：

- 工作 1 到 2 年是初级。
- 工作 3 到 5 年是中级。
- 工作 5 年以上就是高级。

这只是非常粗略的外部标签。

真实组织里，职级更看重影响范围。

初级工程师主要影响自己手头的任务。

中级工程师可以影响一个模块或一条链路。

高级工程师可以影响一个系统、一个方向，甚至一个团队的工作方式。

技术负责人影响的是更大范围的技术决策、业务结果和组织能力。

大模型工程岗位也是如此。

会写一个 RAG demo，不等于能负责企业知识库问答系统。

会接 function calling，不等于能负责 Agent 平台。

会部署一个模型，不等于能负责推理服务的成本、稳定性和容量规划。

会做一次评测，不等于能建立团队级评测体系。

成长的核心问题是：你能稳定解决多大范围、多高不确定性、多强约束下的问题。

64.2 初级大模型工程师：把明确任务做好

初级工程师最重要的能力是可靠完成明确任务。

这里的“明确任务”包括：

- 修复一个已知 bug。
- 接入一个已有模型 API。
- 增加一个工具调用。
- 补充一组评测样例。
- 写一个日志分析脚本。
- 为某个接口增加监控字段。

- 根据设计文档实现一个小功能。

初级阶段不要急于证明自己“能做架构”。

更重要的是把基础动作做扎实：

- 需求理解准确。
- 代码可读。
- 测试覆盖关键路径。
- 文档能说明使用方式。
- 遇到阻塞及时同步。
- 不把风险藏到最后。
- 不在不了解上下文时随意改动线上链路。

对大模型工程师，初级阶段尤其要养成几个习惯。

第一，凡是模型效果问题，都要保留样例。

不要只说“模型答得不好”，要记录问题、输入、检索结果、模型输出、期望答案和失败类型。

第二，凡是线上问题，都要看日志和 trace。

不要凭感觉猜是 prompt、模型、检索还是工具调用的问题。

第三，凡是改 prompt 或参数，都要能回归。

不要只凭一次 demo 判断改动有效。

第四，凡是接工具或外部系统，都要考虑失败。

工具超时、返回空、权限不足、重复执行、部分成功，都是必须考虑的情况。

初级阶段的目标不是“看起来很厉害”，而是让团队放心把明确任务交给你。

64.3 中级大模型工程师：负责一个模块或链路

中级工程师的关键变化是：你不再只是执行别人拆好的任务，而是能负责一个模块或链路。

例如：

- 负责 RAG 的文档解析和切分链路。
- 负责检索和 rerank 模块。
- 负责 Agent 工具注册和权限校验。
- 负责模型评测平台的一部分。
- 负责推理服务的压测和监控。
- 负责某个业务场景的大模型应用落地。

负责模块意味着你要关心更多事情：

- 模块输入输出是否清晰。
- 错误如何处理。
- 指标如何监控。
- 上下游如何协作。
- 新需求如何扩展。
- 线上问题如何排查。
- 文档是否能帮助别人接入。

中级工程师和初级工程师的区别，不在于写代码速度快一点，而在于是否能主动发现和补齐模块中的空洞。

例如你负责 RAG 的文档解析链路，不能只完成“把 PDF 转成文本”。你还要思考：

- 表格如何处理。
- 页眉页脚如何去除。
- 标题层级如何保留。
- 条款编号如何保留。

- 图片和扫描件如何处理。
- 解析失败如何告警。
- 文档更新后如何增量重建索引。
- 解析质量如何评估。

如果你负责 Agent 工具调用，也不能只完成“把函数接上”。你还要思考：

- schema 是否清晰。
- 参数是否可验证。
- 工具是否幂等。
- 权限是否按用户隔离。
- 高风险操作是否需要确认。
- 调用链路是否可追踪。
- 失败后是否能恢复。

中级阶段的目标是成为某个局部领域的 owner。

64.4 高级大模型工程师：定义问题并做取舍

高级工程师不只是解决别人定义好的问题，而是能定义什么问题值得解决。

这在大模型领域尤其重要，因为表面问题和真实问题经常不一样。

业务方说：

模型不够聪明。

真实问题可能是：

- 文档没有被正确解析。
- 检索召回缺失。
- prompt 没有约束引用。
- 用户问题过于模糊。
- 知识库本身缺失答案。
- 评测样例和真实流量不一致。

产品说：

我们需要一个 Agent 自动完成所有操作。

真实问题可能是：

- 只读查询已经能覆盖 80% 需求。
- 自动执行高风险操作收益不高但风险很大。
- 用户需要的是任务进度可见，而不是完全自动化。

平台说：

推理成本太高。

真实问题可能是：

- 长 prompt 中有大量重复上下文。
- 简单请求用了过强模型。
- batch 策略不合理。
- 缓存没有命中。
- 输出长度没有控制。

高级工程师的价值，在于能把模糊抱怨转成可分析的问题，把复杂问题拆成可验证假设，并在效果、成本、延迟、安全和交付时间之间做取舍。

高级工程师常问的问题不是“怎么实现”，而是：

- 这个问题是否真的值得做？
- 成功标准是什么？

- 有没有更低成本的方案？
- 哪些风险必须先控制？
- 哪些指标能证明改动有效？
- 如果失败，如何回滚？
- 这个方案会不会增加长期维护成本？

64.5 技术负责人：让团队持续变强

技术负责人和高级工程师的区别，是影响对象从“系统”进一步扩展到“团队”。

技术负责人当然要有技术判断，但更重要的是让团队整体变强。

这包括：

- 建立技术方向。
- 拆解复杂项目。
- 识别关键风险。
- 设计协作机制。
- 培养团队成员。
- 建立工程规范。
- 推动复盘和知识沉淀。
- 在业务目标和技术债之间做平衡。

大模型团队中的技术负责人，常常需要同时面对多种角色：

- 产品希望快速上线。
- 算法希望尝试新模型。
- 平台关注成本和稳定性。
- 安全团队关注权限和合规。
- 业务方关注结果是否可用。
- 管理层关注投入产出比。

技术负责人不能只站在单一技术视角说“这个方案最好”。他要能组织不同角色达成可执行共识。

例如在做企业 Agent 平台时，技术负责人要回答：

- 哪些任务允许自动执行？
- 哪些任务必须人工确认？
- 工具权限如何隔离？
- 多租户数据如何防泄漏？
- 如何做 trace 和审计？
- 如何评估 Agent 是否真的提高效率？
- 如何在 demo 压力下避免过度承诺？

这些问题不是单个函数或单个模型能解决的，而是系统和组织共同作用的结果。

64.6 成长路线一：应用工程路线

大模型工程师可以有多条成长路线。

第一条是应用工程路线。

这条路线关注如何把模型能力变成真实业务系统。

典型工作包括：

- RAG 知识库问答。
- 智能客服。
- 企业搜索。
- 代码助手。
- 数据分析 Copilot。
- 文档生成和审核。

- 业务流程 Agent。
- 垂直行业大模型应用。

应用工程路线的核心能力是：

- 理解业务场景。
- 定义用户问题。
- 设计模型交互。
- 处理非结构化数据。
- 建立评测和反馈闭环。
- 控制幻觉、权限和安全风险。
- 把 demo 变成可维护系统。

这条路线很适合既懂工程又愿意理解业务的人。

它的难点是：成功不只取决于技术。你需要和产品、运营、业务专家、法务、安全、平台团队一起工作。

应用工程师越往高级走，越不能只说“模型能力不够”。你要能判断哪些问题应该靠数据、检索、流程、交互、权限、评测或组织协作解决。

64.7 成长路线二：模型与算法工程路线

第二条是模型与算法工程路线。

这条路线更关注模型能力本身，以及如何通过训练、微调、对齐、数据和评测改进模型表现。

典型工作包括：

- SFT。
- DPO、PPO、GRPO 等偏好优化。
- 指令数据构造。
- 合成数据生成。
- 多轮对话优化。
- 领域模型微调。
- reasoning 数据和验证器设计。
- 模型评测和误差分析。

这条路线的核心能力是：

- 理解模型训练机制。
- 理解数据质量和分布。
- 能设计实验。
- 能分析模型行为。
- 能判断指标是否可靠。
- 能把论文方法转成可复现实验。
- 能与工程团队协作上线模型。

模型与算法工程路线容易出现一个误区：只追求方法新，不关心可验证收益。

真正成熟的算法工程师，不是每周换一个新 loss，而是能回答：

- 这个方法解决什么具体问题？
- 和 baseline 比提升在哪里？
- 数据是否公平？
- 评测是否泄漏？
- 是否对某些能力有副作用？
- 成本是否值得？
- 上线后如何监控退化？

如果一个算法改进不能被稳定复现、不能解释收益来源、不能进入真实链路，它的工程价值就有限。

64.8 成长路线三：推理与基础设施路线

第三条是推理与基础设施路线。

这条路线关注如何高效、稳定、低成本地服务模型。

典型工作包括：

- 推理引擎。
- serving 平台。
- GPU 资源调度。
- continuous batching。
- KV cache 管理。
- 量化和编译优化。
- 多模型路由。
- 容量规划。
- 监控、限流和降级。

这条路线的核心能力是：

- 理解模型计算结构。
- 理解 GPU 和显存瓶颈。
- 理解吞吐、延迟和成本权衡。
- 能做压测和性能分析。
- 能设计高可用服务。
- 能处理多租户和资源隔离。
- 能把底层优化转化为业务收益。

推理 infra 的难点是，很多问题不是平均值问题，而是尾部问题。

平均延迟看起来不错，不代表 P95、P99 稳定。

单模型压测结果不错，不代表多租户混部时稳定。

小流量没问题，不代表高峰期队列不会堆积。

高级推理工程师需要能讲清：

- 一个请求为什么慢。
- 显存为什么碎片化。
- batch 为什么没有打满。
- 为什么吞吐提高后首 token 延迟变差。
- 什么场景适合量化。
- 什么场景适合 speculative decoding。
- 如何在 SLA 和成本之间做取舍。

这条路线技术深度高，也很接近 AI 时代的底层基础设施。

64.9 成长路线四：评测、安全与治理路线

第四条是评测、安全与治理路线。

这条路线在早期容易被低估，但越到真实业务落地越重要。

典型工作包括：

- 模型能力评测。
- RAG 评测。
- Agent 任务评测。
- 红队测试。
- 幻觉检测。
- 安全策略。
- 权限和审计。

- 数据合规。
- 发布门禁。

这条路线的核心能力是：

- 把模糊质量问题转成可测指标。
- 设计可靠评测集。
- 识别指标和真实用户体验的偏差。
- 建立上线前后的回归机制。
- 识别模型滥用和越权风险。
- 在效率和安全之间做分层策略。

随着 Agent 和自动化操作越来越多，评测与治理会越来越重要。

一个问答系统答错，可能只是用户体验问题。

一个 Agent 错误执行退款、删除数据或发送邮件，就可能是业务事故。

因此，高级评测和治理工程师不只是“写测试题”，而是在为模型系统建立安全边界和可信发布机制。

64.10 不同路线可以交叉，但要有主线

很多大模型工程师会同时接触多个方向。

例如应用工程师需要懂一点评测和推理成本；推理工程师需要理解业务流量形态；算法工程师需要理解数据和上线链路。

交叉是好事。

但完全没有主线会带来问题：什么都懂一点，关键时刻没有一个领域能承担责任。

建议每个阶段明确自己的主线能力。

例如：

- 当前主线是 RAG 应用工程，辅线是评测。
- 当前主线是 Agent 平台，辅线是安全治理。
- 当前主线是推理 serving，辅线是模型量化。
- 当前主线是训练数据，辅线是模型评测。

主线决定你要在哪类问题上形成深度。

辅线决定你能和哪些团队有效协作。

高级工程师通常不是“只懂一个点”的人，但一定有一个足够硬的主轴。

64.11 如何判断自己是否在成长

成长不能只靠感觉。

可以用下面的问题自查。

第一，你负责的问题是否更模糊了？

如果你一直只做别人拆好的任务，说明影响范围没有明显扩大。

第二，你是否能独立定义验收标准？

如果每件事都要别人告诉你怎么才算完成，说明还停留在执行层。

第三，你是否能解释系统 trade-off？

例如为什么选择 rerank 而不是换 embedding，为什么限制 Agent 自动执行，为什么先做缓存而不是换模型。

第四，别人是否会主动来问你某个领域的问题？

这说明你已经在局部建立专业信誉。

第五，你是否沉淀了可复用资产？

包括文档、工具、评测集、平台能力、排障手册、设计规范。

第六，你是否减少了团队的重复劳动？

高级工程师的价值不只是自己干活，而是让别人也更高效。

第七，你是否能处理失败？

遇到线上问题、实验失败、需求变化时，你是否能稳定拆解、沟通和复盘。

如果这些问题的答案越来越积极，你就在成长。

64.12 晋升材料怎么准备

很多工程师平时做了不少事，但晋升时讲不清楚影响。

原因是只记录了任务，没有记录价值。

晋升材料通常要回答四个问题：

1. 你解决了什么重要问题？
2. 你采取了什么关键行动？
3. 结果如何，有什么证据？
4. 这个结果为什么体现更高职级能力？

不要只写：

完成 RAG 系统优化。

要写成：

负责企业知识库问答的召回质量优化。通过线上失败 case 标注，将问题归因为文档解析、标题层级丢失、召回排序和拒答边界四类。设计并上线标题路径 metadata、相邻片段扩展和 rerank 规则调整，使离线 Top-3 证据命中率从 68% 提升到 82%，线上引用不完整反馈下降。并沉淀了每周失败 case 回流流程，后续被两个业务知识库复用。

这个表述包含：问题、动作、结果、影响扩散。

晋升材料最忌讳堆列表：

- 做了 A。
- 做了 B。
- 做了 C。

更好的方式是讲主线：

- 业务背景是什么。
- 核心矛盾是什么。
- 你如何拆解。
- 你如何推动。
- 结果如何验证。
- 经验如何沉淀和复用。

大模型工程岗位尤其要避免只讲“用了什么新技术”。晋升看的是影响，而不是技术名词数量。

64.13 如何避免职业成长中的常见陷阱

第一，追热点而不沉淀。

今天学 RAG，明天学 Agent，后天学多模态，最后没有一个方向形成作品和方法论。

第二，只做 demo，不做系统。

demo 能展示可能性，但系统要处理稳定性、权限、监控、评测、成本和维护。

第三，只关注模型，不关注数据。

很多效果问题最终都来自数据质量、数据覆盖和数据链路。

第四，只关注技术，不关注业务。

如果你不知道用户是谁、痛点是什么、成功标准是什么，就很难做出真正有价值的系统。

第五，只关注个人产出，不关注协作。

越往高级走，越需要通过别人产生影响。

第六，把忙碌当成长。

忙不等于成长。重复做低杠杆任务，只会增加疲惫，不会自然变高级。

第七，回避困难反馈。

真正的成长往往来自不舒服的反馈：设计不够稳、沟通不够清楚、判断不够有证据、没有站在业务视角。

64.14 建立自己的技术资产

长期成长需要技术资产。

技术资产不是收藏链接，也不是囤课程，而是你能够反复复用、持续升级的东西。

例如：

- 项目复盘文档。
- 面试项目表达稿。
- RAG 失败 case 分类表。
- Agent 工具设计 checklist。
- 推理压测模板。
- 模型评测样例库。
- 线上事故复盘模板。
- 常见技术方案对比表。
- 自己写过的可复用脚本或工具。

这些资产有两个作用。

第一，它们提升你的工作效率。

第二，它们帮助你形成自己的方法论。

比如你做过三次 RAG 项目，如果每次都从零开始，那只是经验重复。

如果你能沉淀出一套流程：数据检查、文档解析、chunk 策略、召回评测、引用验证、拒答边界、线上日志回流，那你就在形成专业能力。

高级工程师通常都有自己的工具箱。

不一定公开，不一定很复杂，但在关键时刻能帮助他们更快拆解问题。

64.15 技术影响力不是写文章那么简单

很多人把技术影响力理解成写博客、做分享、发开源项目。

这些当然有价值，但影响力的本质不是曝光，而是让别人因为你的工作变得更好。

在团队内部，技术影响力可以表现为：

- 你的文档让新人更快上手。
- 你的评测流程减少了线上回归。
- 你的工具被多个项目复用。
- 你的复盘推动了流程改进。
- 你的设计评审帮助团队避开风险。

- 你的技术判断成为团队决策依据。

在团队外部，技术影响力可以表现为：

- 分享真实项目经验。
- 开源可复用工具。
- 写清楚复杂概念。
- 参与标准、协议或社区讨论。
- 帮助更多人理解大模型工程化。

但不要为了影响力而影响。

没有真实项目和深度思考支撑的输出，很容易变成概念搬运。

最好的技术影响力来自真实问题、真实取舍和真实复盘。

64.16 小结

大模型工程师的成长，本质上是影响范围和问题复杂度的扩大。

初级阶段，要把明确任务做好，建立可靠性。

中级阶段，要负责模块或链路，成为局部 owner。

高级阶段，要能定义问题、做取舍、处理模糊性和系统性风险。

技术负责人阶段，要让团队持续变强，建立方向、机制、规范和方法论。

大模型工程师可以走应用工程、模型算法、推理基础设施、评测安全治理等路线。路线可以交叉，但要有主线。

长期成长不能只靠追热点，而要靠持续沉淀技术资产、项目复盘、可复用工具、评测体系和系统化判断。

最终，一个优秀的大模型工程师，不只是会使用模型的人，而是能把模型能力、安全边界、工程系统、业务价值和团队协作连接起来的人。

第 65 章：大模型工程师如何持续学习和跟进技术演进

上一章讲了从初级到高级的大模型工程师成长路线。

但还有一个更现实的问题：大模型领域变化太快，模型、框架、训练方法、推理引擎、Agent 协议、评测方法几乎每个月都有新东西。一个工程师如果只靠零散刷资讯，很容易陷入两种状态。

一种是焦虑：每天都觉得自己落后了。

另一种是麻木：新概念太多，最后干脆不跟了。

真正有效的学习，不是追逐所有热点，而是建立一套自己的技术雷达和学习系统。

这一章讨论：大模型工程师如何在快速变化中保持判断力，如何筛选值得学的技术，如何把学习转化为项目能力和职业竞争力。

65.1 不要把“知道新名词”当成学习

大模型领域最容易制造名词焦虑。

今天有人讨论新的 reasoning model，明天有人讨论新的 Agent 框架，后天又出现新的推理优化、上下文压缩、多模态生成、MCP 工具生态、RAG 变种和评测基准。

如果学习目标只是“我听说过这个词”，那么学习会变成无止境的信息消费。

你可能知道很多名词：

- MoE。
- LoRA。
- DPO。

- GRPO。
- RAG。
- rerank。
- speculative decoding。
- function calling。
- MCP。
- Agent memory。
- long context。
- verifier。
- test-time scaling。

但面试或真实工作中，别人真正关心的不是你是否听过，而是你能否回答：

- 它解决什么问题？
- 它适合什么场景？
- 它和旧方法相比改变了什么？
- 它的成本和风险是什么？
- 它如何落到工程系统里？
- 如何验证它真的有效？

所以，学习新技术时不要停在“知道”。至少要进入“能解释、能判断、能复现、能应用”中的某一层。

65.2 建立四层学习目标

大模型工程师不可能对所有方向都深入。

更合理的方式是把学习目标分成四层。

第一层是知道。

你知道某个技术存在，大概知道它属于哪个方向，能在需要时进一步查资料。

例如你是应用工程师，可以知道 speculative decoding 是推理加速方法，但不一定马上掌握内核实现。

第二层是理解。

你能讲清它的基本原理、适用场景和主要限制。

例如你能解释为什么 rerank 可以改善 RAG 证据排序，为什么它会增加延迟，为什么需要结合召回质量一起看。

第三层是使用。

你能在项目中正确使用它，并处理常见问题。

例如你不仅会调用 embedding 模型，还知道如何选择 chunk 大小、如何评估召回、如何处理 metadata、如何做失败 case 回流。

第四层是改造。

你能根据业务和系统约束改造它，甚至形成团队方法论。

例如你不只是使用一个 Agent 框架，而是能设计工具权限、执行确认、审计 trace、回滚机制和评测门禁。

不同知识点应该放在不同层级。

对主线方向，至少要达到使用和改造。

对辅线方向，达到理解即可。

对远离当前工作的方向，知道它存在就够了。

学习焦虑往往来自一个错误期待：以为所有东西都必须深入掌握。

这既不现实，也没有必要。

65.3 技术雷达：把新技术放进坐标系

面对新技术，第一步不是马上学，而是给它找位置。

可以建立一个简单的技术雷达，把大模型相关知识分成几类：

- 模型架构：Transformer、MoE、长上下文、多模态、reasoning model。
- 训练与对齐：预训练、SFT、偏好优化、RL、合成数据。
- 推理与 serving：batching、KV cache、量化、并行、路由、缓存。
- 应用工程：RAG、Agent、工具调用、工作流、上下文管理。
- 评测与安全：benchmark、红队、幻觉检测、权限、审计、发布门禁。
- 数据工程：数据采集、清洗、标注、去重、质量评估、反馈回流。
- 产品与商业化：场景选择、ROI、用户体验、成本控制、落地路径。

每当遇到一个新概念，先问：它属于哪一类？它影响哪条链路？

例如 MCP 不是一个孤立名词，它属于工具协议和 Agent 生态，影响的是模型与外部工具、资源、上下文之间的连接方式。

例如 long context 不只是模型能力，它会影响应用的上下文组织、推理成本、延迟、召回策略和评测方式。

例如 reasoning model 不只是“会更思考”，它会影响问题拆解、验证器设计、推理时计算成本、任务选择和失败模式分析。

把技术放入坐标系后，你就不容易被标题牵着走。

65.4 判断一项技术是否值得深入

不是所有新技术都值得立刻深入。

可以用下面几个问题判断。

第一，它是否解决真实痛点？

如果一个方法只是在 benchmark 上好看，但你的项目痛点是数据质量、权限控制或上线稳定性，那么它未必值得优先学习。

第二，它是否影响你的主线能力？

如果你的主线是 RAG 应用工程，那么文档解析、chunk、embedding、rerank、评测、引用验证、反馈回流都比追每一个新模型更重要。

如果你的主线是推理 infra，那么 continuous batching、KV cache、量化、显存管理、压测和调度就应该优先。

第三，它是否已经进入工程可用阶段？

有些技术在论文或 demo 阶段很吸引人，但工程成本很高、依赖条件苛刻、复现难度大。你可以关注，但不一定马上投入大量时间。

第四，它是否改变了系统边界？

真正值得关注的技术，往往会改变系统设计方式。

例如 function calling 改变了模型和工具的关系；长上下文改变了部分检索和上下文管理方式；多模态模型改变了文档理解和交互形态；推理加速改变了成本结构。

第五，它是否有可验证收益？

学习一项技术后，最好能设计一个小实验验证它的价值。

如果完全无法验证，只能停留在概念层，就不要把它当作核心能力。

65.5 每周学习节奏：少而稳定

持续学习不需要每天都投入很长时间。

更有效的是少而稳定。

一个可执行的节奏是：

- 每周选 1 到 2 个新主题。
- 每个主题只做一页笔记。
- 每周精读 1 篇论文、技术博客或源码文档。
- 每周做 1 个小实验或复现。
- 每月整理一次技术雷达。
- 每季度复盘一次主线能力。

这里的重点不是数量，而是闭环。

很多人收藏了大量文章，但没有任何输出。

更好的方式是每次学习后留下最小成果：

- 一页概念笔记。
- 一个对比表。
- 一个可运行 demo。
- 一个失败 case 分析。
- 一个评测脚本。
- 一个排障 checklist。
- 一段可以放进项目文档的说明。

学习成果越接近真实项目，价值越高。

如果你本周学了 rerank，最好不要只写“rerank 可以提升排序质量”。更好的输出是：找一个小型知识库，构造 20 个问题，对比 only embedding 和 embedding + rerank 的证据命中率、延迟和失败类型。

如果你本周学了 function calling，最好不要只写“模型可以调用函数”。更好的输出是：实现一个带参数校验、错误处理、权限检查和调用日志的工具调用 demo。

65.6 如何读论文：先抓问题，再看方法

很多工程师读论文时容易陷入细节。

一上来就看公式、模型结构、实验表格，很快就觉得困难。

更好的顺序是先抓问题。

读一篇论文时，可以按下面顺序做笔记：

1. 这篇论文要解决什么问题？
2. 为什么旧方法不够好？
3. 它的核心假设是什么？
4. 它提出了什么关键机制？
5. 实验如何证明有效？
6. 它在哪些场景可能失效？
7. 如果放进工程系统，成本是什么？

例如读一篇关于长上下文的论文，不要只记模型能支持多少 token。

更重要的是理解：

- 长上下文要解决什么任务？
- 模型是否真的使用了远距离信息？
- 注意力或位置编码如何扩展？
- 推理成本如何变化？
- benchmark 是否能代表真实业务？

- 在 RAG 场景中，它是替代检索，还是补充检索？

读论文不是为了背论文，而是为了获得判断框架。

面试中，如果你能讲清一篇论文的背景、假设、收益、限制和工程影响，比背出更多细节更有价值。

65.7 如何读源码：从运行路径开始

源码学习也容易走偏。

很多人打开一个框架源码，从目录结构开始看，最后迷失在大量抽象里。

更有效的方式是从一个运行路径切入。

例如你想理解一个 Agent 框架，不要先看所有模块。可以先跑一个最小示例，然后追踪：

- 用户输入如何进入系统。
- prompt 如何构造。
- 工具 schema 如何注册。
- 模型输出如何解析。
- 工具如何被调用。
- 错误如何处理。
- trace 如何记录。
- 最终结果如何返回。

如果你想理解推理引擎，可以从一个请求开始追踪：

- 请求如何排队。
- token 如何被调度。
- batch 如何形成。
- KV cache 如何分配。
- 解码循环如何执行。
- 流式输出如何返回。
- 超时和取消如何处理。

如果你想理解 RAG 框架，可以从一次问答开始追踪：

- query 如何改写。
- embedding 如何生成。
- 向量检索如何执行。
- metadata 如何过滤。
- rerank 如何排序。
- context 如何拼接。
- 引用如何生成。
- 失败 case 如何记录。

源码学习的目标不是把每个文件都看完，而是掌握关键路径和扩展点。

65.8 如何做小实验：控制变量

大模型学习必须做实验。

但实验不是随便跑几个 demo。

一个有效的小实验至少要有三个要素：问题、变量、指标。

例如你想研究 chunk size 对 RAG 的影响。

问题是：不同 chunk size 是否影响证据命中率和回答质量？

变量是：chunk size、overlap、是否保留标题路径。

指标是：Top-K 证据命中率、引用完整性、回答正确率、平均延迟、token 成本。

如果同时更换 embedding、rerank、prompt 和 chunk 策略，就无法知道改进来自哪里。

再比如你想评估一个更强模型是否值得上线。

问题是：更强模型是否在关键业务问题上显著提升？

变量是：模型版本。

指标是：正确率、拒答准确率、幻觉率、平均成本、P95 延迟、用户反馈。

控制变量的能力，是大模型工程师区别于“调参碰运气”的关键。

实验结果也要记录失败。

很多失败实验非常有价值，因为它们说明某条路不值得继续投入。

65.9 学习要绑定真实项目

最好的学习来自真实问题。

如果你当前在做 RAG，就围绕 RAG 建立学习主线。

不要今天看训练框架，明天看推理内核，后天看 Agent 协议，结果当前项目的问题没有任何改善。

项目驱动学习可以这样做：

- 先列出当前项目最痛的 3 个问题。
- 为每个问题找 2 到 3 个可能方向。
- 每周只验证其中一个方向。
- 把实验结果回填到项目文档。
- 把有效方案沉淀成 checklist 或工具。

例如当前项目的痛点是企业知识库问答引用不准。

学习路径可以是：

- 文档解析质量。
- 标题路径和段落结构保留。
- chunk 策略。
- query 改写。
- hybrid search。
- rerank。
- 引用生成与校验。
- 失败 case 分类。

这比泛泛地学“RAG 全家桶”更有效。

因为你每学一个点，都能立刻验证它是否改善项目。

65.10 面试前如何补齐知识短板

面试准备阶段，学习方式要更有针对性。

不要试图重新学习整个大模型领域。

更合理的是围绕岗位要求和项目表达补短板。

第一，拆岗位 JD。

把 JD 中的要求分成几类：模型、训练、推理、RAG、Agent、评测、工程、业务。

第二，标记自己的强弱项。

哪些是能深入讲项目的，哪些只是了解，哪些完全空白。

第三，优先补和项目相关的短板。

如果你的项目是 RAG，但你讲不清 embedding、召回、rerank、评测，那应该优先补这些，而不是先去学复杂的 RLHF。

第四，为每个短板准备一个最小闭环。

例如：

- 学 embedding：准备向量检索、相似度、召回失败原因、embedding 模型选择。
- 学 rerank：准备为什么需要 rerank、延迟成本、评测指标、失败案例。
- 学 Agent：准备工具 schema、权限、幂等、错误恢复、trace、评测。
- 学推理：准备吞吐、首 token 延迟、P95、batching、KV cache、成本。

第五，用面试问题反向检验。

如果你能回答“为什么这样做”“有没有别的方案”“怎么评估”“线上出问题怎么排查”，说明短板已经从名词层进入工程层。

65.11 不同方向的学习优先级

不同工程师的学习重点不一样。

应用工程方向可以优先关注：

- RAG 质量链路。
- Agent 工具调用和权限。
- 多轮对话和上下文管理。
- 评测和日志回流。
- 业务流程建模。
- 成本、延迟和安全边界。

模型算法方向可以优先关注：

- 训练数据构造。
- SFT 和偏好优化。
- reasoning 数据和验证器。
- 模型评测和误差分析。
- 多任务能力和副作用。
- 论文复现和实验设计。

推理基础设施方向可以优先关注：

- 推理引擎结构。
- batch 调度。
- KV cache。
- 量化。
- 并行策略。
- GPU 利用率。
- 服务稳定性和容量规划。

评测安全方向可以优先关注：

- 评测集设计。
- 指标和真实体验的差异。
- 红队测试。
- 越权和数据泄漏。
- Agent 高风险操作控制。
- 发布门禁和审计。

这些方向不是互斥的。

但在一个阶段内，最好有主线和辅线。

主线保证深度，辅线保证协作。

65.12 如何避免信息流污染

大模型领域的信息流质量参差不齐。

有些文章是认真复现，有些只是改写论文摘要，有些是营销内容，有些标题非常夸张。

过滤信息时可以看几个信号。

高质量内容通常会包含：

- 清楚的问题背景。
- 可复现的实验设置。
- 对 baseline 的比较。
- 失败案例。
- 成本和限制。
- 工程落地细节。
- 对适用场景的边界说明。

低质量内容常见特征是：

- 只堆名词。
- 只给结论不给证据。
- 把 demo 当成产品。
- 把 benchmark 当成全部能力。
- 不讨论成本、延迟和失败。
- 用“颠覆”“终结”“完全替代”这类绝对化表达。

不要让信息流决定你的学习计划。

信息流可以提供线索，但学习计划应该由你的主线能力、当前项目和职业目标决定。

65.13 把学习沉淀成可复用资产

学习如果不沉淀，很快会消失。

建议建立自己的学习资产库。

不需要复杂工具，一个目录或笔记本就可以。

可以包括：

- 论文笔记。
- 源码阅读笔记。
- 实验记录。
- 项目复盘。
- 面试问答。
- 技术方案对比。
- 排障 checklist。
- 常见失败 case。
- 自己实现的小工具。

每条记录尽量包含四个部分：

1. 背景：为什么看这个问题？
2. 结论：学到了什么？
3. 证据：有什么实验、代码或案例？
4. 复用：下次遇到什么场景可以用？

这样的沉淀会逐渐形成你的个人技术资产。

几个月后，它不仅能帮助你工作，也能帮助你准备面试、写晋升材料和做技术分享。

65.14 学习的最终目标是提高判断力

很多人以为学习是为了掌握更多技术。

这只能说对了一半。

对大模型工程师来说，学习的最终目标是提高判断力。

判断力包括：

- 知道什么时候该换模型，什么时候该修数据。
- 知道什么时候该加 Agent，什么时候只需要流程自动化。
- 知道什么时候该上长上下文，什么时候该优化检索。
- 知道什么时候该做微调，什么时候 prompt 和数据就够了。
- 知道什么时候该追新框架，什么时候该保持简单。
- 知道什么时候指标提升可信，什么时候可能是评测泄漏。
- 知道什么时候 demo 值得产品化，什么时候应该止损。

判断力来自持续学习，但更来自真实项目、实验、失败和复盘。

如果一个工程师每次学习新技术后，只是多了一个名词，那么成长很有限。

如果他能因此更准确地定义问题、选择方案、控制风险、验证结果，那么学习就真正转化成了工程能力。

65.15 小结

大模型领域变化快，但不需要追逐所有热点。

有效学习的关键，是建立技术雷达，把新技术放进模型架构、训练对齐、推理 serving、应用工程、评测安全、数据工程和产品落地等坐标系中。

对不同知识点，要设置不同学习层级：知道、理解、使用、改造。主线方向要深入，辅线方向要能协作，远离当前目标的方向只需保持感知。

学习应该少而稳定，并尽量绑定真实项目。论文阅读要先抓问题，源码阅读要从运行路径开始，小实验要控制变量。

真正有价值的学习输出，不是收藏链接，而是笔记、实验、demo、评测脚本、失败 case、checklist 和项目复盘。

最终，大模型工程师持续学习的目的不是追热点，而是在快速变化中持续提高判断力。

第 66 章：入职后的工程习惯与长期可信度建设

前几章讨论了面试、入职、职业成长和持续学习。

但真正决定一个大模型工程师能否长期走远的，不只是知识储备，也不只是某一次项目表现，而是日常工程习惯。

在真实团队中，一个人是否值得信任，往往不是通过一次漂亮演示建立的，而是在一次次需求澄清、方案设计、代码提交、问题排查、上线验证和复盘建立的。

大模型工程尤其如此。

因为它天然包含很多不确定性：模型输出不稳定，评测不完全可靠，用户问题长尾，成本和延迟敏感，安全边界复杂，线上问题往往难以复现。

如果工程习惯不好，一个看起来很强的模型应用也可能很快变成难维护、难排查、难迭代的系统。

这一章讨论：入职后如何通过日常工程习惯建立长期可信度。

66.1 可信度来自稳定交付，而不是一次爆发

很多新人入职后希望快速证明自己，于是容易追求“大动作”：重构系统、替换框架、引入新模型、设计复杂架构。

这种心态可以理解，但风险也很高。

团队真正需要的是稳定交付。

稳定交付包括：

- 需求理解准确。
- 方案范围清楚。
- 进度预期可靠。
- 代码质量可维护。
- 风险提前暴露。
- 上线前验证充分。
- 出问题能快速定位。
- 复盘后能减少同类问题。

如果你每次都能把小需求做稳，团队会逐渐把更关键的任务交给你。

如果你偶尔做出一个很炫的 demo，但经常延期、遗漏边界、上线后难排查，团队对你的信任会下降。

大模型工程师尤其不能只靠 demo 建立信誉。

demo 能证明可能性，生产系统要证明可靠性。

66.2 接需求时先确认问题，而不是急着给方案

很多工程问题一开始并不清楚。

产品或业务方可能会说：

- “我们想做一个智能客服。”
- “希望回答更准确。”
- “能不能加一个 Agent？”
- “这个模型效果不好，换一个更强的。”
- “用户希望系统能自动处理工单。”

如果工程师立刻进入实现，很容易做偏。

正确的第一步是确认问题。

可以问：

- 当前具体失败案例是什么？
- 用户最不能接受的问题是哪类？
- 是回答错误、证据错误、拒答错误，还是流程失败？
- 有没有线上日志、用户反馈或人工标注样本？
- 目标是提升准确率、降低成本、减少人工、提高响应速度，还是改善体验？
- 成功标准是什么？
- 有哪些不能触碰的安全、合规或业务边界？

大模型项目里，“效果不好”不是一个可执行需求。

它需要被拆成可定位、可验证、可复盘的问题。

例如知识库问答效果不好，可能来自文档解析失败、chunk 切分不合理、召回不准、rerank 排序错误、prompt 指令冲突、模型不适合、引用生成错误或评测集偏差。

如果不先定位问题，直接换模型或加 Agent，只是在增加复杂度。

66.3 写方案时明确范围、假设和验证方式

可靠的工程师写方案，不只是描述“我要做什么”，还会说明“我不做什么”“我基于什么假设”“我如何证明有效”。

一个简洁但有效的方案可以包含：

- 背景：为什么要做？
- 目标：要改善什么指标或体验？
- 非目标：本次明确不解决什么？
- 方案：核心改动是什么？
- 影响范围：涉及哪些服务、数据、接口和流程？
- 风险：可能出什么问题？
- 验证：如何离线评测、灰度和线上观察？
- 回滚：失败后如何恢复？

非目标非常重要。

没有非目标，需求会不断膨胀，工程师会陷入模糊承诺。

例如“优化 RAG 效果”这个需求，可以明确本次只优化文档解析和 chunk 策略，不涉及模型替换、不涉及权限系统、不涉及多轮对话记忆。

假设也很重要。

例如你认为效果差主要来自召回不足，就应该说明这个判断来自哪些失败案例或统计数据。

如果后续实验发现假设不成立，就要及时调整，而不是继续把原方案做完。

66.4 进度沟通要暴露不确定性

很多工程师害怕暴露不确定性。

他们觉得说“不确定”“可能有风险”“需要先验证”会显得自己不够强。

实际情况相反。

在复杂项目中，能提前识别不确定性，是成熟工程师的重要标志。

进度沟通不要只说：

- “快好了。”
- “差不多了。”
- “应该没问题。”

这些说法没有信息量。

更好的表达是：

- “接口改造已完成，离线评测还没跑完，主要风险是 P95 延迟可能超过目标。”
- “当前准确率提升明显，但引用错误率没有下降，明天会先定位引用链路。”
- “方案 A 已验证不可行，因为召回阶段丢失了关键表格信息，准备转向文档解析优化。”
- “上线前还缺压测和回滚验证，如果今天要发版，风险偏高。”

这样的沟通会让团队知道真实状态，也能更早做决策。

可信度不是来自永远报喜，而是来自状态透明。

66.5 代码提交要让人容易 review

大模型工程中，代码变更往往横跨 prompt、检索、模型调用、评测、配置、日志和前后端接口。

如果提交混乱，review 成本会非常高。

好的提交习惯包括：

- 一个提交解决一个清晰问题。
- 避免把重构、格式化和功能改动混在一起。
- PR 描述说明背景、改动、验证和风险。
- 对 prompt、阈值、模型参数等变更说明原因。
- 对评测结果给出样本和指标，而不是只说“效果更好”。
- 对高风险路径给出回滚方式。

大模型系统里，prompt 改动也是代码改动。

一个 prompt 多加一句话，可能影响拒答、引用、工具调用、输出格式和安全边界。

所以 prompt 变更也要 review，也要验证，也要记录原因。

不要把 prompt 当作临时文本。

它是生产行为的一部分。

66.6 日志和 trace 是工程生命线

大模型系统如果没有日志和 trace，线上问题会非常难查。

用户说“它答错了”，你需要知道：

- 用户原始输入是什么？
- 系统做了哪些 query rewrite？
- 检索到了哪些文档？
- rerank 分数是多少？
- 最终传给模型的上下文是什么？
- 使用了哪个模型和参数？
- 模型原始输出是什么？
- 是否触发了工具调用？
- 工具返回了什么？
- 是否经过安全过滤或格式修复？
- 最终返回给用户的内容是什么？

没有这些信息，排查只能靠猜。

日志和 trace 的设计要注意两点。

第一，足够定位问题。

关键链路必须可追踪，尤其是检索、prompt、工具调用、模型输出、后处理和错误分支。

第二，避免泄漏敏感信息。

用户隐私、企业文档、密钥、权限数据不能随意写入日志。必要时要脱敏、采样、加权限控制和保留周期。

高级工程师不会等线上事故发生后才补日志。

他会在设计系统时就考虑可观测性。

66.7 评测要进入日常开发流程

很多团队的大模型项目失败，不是因为没调 prompt，而是因为没有稳定评测。

没有评测，所有优化都靠主观感受。

今天看起来变好了，明天另一个场景可能变差。

日常开发中至少应该有三类评测。

第一，回归评测。

每次改 prompt、模型、检索策略或工具逻辑，都要检查核心样本是否退化。

第二，目标评测。

针对本次需求定义指标，例如引用准确率、工单分类准确率、工具调用成功率、拒答准确率、延迟和成本。

第三，失败案例评测。

把线上出现的典型失败 case 回收进评测集，防止同类问题反复出现。

评测不一定一开始就很复杂。

早期可以是几十个高质量样本、人工标注标准和一个可重复运行的脚本。

比起追求完美评测，更重要的是让评测成为习惯。

66.8 上线前要检查失败路径

上线前不要只验证成功路径。

大模型系统有很多失败路径：

- 模型超时。
- 模型限流。
- 检索无结果。
- 检索结果权限不足。
- 工具调用参数错误。
- 工具执行失败。
- 模型输出格式不合法。
- 多轮对话上下文过长。
- 用户输入包含攻击指令。
- 外部服务返回异常。
- 成本突然升高。

很多线上事故不是主流程不会跑，而是失败路径没有处理。

上线前可以准备一个简单 checklist：

- 超时怎么办？
- 重试几次？
- 是否会重复执行副作用操作？
- 用户看到什么错误提示？
- 是否记录足够日志？
- 是否会泄漏内部信息？
- 是否有降级方案？
- 是否能快速回滚？

Agent 类系统尤其要注意副作用。

如果工具会发邮件、改数据库、下订单、提交工单，就不能简单重试，也不能让模型自由决定所有高风险动作。

需要有幂等、确认、权限和审计。

66.9 复盘不是追责，而是改系统

线上问题发生后，团队通常会复盘。

低质量复盘会停留在：

- “下次注意。”
- “加强测试。”
- “开发不够仔细。”

这些结论很难防止问题复发。

高质量复盘关注系统性改进。

可以问：

- 问题为什么没有在开发阶段发现？
- 评测集为什么没有覆盖？
- 日志为什么不足以快速定位？
- 灰度为什么没有拦住？
- 告警为什么没有及时触发？
- 回滚为什么不够快？

- 文档或 checklist 是否缺失？
- 哪个流程可以自动化？

复盘的产出应该是具体动作。

例如新增一个回归样本、补一个日志字段、增加一个发布门禁、修改一个工具权限、完善一个异常处理分支。

真正成熟的团队，会把事故转化为系统韧性。

真正成熟的工程师，会在复盘中推动机制改进，而不是只解释自己为什么没错。

66.10 文档是长期协作工具

很多工程师不喜欢写文档，觉得写文档浪费时间。

但在大模型工程中，文档非常重要。

因为系统行为不仅由代码决定，还由模型、prompt、数据、评测集、配置、阈值、工具权限和业务规则共同决定。

如果这些信息只存在于某个人脑子里，系统就很难维护。

有价值的文档包括：

- 系统架构图。
- 请求链路说明。
- prompt 设计说明。
- 模型和参数选择依据。
- 数据来源和清洗规则。
- 评测集构造方法。
- 指标定义。
- 上线和回滚流程。
- 常见问题排查手册。
- 重要设计决策记录。

文档不需要一开始很长。

关键是及时、准确、可更新。

如果一个需求做完后，别人无法通过文档理解系统行为，那么交付是不完整的。

66.11 不要过度工程，也不要欠债不还

工程习惯的难点在于平衡。

过度工程会让项目变慢。

技术债太多会让项目变脆。

大模型项目早期需要快速试错，因此不可能一开始就设计完美架构。

但也不能把所有 demo 代码直接带到生产。

可以用一个简单判断标准：

- 探索阶段，优先验证问题是否成立。
- 试点阶段，补齐基本日志、评测、权限和异常处理。
- 生产阶段，补齐稳定性、可观测性、回滚、成本控制和边界。

不同阶段的工程标准不同。

问题不在于早期写了临时代码，而在于临时代码进入生产后没有被治理。

成熟工程师会明确标记技术债，并在合适时机偿还。

例如为了快速验证先写死一个 prompt 可以接受，但上线前应该把版本、变更记录、评测结果和回滚方式补齐。

66.12 建立跨角色信任

大模型工程师通常要和很多角色协作：产品、算法、后端、前端、数据、测试、运维、安全、法务和业务专家。

长期可信度不只来自代码，也来自跨角色协作。

对产品，要能把技术不确定性转化为可决策的信息。

例如说明某个需求需要先做数据验证，而不是直接承诺上线时间。

对算法，要能提供高质量失败案例、日志和评测结果，而不是只说“模型不行”。

对数据团队，要能说明需要什么字段、标注标准和质量检查，而不是只说“数据越多越好”。

对运维和平台，要能说明容量、限流、监控、告警和回滚要求。

对安全和法务，要能说明数据流、权限边界、日志保留和高风险操作控制。

一个高级工程师的价值，往往体现在他能让不同角色围绕同一个问题形成共识。

66.13 个人可信度的负面清单

有些行为会快速消耗团队信任。

需要主动避免。

例如：

- 没弄清需求就承诺。
- 明知有风险但不说。
- 用“模型就是这样”解释所有问题。
- 没有评测就宣称效果提升。
- 改 prompt 不记录原因。
- 线上问题没有日志可查。
- 把 demo 代码当生产代码。
- 遇到事故只解释责任，不推动改进。
- 经常低估工作量。
- 频繁引入新框架但不负责维护。
- 不写文档，让系统依赖个人记忆。

这些问题单次看可能不严重，但长期会形成标签。

一旦团队认为你“不稳”，后续再证明自己会更难。

66.14 高可信工程师的日常表现

高可信工程师并不一定每天都做惊天动地的事情。

他的日常表现通常很朴素：

- 接需求时先澄清问题。
- 设计方案时明确目标和非目标。
- 开发时保持改动可 review。
- 改模型、prompt、检索策略时有评测。
- 上线前检查失败路径。
- 线上问题能通过 trace 定位。
- 风险会提前同步。
- 复盘会推动机制改进。
- 文档能让别人接手。
- 学到的新东西会转化为团队资产。

这些习惯短期看不显眼，但长期会形成很强的职业复利。

团队会更愿意让你负责关键系统，也更愿意听你的技术判断。

66.15 小结

入职后的长期成长，不只取决于技术知识，也取决于工程习惯。

大模型系统充满不确定性，因此更需要需求澄清、方案边界、评测、日志、trace、失败路径处理、上线验证、复盘和文档。

可信度来自稳定交付和状态透明，而不是一次漂亮 demo。

成熟的大模型工程师，会把每次开发都当作建立信任的机会：问题定义清楚，方案可验证，风险可见，代码可维护，线上可观测，失败可复盘。

长期来看，这些习惯会比单个技术点更能决定一个工程师的上限。

第 67 章：把项目经验沉淀成作品、晋升材料和职业资本

前面几章讨论了面试、入职、成长、持续学习和工程习惯。

这一章继续往后看一个更长期的问题：工作中的项目经验如何沉淀下来。

很多工程师做过不少项目，但几年后回头看，能讲清楚的很少。

代码交付了，需求上线了，问题解决了，但经验没有形成资产。

等到写晋升材料、准备跳槽、做技术分享或带新人时，才发现自己只记得一些零散片段。

大模型工程师尤其容易遇到这个问题。

因为项目里有大量隐性工作：失败 case 分析、prompt 调整、数据清洗、评测集构建、模型对比、成本优化、线上排障、安全边界设计、跨团队协作。这些工作如果不记录，很难在事后完整复原。

真正成熟的工程师，不只是完成项目，还会把项目沉淀成可复用资产。

这些资产会在三个场景中发挥作用：晋升、跳槽和长期职业品牌。

67.1 项目做完不等于经验沉淀完成

很多人以为项目上线就是结束。

但从个人成长角度看，项目上线只是第一步。

如果没有复盘和沉淀，项目经验会快速流失。

几个月后你可能只记得：

- 做过一个 RAG 系统。
- 接过一个客服 Agent。
- 优化过推理成本。
- 调过一个 prompt。
- 接入过一个模型服务。

这些描述太粗糙，无法体现工程能力。

真正有价值的是更具体的信息：

- 当时的业务问题是什么？
- 为什么旧方案不够好？
- 你负责了哪部分？
- 有哪些关键技术决策？
- 做过哪些实验？
- 哪些方案失败了？
- 最后指标提升多少？
- 成本、延迟、稳定性是否改善？
- 上线后遇到什么问题？

- 你沉淀了什么机制？

项目经验不是自然积累的。

项目经验需要被结构化记录、抽象和表达。

67.2 什么是职业资产

职业资产不是简历上的一句话。

它是你能反复复用、能证明能力、能迁移到新场景的成果。

对大模型工程师来说，职业资产可以包括：

- 一个完整项目复盘。
- 一套评测方法。
- 一个线上问题排查案例。
- 一个成本优化方案。
- 一个 Agent 安全控制设计。
- 一组高质量失败 case 分类。
- 一个可运行 demo。
- 一篇内部技术文档。
- 一次技术分享。
- 一个开源项目或工具。
- 一套面试项目表达模板。

职业资产有两个特点。

第一，它可以证明你做过真实工作。

不是只说“我了解 RAG”，而是能展示你如何解决召回不准、引用错误、延迟过高和权限过滤问题。

第二，它可以迁移。

你在一个知识库问答项目中沉淀的评测方法，也可以迁移到客服问答、文档助手、代码助手或销售助手中。

一个项目如果只完成交付，没有沉淀成资产，它对职业成长的贡献会小很多。

67.3 每个项目都应该留下项目卡片

建议为每个重要项目写一张“项目卡片”。

它不需要很长，但要结构清楚。

可以包含以下部分：

- 项目背景：为什么要做？
- 业务目标：要解决什么问题？
- 技术场景：RAG、Agent、训练、推理、评测、数据还是平台？
- 你的角色：负责什么，不负责什么？
- 核心难点：最难的 2 到 3 个问题是什么？
- 方案设计：关键架构和决策是什么？
- 实验过程：比较过哪些方案？
- 指标结果：上线前后有什么变化？
- 线上问题：遇到过什么事故或失败案例？
- 复盘沉淀：留下了什么工具、文档、流程或经验？

项目卡片最好在项目刚结束时写。

因为这时细节还清楚。

如果等半年后再写，很多关键数据、失败案例和技术取舍都会忘记。

项目卡片不是流水账，而是能力证据。

它以后可以变成简历项目、晋升材料、面试回答、技术分享或内部培训内容。

67.4 记录指标，不要只记录感受

大模型项目很容易用感受描述结果。

例如：

- “效果明显提升。”
- “用户反馈更好了。”
- “延迟优化了很多。”
- “成本降低不少。”

这些说法在复盘、晋升和面试中都不够有力。

更好的方式是记录指标。

指标可以包括：

- 正确率。
- 召回命中率。
- 引用准确率。
- 工具调用成功率。
- 拒答准确率。
- 幻觉率。
- 人工接管率。
- 用户满意度。
- 首 token 延迟。
- P95 延迟。
- 单次请求成本。
- 缓存命中率。
- GPU 利用率。
- 故障率。

不是每个项目都有完美指标。

但至少记录当时能获得的证据。

例如：“在 200 条人工标注样本上，引用准确率从 71% 提升到 86%，P95 延迟从 4.8 秒增加到 5.3 秒，最终通过缓存和并发优化降到 4.1 秒。”

这比“效果提升明显”更可信。

如果没有指标，也要记录为什么没有，以及后来是否补了评测机制。

67.5 失败方案也是资产

很多人只记录成功方案。

但在大模型工程中，失败方案非常有价值。

因为大模型项目充满试错，真正的经验往往来自失败。

例如：

- 换更大模型后准确率提升不明显，但成本翻倍。
- 加长上下文后召回问题没有解决，反而延迟升高。
- Agent 自动规划在 demo 中很好，但线上稳定性不足。
- 微调后特定任务提升，但通用问答能力下降。
- rerank 提升证据排序，但对短 query 效果有限。
- prompt 加强格式约束后，拒答率异常升高。

这些失败经验能体现判断力。

面试官或晋升评委并不只想听你做成了什么，也想知道你如何识别错误方向、如何止损、如何调整方案。

记录失败方案时，可以写：

- 目标是什么？
- 假设是什么？
- 怎么验证？
- 结果为什么不符合预期？
- 最后如何调整？
- 对后续项目有什么启发？

一个工程师如果能清楚讲出失败方案，往往比只讲成功故事更可信。

67.6 把项目抽象成可迁移能力

项目本身可能过时，但能力可以迁移。

例如你做过一个企业知识库问答系统。

表面上看，这是一个 RAG 项目。

但可以抽象出很多能力：

- 文档解析和结构化能力。
- 检索链路设计能力。
- 评测集构建能力。
- 引用和证据校验能力。
- 权限过滤能力。
- 延迟和成本优化能力。
- 线上失败 case 分析能力。
- 跨业务专家协作能力。

这些能力可以迁移到合同审查、客服助手、销售知识库、代码文档问答和内部流程助手。

再比如你做过一个工单 Agent。

它可以抽象为：

- 任务拆解能力。
- 工具 schema 设计能力。
- 参数校验能力。
- 幂等和重试能力。
- 高风险操作确认能力。
- trace 和审计能力。
- 人工接管机制设计能力。

抽象能力的好处是，你不会被某个具体项目限制。

当别人问“你能不能做另一个场景”时，你可以解释哪些能力可以复用，哪些需要重新验证。

67.7 晋升材料要讲影响力，不只是工作量

晋升材料常见问题是堆工作量。

例如：

- 完成了 10 个需求。
- 接入了 3 个模型。
- 写了 5 个服务。
- 优化了多个 prompt。

这些信息有用，但不够。

晋升更关心影响力。

影响力包括：

- 业务结果：节省多少人工、提升多少转化、减少多少错误。
- 技术结果：提高稳定性、降低成本、提升可维护性。
- 团队结果：沉淀工具、流程、文档，让别人效率提高。
- 长期结果：建立评测、监控、发布门禁，减少重复事故。

一个更好的表达是：

“负责企业知识库问答质量优化，定位主要瓶颈来自文档解析和证据排序。通过重构解析链路、引入标题路径保留和 rerank 评测机制，在 300 条业务标注样本上将证据命中率从 68% 提升到 84%，引用错误率下降 35%。同时沉淀了 RAG 回归评测脚本和失败 case 分类，被后续 3 个业务线复用。”

这段表达包含问题、动作、结果和复用。

比“优化了 RAG 效果”强得多。

67.8 简历项目要讲清边界

写简历时，很多人喜欢把项目写得很大。

但如果边界不清，面试时很容易被追问穿。

可信的简历项目应该清楚说明：

- 项目规模。
- 你的职责。
- 技术难点。
- 关键方案。
- 量化结果。
- 你的独立贡献。

不要把团队成果全部写成个人成果。

例如整个团队做了模型训练、服务部署、业务接入和前端产品，而你主要负责 RAG 评测与检索优化，就应该聚焦这部分。

面试官并不要求你负责所有事情。

他们更关心你负责的部分是否真实、深入、可验证。

边界清楚反而更可信。

如果你确实参与了跨模块协作，也可以写“参与”“协同”“推动”，但不要混淆成“独立负责”。

67.9 面试表达要从故事变成结构

项目面试不是讲故事会。

好的项目表达应该有结构。

可以使用一个简单框架：

1. 背景：业务为什么需要这个系统？
2. 问题：当时最核心的问题是什么？
3. 方案：你设计或实现了什么？
4. 难点：遇到哪些关键挑战？
5. 验证：如何证明方案有效？
6. 结果：指标和业务结果是什么？
7. 复盘：如果重做，会改进什么？

大模型项目尤其要准备追问。

例如 RAG 项目可能被追问：

- 为什么不用长上下文直接塞全文？
- chunk size 怎么选？
- embedding 模型怎么评估？
- rerank 为什么有效？
- 如何处理权限过滤？
- 引用准确率怎么评估？
- 用户问法变化怎么处理？
- 线上幻觉怎么定位？

Agent 项目可能被追问：

- 工具 schema 如何设计？
- 模型选错工具怎么办？
- 参数错误怎么办？
- 高风险操作如何确认？
- 如何防止重复执行？
- trace 怎么记录？
- 如何评测 Agent 成功率？

提前把项目沉淀成结构，面试时就不会临场乱讲。

67.10 内部分享是放大影响力的方式

项目经验如果只停留在个人手里，影响力有限。

把经验整理成内部分享，可以帮助团队减少重复踩坑。

分享不一定要很宏大。

可以是：

- 一次 RAG 失败 case 复盘。
- 一次 prompt 变更事故复盘。
- 一次模型评测方法分享。
- 一次 Agent 工具调用设计分享。
- 一次推理成本优化记录。
- 一次线上排障流程说明。

好的内部分享要少讲空泛概念，多讲具体案例。

例如不要只讲“RAG 很重要”，而是讲“我们为什么把 chunk 从 1000 token 改到 400 token，为什么最初效果变差，后来通过标题路径和 rerank 才提升”。

这样的分享能体现你的实践深度，也能成为晋升材料中的团队影响力证据。

67.11 开源和公开写作要注意边界

很多大模型工程师希望通过开源、博客或公开分享建立个人品牌。

这是好事，但要注意边界。

不能泄漏公司代码、数据、客户信息、内部指标、未公开架构和敏感业务细节。

更安全的方式是抽象方法论。

例如可以写：

- RAG 评测集如何设计。
- Agent 工具调用有哪些失败模式。
- prompt 变更为什么需要回归测试。

- 大模型应用如何做成本评估。
- 知识库问答如何做引用校验。

不要写：

- 具体客户数据。
- 内部接口和架构细节。
- 未公开模型效果指标。
- 公司专有数据处理流程。
- 可能暴露安全边界的信息。

公开输出的价值不在于泄漏细节，而在于展示你的思考能力和工程方法。

67.12 建立个人项目档案库

建议长期维护一个个人项目档案库。

可以按项目建立目录，保存：

- 项目卡片。
- 设计方案。
- 关键指标。
- 实验记录。
- 失败 case。
- 复盘文档。
- 分享材料。
- 简历版本。
- 面试问答。

如果公司有保密要求，私人档案中不要保存敏感数据和内部代码。

可以保存脱敏后的结构化总结。

例如不要保存真实用户输入，可以保存失败类型和抽象示例。

不要保存内部文档内容，可以保存文档解析问题的分类和处理方法。

项目档案库的价值会随着时间增加。

当你准备晋升时，它可以快速变成材料。

当你准备跳槽时，它可以快速变成简历和面试答案。

当你带新人时，它可以变成培训内容。

当你做技术选型时，它可以帮助你避免重复踩坑。

67.13 不同阶段的沉淀重点

不同职业阶段，沉淀重点不一样。

初级工程师要重点沉淀：

- 自己完成了什么模块。
- 学到了哪些工程规范。
- 遇到的问题如何解决。
- 如何通过测试和 review 提升质量。

中级工程师要重点沉淀：

- 独立负责的项目。
- 技术方案和取舍。
- 指标提升。
- 线上问题处理。

- 跨角色协作。

高级工程师要重点沉淀：

- 系统性问题定义。
- 复杂项目推进。
- 技术方向判断。
- 平台化和机制建设。
- 团队影响力。
- 可复用方法论。

资深工程师或负责人要重点沉淀：

- 技术战略。
- 组织效率提升。
- 多团队协同。
- 风险治理。
- 人才培养。
- 长期技术资产建设。

沉淀内容要和你的职业目标匹配。

如果你想从中级走向高级，就不能只记录自己写了多少代码，而要记录你如何定义问题、推动方案、影响团队和形成机制。

67.14 一次项目复盘示例

下面给出一个简化示例。

项目背景：企业内部知识库问答上线后，用户反馈答案看似流畅但引用不准，业务专家不信任系统。

业务目标：提升回答可信度，减少人工二次核查成本。

我的职责：负责 RAG 质量分析、检索链路优化和回归评测建设。

核心问题：失败 case 分析发现，主要问题不是模型生成能力不足，而是文档解析丢失标题层级，导致 chunk 缺少上下文；同时 embedding 召回后排序不稳定，相关证据经常排在 Top-K 之外。

方案设计：重构文档解析流程，保留标题路径、表格说明和段落层级；调整 chunk size 和 overlap；引入 rerank；建立 300 条业务问题评测集，指标包括证据命中率、引用准确率和 P95 延迟。

实验结果：证据命中率从 68% 提升到 84%，引用错误率下降 35%，P95 延迟初期从 3.9 秒升至 5.2 秒，后通过缓存和并发优化降至 4.3 秒。

失败方案：曾尝试直接换更大模型，回答流畅度提升，但引用错误没有明显下降，成本增加约 2 倍，因此没有上线。

沉淀资产：形成 RAG 失败 case 分类、回归评测脚本、文档解析 checklist 和上线前引用校验流程，被后续两个知识库项目复用。

这个示例比“负责企业知识库问答优化”更有信息密度。

它展示了问题定位、方案取舍、指标结果、失败经验和团队复用。

67.15 小结

项目经验不会自动变成职业资本。

大模型工程师需要主动把项目沉淀成项目卡片、指标记录、失败案例、评测方法、复盘文档、内部分享和可复用工具。

职业资产的价值在于它能证明真实能力，并能迁移到晋升、跳槽、面试、分享和带人场景中。

做项目时，不仅要问“这个需求有没有上线”，还要问：我是否留下了可复用的经验？我是否记录了关键指标？我是否讲得清楚技术取舍？我是否把失败转化成判断力？

长期来看，能持续沉淀项目资产的工程师，会比只完成任务的工程师成长更快，也更容易在职业竞争中形成差异化。

第 68 章：从执行者到技术负责人

前一章讨论了如何把项目经验沉淀成职业资产。

当一个工程师积累了一些项目经验后，下一步通常不是简单地做更多需求，而是开始负责一个方向、一个模块或一个子系统。

这时角色会发生变化。

你不再只是“把分配给你的任务做好”，而是要对一块技术区域的长期结果负责。

这就是从执行者到技术负责人的转变。

大模型工程中的技术负责人，不一定是正式管理者，也不一定有下属。

他可能负责 RAG 质量链路，可能负责 Agent 工具平台，可能负责推理服务稳定性，可能负责评测体系，可能负责数据闭环，可能负责某个业务 AI 产品的技术架构。

这一章讨论：如何完成这个角色转变，以及技术负责人真正需要负责什么。

68.1 执行者和技术负责人的区别

执行者关注任务完成。

技术负责人关注系统结果。

执行者常问：

- 这个需求什么时候做完？
- 接口怎么实现？
- 代码怎么改？
- 测试怎么过？
- bug 怎么修？

技术负责人还要问：

- 这个方向真正要解决什么问题？
- 当前最大瓶颈在哪里？
- 系统未来半年会不会撑不住？
- 哪些能力应该平台化？
- 哪些技术债必须尽快处理？
- 指标是否能证明业务价值？
- 团队是否有足够的评测、监控和回滚能力？
- 如果我不在，别人能否接手？

执行者可以只对局部任务负责。

技术负责人必须对端到端效果负责。

例如一个 RAG 执行者可能只负责实现 rerank 接入。

而 RAG 技术负责人要关心：文档解析是否可靠，召回是否可评估，rerank 是否值得成本，引用是否可信，权限是否正确，用户反馈是否能回流，线上失败 case 是否被纳入评测。

这就是角色差异。

68.2 技术负责人首先要定义问题

很多技术负责人失败，不是因为写代码能力不够，而是因为问题定义不清。

如果问题定义错了，后面做得越快，偏得越远。

大模型项目尤其容易被错误问题牵引。

例如业务说“我们要一个 Agent”。

真正的问题可能是流程自动化，而不是一定要让模型自主规划。

业务说“模型不够聪明”。

真正的问题可能是知识库召回不准、上下文缺失或工具返回错误。

业务说“希望回答更自然”。

真正的问题可能是用户不信任答案，因为没有引用和可追溯证据。

技术负责人要把模糊诉求转化成清楚问题。

可以用下面的方式定义问题：

- 现象是什么？
- 影响了谁？
- 频率有多高？
- 严重程度如何？
- 目前有什么证据？
- 根因假设是什么？
- 可以用什么指标验证？
- 解决后业务会得到什么收益？

问题定义越清楚，团队越容易形成共识。

68.3 技术负责人要管理技术方向，而不是追热点

大模型领域热点很多。

技术负责人如果没有判断力，很容易把团队带进频繁换方案的状态。

今天换新模型，明天换新框架，后天加 Agent，下周重构 RAG，再下周尝试微调。

结果每个方向都做了一点，但没有任何能力真正沉淀。

技术方向管理的核心是排序。

你需要判断：

- 哪些问题最值得解决？
- 哪些能力是基础设施？
- 哪些优化只是锦上添花？
- 哪些技术现在不成熟？
- 哪些项目应该止损？
- 哪些投入会带来长期复用？

例如一个企业知识库问答方向，可能同时有很多想法：换更强模型、加 rerank、做 query rewrite、支持长上下文、优化文档解析、做反馈回流、做多轮记忆、做权限过滤。

技术负责人不能简单地全部做。

他要根据当前瓶颈排序。

如果失败 case 显示大量问题来自文档解析和权限过滤，那么优先级可能不是换模型，而是先补数据链路和安全边界。

68.4 建立方向级指标体系

执行单个需求时，可以只看局部指标。

负责一个方向时，必须建立方向级指标体系。

没有指标，技术负责人就无法判断方向是否变好。

以 RAG 方向为例，指标可以分层：

- 检索层：召回命中率、Top-K 命中率、rerank 后命中率。
- 生成层：回答正确率、引用准确率、拒答准确率、幻觉率。
- 体验层：用户满意度、人工接管率、追问率。
- 性能层：首 token 延迟、P95 延迟、吞吐、错误率。
- 成本层：单次请求 token 成本、模型调用成本、缓存命中率。
- 安全层：越权召回率、敏感信息泄漏率、攻击拦截率。

以 Agent 方向为例，指标可以包括：

- 任务完成率。
- 工具选择准确率。
- 参数填写准确率。
- 工具调用成功率。
- 人工接管率。
- 高风险操作误触发率。
- 平均步骤数。
- 失败恢复率。
- 审计完整率。

指标体系不需要一开始完美。

但必须逐步建立。

技术负责人要推动团队从“感觉变好”转向“有证据地变好”。

68.5 设计架构时要考虑演进，而不是只满足当前需求

执行者实现当前需求。

技术负责人要考虑系统演进。

这并不是要求一开始做很复杂的架构。

相反，好的负责人通常会保持架构简单，但为关键变化点留出空间。

大模型系统中常见变化点包括：

- 模型会更换。
- prompt 会频繁迭代。
- embedding 模型会升级。
- 检索策略会调整。
- 工具 schema 会变化。
- 评测集会扩展。
- 数据权限会变复杂。
- 成本策略会变化。
- 安全规则会变严格。

如果架构完全写死，后续每次调整都会变成大改。

例如模型调用层最好不要散落在业务代码中。

prompt 版本最好有记录。

检索和 rerank 最好可以独立评测。

工具调用最好有统一的参数校验和日志。

权限过滤最好不要依赖 prompt 约束。

这些不是过度设计，而是对高频变化点做基本隔离。

68.6 技术负责人要控制复杂度

很多人以为技术负责人要设计复杂系统。

其实更重要的是控制复杂度。

大模型项目很容易变复杂：多模型、多 prompt、多工具、多流程、多数据源、多评测集、多配置、多异常分支。

复杂度失控后，团队会遇到：

- 新人看不懂系统。
- 改一个 prompt 影响多个场景。
- 指标变差不知道原因。
- 线上问题无法复现。
- 配置互相覆盖。
- 工具调用链路不可追踪。
- 测试成本不断升高。

技术负责人要定期问：

- 这个抽象是否真的必要？
- 这个框架是否解决了核心问题？
- 这个配置是否有人理解？
- 这个分支是否可以删除？
- 这个能力是否应该合并？
- 这个流程是否可以标准化？

控制复杂度不是保守。

它是为了让系统能持续迭代。

68.7 建立评审机制，而不是靠个人经验

当方向变大后，不能只靠负责人一个人把关。

需要建立机制。

常见机制包括：

- 方案评审。
- prompt 变更评审。
- 评测集变更评审。
- 模型切换评审。
- 工具权限评审。
- 上线 checklist。
- 事故复盘。
- 技术债定期梳理。

机制的目的不是增加流程负担，而是减少低级风险。

例如模型切换前，应该明确：为什么切换、对比样本是什么、收益是多少、成本和延迟如何变化、是否有回滚方案。

例如 Agent 新增工具前，应该明确：工具是否有副作用、权限如何控制、参数如何校验、失败如何处理、调用日志如何记录。

好的技术负责人会把自己的经验变成团队流程。

这样即使他不参与每个细节，系统质量也不会完全依赖个人记忆。

68.8 技术负责人要会做取舍

负责方向时，最难的不是提出方案，而是做取舍。

取舍通常发生在这些维度之间：

- 准确率和延迟。
- 效果和成本。
- 灵活性和稳定性。
- 自动化和可控性。
- 通用平台和业务定制。
- 快速上线和长期架构。
- 模型能力和工程约束。
- 用户体验和安全边界。

例如 Agent 系统中，更高的自主性可能带来更强能力，但也带来不可控风险。

对于高风险业务，技术负责人可能要选择更保守的工作流编排，而不是完全开放的自主 Agent。

例如 RAG 中，rerank 可能提升准确率，但增加延迟和成本。

如果用户场景对实时性要求很高，就可能需要缓存、分层检索或只对低置信 query 启用 rerank。

技术负责人不是永远选择最先进方案，而是在约束下选择最合适方案。

68.9 推动跨团队协作

大模型方向很少能由单个工程师独立完成。

技术负责人需要和多个团队协作。

例如一个企业 AI 助手可能涉及：

- 业务团队提供场景和专家反馈。
- 数据团队提供文档、标注和清洗。
- 算法团队提供模型和评测支持。
- 平台团队提供推理服务和监控。
- 后端团队负责权限、接口和流程集成。
- 前端团队负责交互体验。
- 安全团队负责合规、审计和风险控制。

技术负责人要能把不同团队的语言翻译成共同目标。

业务方关心效率和风险。

算法方关心数据和模型效果。

平台方关心稳定性和成本。

安全方关心边界和审计。

产品方关心体验和上线节奏。

如果没有人负责对齐，项目会变成各做各的。

技术负责人需要推动共识：问题是什么，目标是什么，谁负责什么，风险在哪里，什么时候评估结果。

68.10 技术负责人要培养接班人

如果一个方向完全依赖一个人，说明这个方向还不健康。

技术负责人应该有意识地降低个人单点风险。

可以通过几种方式：

- 写清架构文档。
- 记录关键决策。
- 让其他人参与方案评审。
- 把排障流程写成 checklist。
- 把常见问题沉淀成文档。
- 让团队成员轮流负责小模块。
- 在 code review 中解释设计意图。
- 把实验脚本和评测流程工具化。

培养接班人不是削弱自己的价值。

相反，它说明你能把个人能力转化为组织能力。

当你能让别人接手现有系统，你才有空间负责更大的方向。

68.11 如何判断自己是否准备好负责一个方向

可以用下面的问题自测。

如果你已经能比较稳定地回答这些问题，说明你正在接近技术负责人角色。

- 我是否能讲清这个方向的业务目标？
- 我是否知道当前最大技术瓶颈？
- 我是否知道哪些指标最重要？
- 我是否知道线上最常见失败模式？
- 我是否能判断新技术是否值得引入？
- 我是否能设计评测和回归机制？
- 我是否能识别高风险变更？
- 我是否能推动跨团队协作？
- 我是否能让别人理解并接手我的系统？
- 我是否能将经验沉淀成流程、文档和工具？

如果这些问题大多回答不上来，说明你可能还停留在执行者阶段。

这并不可怕。

成长的关键是从现在开始主动补齐这些视角。

68.12 常见误区

从执行者转向技术负责人时，有几个常见误区。

第一个误区是事事亲自做。

负责人不是所有代码都自己写，而是确保关键问题被正确解决。

如果你什么都亲自做，方向规模会被你的个人时间限制住。

第二个误区是只做架构不碰细节。

大模型系统的很多问题藏在细节里：prompt、数据、异常 case、权限、日志、指标口径。

如果完全脱离细节，方案容易空泛。

第三个误区是用权威代替证据。

技术负责人不能只说“我觉得这样更好”。

要尽量用数据、实验、失败 case 和工程约束支撑判断。

第四个误区是追求大而全平台。

平台化应该来自重复需求，而不是预先想象。

没有足够业务验证前，过早平台化可能拖慢项目。

第五个误区是忽视安全和成本。

大模型应用如果只看效果，很容易在上线后被成本、延迟、权限和合规问题反噬。

68.13 一个方向负责人的工作样例

假设你负责一个企业 RAG 方向。

你的工作不只是接需求。

你可能需要做：

- 梳理当前知识库问答的主要失败类型。
- 建立核心业务问题评测集。
- 定义召回、引用、幻觉、延迟和成本指标。
- 评估文档解析、chunk、embedding、rerank、query rewrite 的优先级。
- 设计权限过滤和引用校验机制。
- 建立 prompt 和模型变更回归流程。
- 推动线上反馈进入失败 case 池。
- 输出 RAG 接入规范，供其他业务线复用。
- 和平台团队协作降低推理成本。
- 和安全团队确认敏感数据处理方式。

这才是方向负责人的视角。

他不是只做一个功能，而是让这个方向持续变得更可靠、更可复用、更可度量。

68.14 从技术负责人到更高层级

技术负责人再往上，通常会走向更大的技术影响力。

例如从负责一个模块，到负责多个系统；从负责一个业务线，到负责平台能力；从解决项目问题，到定义技术战略。

更高层级会更关注：

- 多团队协同。
- 技术路线选择。
- 资源投入优先级。
- 长期平台能力。
- 风险治理。
- 人才培养。
- 技术文化。
- 业务和技术之间的战略连接。

但这些能力不是突然出现的。

它们都从负责一个小方向开始。

如果你能把一个小方向负责清楚，下一步才有资格负责更大范围。

68.15 小结

从执行者到技术负责人，是大模型工程师职业成长中的关键转变。

执行者关注任务完成，技术负责人关注系统结果、方向演进和长期能力建设。

技术负责人需要定义问题、管理技术方向、建立指标体系、控制复杂度、推动评审机制、做取舍、协调跨团队合作，并把个人经验转化为团队资产。

这个角色不一定需要正式头衔。

当你开始主动对一个方向的质量、稳定性、成本、安全、评测和可维护性负责时，你就已经在承担技术负责人的工作。

长期来看，这种责任意识和系统视角，是从中级工程师走向高级、资深和技术领导者的核心能力。

第 69 章：技术判断与方案取舍

上一章讨论了从执行者到技术负责人。

技术负责人最核心的能力之一，就是技术判断。

所谓技术判断，不是知道更多名词，也不是永远选择最新方案，而是在具体约束下选择更合适的路径。

大模型工程里，很多问题没有标准答案。

要不要换更强模型？

要不要做微调？

要不要上 Agent？

要不要使用长上下文？

要不要引入 rerank？

要不要自建推理服务？

要不要把 demo 平台化？

这些问题都不能只靠感觉回答。

这一章讨论：大模型工程师如何形成可靠的技术判断，如何在效果、成本、延迟、安全、复杂度和业务价值之间做取舍。

69.1 技术判断不是偏好，而是有证据的选择

很多技术争论表面上是在讨论方案，实际是在表达偏好。

有人偏好最新模型，有人偏好开源方案，有人偏好简单架构，有人偏好平台化，有人偏好端到端自动化。

偏好本身没有问题，但偏好不能替代判断。

技术判断至少需要回答：

- 要解决的问题是什么？
- 当前约束是什么？
- 可选方案有哪些？
- 每个方案的收益是什么？
- 每个方案的成本是什么？
- 主要风险是什么？
- 如何验证？
- 如果失败，如何回退？

如果这些问题回答不清，那么所谓判断很可能只是直觉。

成熟工程师可以有直觉，但最终要把直觉转化为证据、实验和可解释的取舍。

69.2 先判断问题类型，再选择技术手段

大模型项目中最常见的错误，是拿技术手段直接套问题。

例如：

- 效果不好，就换大模型。
- 回答不准，就调 prompt。

- 流程复杂，就上 Agent。
- 知识太多，就上长上下文。
- 个性化不足，就微调。
- 延迟高，就换推理框架。

这些反应不一定错，但都太快了。

更好的顺序是先判断问题类型。

常见问题类型包括：

- 知识缺失：模型没有看到必要信息。
- 召回错误：相关信息没有被检索到。
- 排序错误：相关信息被检索到但位置靠后。
- 理解错误：模型看到了信息但没理解。
- 指令冲突：prompt、系统规则和上下文互相干扰。
- 数据质量差：文档、标注或工具返回本身有问题。
- 权限错误：模型看到了不该看的信息，或看不到该看的信息。
- 工具失败：参数、接口、权限、幂等或异常处理有问题。
- 评测偏差：指标不能反映真实用户体验。
- 产品问题：交互方式、预期管理或业务流程设计不合理。

不同问题对应不同技术手段。

如果是知识缺失，可能需要 RAG、工具调用或上下文补充。

如果是召回错误，应该看文档解析、chunk、embedding、query rewrite 或 hybrid search。

如果是排序错误，可以考虑 rerank。

如果是工具失败，换模型未必有用，应该检查 schema、参数校验、错误恢复和权限。

如果是评测偏差，继续优化模型可能只是优化了错误指标。

先判断问题类型，才能避免错误投入。

69.3 换模型不是万能解法

大模型工程中，换更强模型是最直接的想法。

它有时很有效。

但它不是万能解法。

换模型通常能改善：

- 语言理解能力。
- 复杂推理能力。
- 指令遵循能力。
- 多语言能力。
- 代码生成能力。
- 鲁棒性。

但换模型不一定能解决：

- 知识库没有召回到正确证据。
- 文档解析丢失表格结构。
- 工具接口返回错误。
- 权限过滤逻辑错误。
- prompt 要求互相冲突。
- 评测集质量差。
- 用户需求本身不清楚。
- 产品交互没有管理预期。

判断是否换模型，可以问：

- 当前失败案例中，有多少是模型能力不足？
- 如果给模型正确上下文，它能不能回答对？
- 更强模型在核心样本上提升多少？
- 成本和延迟增加多少？
- 是否会引入新的格式、风格或安全问题？
- 是否有回滚和灰度方案？

一个简单实验是 oracle context 测试。

也就是人工把正确证据放进上下文，看模型是否能回答正确。

如果有正确上下文时模型仍然答错，说明模型能力可能是瓶颈。

如果有正确上下文时模型能答对，但线上答错，瓶颈更可能在检索、上下文组织或工具链路。

69.4 微调、RAG 和 prompt 的取舍

很多人会问：这个问题应该用 prompt、RAG，还是微调？

没有统一答案，但可以从问题性质判断。

prompt 适合解决：

- 输出格式约束。
- 角色和语气控制。
- 简单任务步骤说明。
- 小范围行为调整。
- 已有上下文下的推理和总结。

RAG 适合解决：

- 外部知识更新快。
- 需要引用和可追溯证据。
- 企业私有知识。
- 知识库规模较大。
- 不希望把知识固化进模型参数。

微调适合解决：

- 稳定重复的任务模式。
- 特定输出风格。
- 特定领域表达习惯。
- 大量高质量标注样本可用。
- prompt 太长或成本太高。
- 基座模型具备能力但行为不稳定。

不适合微调的情况包括：

- 数据少且质量不稳定。
- 知识频繁变化。
- 主要问题是检索不到证据。
- 需要严格引用来源。
- 没有可靠评测集。
- 上线后无法监控副作用。

很多企业知识问答问题，第一优先级通常不是微调，而是文档解析、检索、rerank、引用校验和评测。

微调不是不能做，而是要在问题明确、数据质量足够、评测可靠时做。

69.5 Agent 不是所有自动化的答案

Agent 很吸引人，因为它看起来可以自动规划、调用工具、完成任务。

但不是所有自动化都需要 Agent。

如果业务流程清楚、步骤固定、规则明确，用普通工作流可能更稳定、更便宜、更可控。

Agent 更适合：

- 用户目标多样。
- 步骤不固定。
- 需要根据上下文选择工具。
- 需要处理开放问题。
- 需要在多个信息源之间推理。

Agent 不适合或需要谨慎的场景包括：

- 高风险不可逆操作。
- 强合规流程。
- 步骤固定且规则明确。
- 对延迟和成本极敏感。
- 缺少工具权限和审计机制。
- 没有可靠评测和人工接管。

判断是否上 Agent，可以问：

- 任务是否真的需要动态规划？
- 工具调用是否有副作用？
- 模型选错工具的代价是什么？
- 是否能限制操作空间？
- 是否有确认、回滚和审计？
- 是否能评测任务完成率？

很多时候，最合适的方案是“工作流 + 局部 LLM 能力”，而不是完全自主 Agent。

例如工单处理可以用固定流程控制状态流转，只在意图识别、信息抽取、回复生成和异常解释环节使用模型。

这样既能利用模型能力，又能保持系统可控。

69.6 长上下文和 RAG 不是简单替代关系

长上下文模型出现后，很多人认为 RAG 会被替代。

这个判断过于简单。

长上下文确实改变了部分应用方式。

它适合：

- 单个文档很长但数量有限。
- 用户需要围绕完整材料做分析。
- 上下文结构比较完整。
- 任务对跨段落关系敏感。

但 RAG 仍然有价值：

- 知识库很大，无法全部塞入上下文。
- 需要权限过滤。
- 需要引用和证据排序。
- 需要控制成本和延迟。
- 需要增量更新。
- 需要可解释的召回链路。

长上下文也有成本和风险：

- token 成本高。
- 延迟可能增加。
- 模型不一定真正利用远距离信息。
- 无关信息过多可能干扰回答。
- 上下文构造仍然需要策略。

更合理的判断是：长上下文和 RAG 可以互补。

RAG 负责从大规模知识库中筛选相关信息，长上下文负责承载更多结构化证据和复杂任务背景。

是否使用长上下文，应该通过真实任务评测，而不是只看模型支持多少 token。

69.7 自建模型服务还是调用 API

企业落地大模型时，经常面对自建模型服务和调用外部 API 的选择。

调用 API 的优势是：

- 启动快。
- 模型能力强。
- 运维成本低。
- 可以快速试错。
- 不需要管理 GPU 和推理引擎。

调用 API 的风险是：

- 数据合规限制。
- 成本不可控。
- 延迟和稳定性受外部影响。
- 模型版本变化可能影响效果。
- 定制能力有限。

自建服务的优势是：

- 数据可控。
- 成本可优化。
- 版本可控。
- 可做深度定制。
- 可与内部系统更紧密集成。

自建服务的成本是：

- GPU 资源和运维复杂。
- 推理框架、监控、扩缩容、容量规划都要自己负责。
- 模型效果可能不如顶级闭源模型。
- 团队需要 infra 能力。

判断时可以问：

- 数据是否允许出域？
- 请求量是否足够大，值得自建？
- 延迟和稳定性是否有强要求？
- 团队是否有推理工程能力？
- 模型能力差距是否可接受？
- 是否需要微调、量化或私有部署？

很多团队会采用混合策略：早期用 API 快速验证，成熟后把高频、低风险、可控任务迁移到自建服务，复杂任务继续使用能力更强的模型。

69.8 开源框架还是自研

大模型应用生态中有很多框架。

它们可以加速开发，但也可能引入不必要的抽象。

选择开源框架时，要看：

- 是否解决你的核心问题？
- 抽象是否容易理解？
- 是否方便调试？
- 是否能满足性能和安全要求？
- 社区是否活跃？
- 版本变化是否稳定？
- 退出成本高不高？

不要因为框架流行就引入。

也不要因为“自研更可控”就什么都重写。

一个实用原则是：

- 通用基础能力优先复用成熟方案。
- 业务核心差异化能力可以自研。
- 高风险链路要确保可观测、可替换、可回滚。

例如向量数据库、基础模型 serving、日志监控可以优先复用成熟工具。

但你的业务评测集、失败 case 分类、权限过滤策略、工具审批流程，往往需要结合业务自研。

69.9 Demo、试点和生产系统的标准不同

技术判断必须考虑项目阶段。

同一个方案，在 demo 阶段合理，在生产阶段可能很危险。

demo 阶段的目标是验证可能性。

可以接受临时 prompt、少量样本、手动配置、简单日志。

试点阶段的目标是验证可用性。

需要基本评测、用户反馈、异常处理、成本估算和灰度机制。

生产阶段的目标是验证可靠性。

需要稳定评测、监控告警、权限控制、审计、回滚、容量规划、安全边界和运维流程。

很多大模型项目的问题，是把 demo 代码直接推进生产。

这会导致：

- prompt 没有版本管理。
- 没有回归评测。
- 没有日志和 trace。
- 没有成本监控。
- 没有失败路径处理。
- 没有权限和安全边界。

技术负责人要明确不同阶段的工程标准。

既不能在 demo 阶段过度工程，也不能在生产阶段继续用 demo 标准。

69.10 用小实验降低决策风险

很多技术争论可以通过小实验解决。

不要在会议里争论三小时，不如设计一个一两天能完成的验证。

小实验要有：

- 明确假设。
- 对比方案。
- 固定样本。
- 指标定义。
- 失败标准。
- 结论记录。

例如是否引入 rerank，可以用 100 到 300 条标注 query 做对比：embedding only 与 embedding + rerank 的 Top-K 命中率、引用准确率、延迟和成本。

例如是否换模型，可以在相同上下文和 prompt 下比较不同模型的正确率、格式稳定性、拒答准确率、成本和 P95 延迟。

例如是否上 Agent，可以挑选 30 个真实任务，比较固定 workflow、LLM 辅助 workflow 和 Agent 的任务完成率、错误类型和人工接管率。

小实验的价值不是得出永久真理，而是减少盲目决策。

69.11 什么时候应该保持简单

保持简单是一种重要判断。

以下情况通常应该优先简单方案：

- 问题还没有被验证。
- 用户需求还不稳定。
- 数据质量还很差。
- 评测体系还没有建立。
- 团队维护能力有限。
- 高级方案收益不明确。
- 上线风险主要来自流程和权限，而不是模型能力。

例如一个业务刚开始试点智能问答，知识库文档还没有整理好，用户问题类型也不清楚。

此时不应该一开始就构建复杂 Agent 平台。

更合理的是先做一个简单 RAG 原型，收集用户问题、失败 case 和文档质量问题。

简单方案不是低水平。

在不确定性高的时候，简单方案可以帮助团队更快学习。

69.12 什么时候应该增加复杂度

复杂度不是绝对坏事。

当复杂度换来明确收益时，它是必要的。

可以增加复杂度的情况包括：

- 同类需求反复出现。
- 手工流程成为瓶颈。
- 线上风险需要机制控制。
- 多业务线需要复用能力。
- 指标显示某个瓶颈长期存在。
- 简单方案已经无法满足规模。
- 技术债开始显著影响交付。

例如早期可以手动整理失败 case。

但当线上每天产生大量反馈，且多个业务线都需要分析失败类型时，就应该建设反馈回流和评测样本管理工具。

早期可以手动配置 prompt。

但当 prompt 频繁变更并影响多个场景时，就应该做版本管理、评审和回归测试。
增加复杂度的前提是看见真实重复问题，而不是想象未来可能需要。

69.13 决策文档：让取舍可追溯

重要技术决策应该留下记录。

决策文档不需要很长，但要说明：

- 背景。
- 问题。
- 约束。
- 可选方案。
- 对比维度。
- 决策结论。
- 风险和缓解措施。
- 验证结果。
- 未来重新评估条件。

例如决定暂不微调，可以记录：

- 当前问题主要来自召回和引用错误。
- 现有标注数据不足以支撑微调。
- 微调无法解决知识更新和引用追溯问题。
- 先投入文档解析、检索评测和 rerank。
- 如果后续稳定任务样本超过一定规模，再重新评估微调。

这样的记录能减少重复争论。

几个月后如果条件变化，团队也能知道当初为什么这样决定。

69.14 技术判断的常见陷阱

大模型工程中有几个常见陷阱。

第一，benchmark 迷信。

某个模型在公开榜单上强，不代表在你的业务场景中强。

第二，demo 幻觉。

一个演示效果好，不代表线上长尾稳定。

第三，新技术偏见。

新方法可能有价值，但也可能只是增加复杂度。

第四，局部最优。

某个模块指标提升，可能导致整体延迟、成本或安全风险变差。

第五，忽视维护成本。

方案上线只是开始，后续监控、排障、升级和人员交接都要成本。

第六，过度相信模型自我纠错。

模型可以辅助检查，但不能替代权限、幂等、审计和工程约束。

第七，缺少退出策略。

引入框架、模型或平台前，没有想清楚如果不好用怎么退出。

识别这些陷阱，本身就是技术判断的一部分。

69.15 小结

技术判断是大模型工程师从执行者走向负责人的关键能力。

可靠判断不是个人偏好，而是在明确问题、约束、方案、收益、成本、风险和验证方式之后做出的选择。

面对换模型、微调、RAG、Agent、长上下文、自建服务、开源框架和平台化等问题时，不要被热点牵引，也不要停留在抽象争论中。

先判断问题类型，再设计小实验，用指标和失败案例支撑决策。

保持简单和增加复杂度都可能正确，关键在于是否匹配当前阶段、真实瓶颈和团队能力。

最终，成熟的大模型工程师不是永远选择最先进技术的人，而是在复杂约束下持续做出可解释、可验证、可回退选择的人。

第 70 章：技术影响力与跨团队沟通

上一章讨论了技术判断与方案取舍。

判断做出来之后，还有一个同样重要的问题：如何让正确的方案真正被团队理解、接受并落地。

很多工程师以为技术影响力来自职位、资历或代码能力。

这些当然重要，但在真实项目中，技术影响力更多来自持续解决问题的能力，以及把复杂问题讲清楚、把不同团队拉到同一张图上的能力。

大模型工程尤其依赖跨团队沟通。

一个 RAG 项目可能涉及业务、数据、后端、平台、安全、算法和产品。

一个 Agent 项目可能涉及工具接口、权限审批、审计、前端交互、任务评测和人工兜底。

一个推理服务项目可能涉及 GPU 资源、模型团队、平台团队、SRE、业务调用方和成本负责人。

如果你只能把自己负责的代码写好，但无法推动这些人形成共识，你的影响力就会停留在局部。

这一章讨论：大模型工程师如何建立技术影响力，如何进行跨团队沟通，如何让方案从“我觉得对”变成“团队愿意一起做”。

70.1 技术影响力不是控制别人

技术影响力不是让别人听你的。

它更接近一种能力：让团队基于更清楚的信息、更可靠的判断和更低的协作成本，做出更好的技术选择。

有影响力的工程师通常能做到：

- 发现真正的问题。
- 把复杂问题讲简单。
- 把模糊争论变成可验证假设。
- 让不同角色理解彼此约束。
- 让方案的收益、成本和风险透明。
- 推动团队在不确定中前进。
- 在结果出来后复盘和修正。

这和“声音最大”不是一回事。

一个人可以很强势，但没有影响力。

因为别人只是被迫接受他的结论，并没有真正理解问题。

真正的技术影响力会让团队能力变强，而不是让团队依赖某个人拍板。

70.2 大模型项目为什么沟通更难

普通软件项目当然也需要沟通。

但大模型项目有几个额外困难。

第一，问题边界模糊。

“回答不好”“不够智能”“用户不满意”“想要 Agent” 这些说法都不是清晰需求。

它们背后可能是模型能力问题、数据问题、检索问题、产品交互问题、权限问题或预期管理问题。

第二，结果具有不确定性。

传统功能通常能明确判断对错。

大模型输出经常是概率性的，同一个 prompt 在不同上下文、不同采样参数、不同模型版本下都可能变化。

第三，团队语言不同。

业务方讲场景和效率。

算法方讲数据、模型和指标。

平台方讲稳定性、吞吐和成本。

安全方讲权限、审计和风险。

产品方讲体验、转化和用户预期。

这些语言如果没有人翻译，就会变成互相误解。

第四，技术热点太多。

RAG、Agent、长上下文、微调、多模态、推理框架、MCP、workflow、LLM-as-a-judge 都可能被拿来当解决方案。

如果没有清晰沟通，团队很容易从一个热点跳到另一个热点。

第五，线上风险更隐蔽。

大模型系统的问题不一定表现为服务挂掉。

它可能表现为引用不准确、越权召回、工具误调用、幻觉、成本飙升、输出格式偶发失败或用户信任下降。

这些问题需要跨团队共同定义和监控。

70.3 先对齐问题，而不是先推方案

跨团队沟通最大的错误，是一开始就推方案。

例如：

- “我们应该上 rerank。”
- “我们应该换成更强模型。”
- “我们应该做 Agent。”
- “我们应该自建推理服务。”

这些话会立刻把讨论带入立场对抗。

更好的方式是先对齐问题。

可以这样问：

- 当前最影响用户的问题是什么？
- 这个问题出现频率是多少？
- 哪些用户或业务受影响最大？
- 目前有什么失败案例？
- 失败案例能否分类？

- 问题更像模型能力不足，还是上下文、数据、工具或产品流程问题？
- 解决后我们希望哪个指标变化？

当问题被清楚定义后，方案讨论会自然很多。

例如不要直接说“加 rerank”。

可以说：

我们分析了最近 100 个 RAG 失败 case，其中 42 个是正确 chunk 已经被召回到 top-20，但没有进入最终上下文。这说明第一阶段 em

这比直接说“我要上 rerank”更容易获得支持。

因为它把方案放在证据之后。

70.4 用共同语言连接不同角色

跨团队沟通的关键，是找到共同语言。

不同角色关心的东西不同。

如果你只用自己的语言表达，别人很难配合。

对业务方，不要只讲模型和框架。

要讲：

- 用户问题是否解决。
- 人工成本是否下降。
- 风险是否可控。
- 上线后如何验证收益。
- 失败时如何兜底。

对产品方，要讲：

- 用户在哪里感知到价值。
- 交互如何管理预期。
- 哪些场景应该拒答或转人工。
- 体验指标如何衡量。
- 反馈如何回流。

对平台或 SRE，要讲：

- 流量规模。
- token 长度分布。
- 延迟和吞吐目标。
- GPU 和存储成本。
- 监控、告警和回滚。
- 峰值和故障场景。

对安全团队，要讲：

- 数据边界。
- 权限过滤。
- 工具调用审计。
- 高风险操作确认。
- 敏感信息处理。
- 攻击和滥用场景。

对算法团队，要讲：

- 评测集。
- 失败类型。
- 数据质量。
- baseline。

- 指标和 ablation。
- 模型或训练方案的适用边界。

同一个技术方案，要能翻译成不同角色能理解的价值和风险。
这不是包装话术，而是减少协作摩擦。

70.5 用文档建立影响力

技术影响力很大一部分来自写作。

口头沟通容易遗忘，也容易被误解。

文档可以让复杂问题被反复阅读、评论和修正。

大模型工程师应该熟练写几类文档。

第一类是问题分析文档。

结构可以是：

- 背景。
- 当前现象。
- 影响范围。
- 失败案例。
- 初步分类。
- 根因假设。
- 下一步验证。

第二类是方案设计文档。

结构可以是：

- 目标。
- 非目标。
- 约束。
- 可选方案。
- 方案对比。
- 推荐方案。
- 风险和回滚。
- 评测和上线计划。

第三类是实验报告。

结构可以是：

- 假设。
- 数据集。
- baseline。
- 对比方案。
- 指标。
- 结果。
- bad case。
- 结论。
- 后续建议。

第四类是决策记录。

结构可以是：

- 我们决定了什么。
- 为什么这样决定。
- 放弃了哪些方案。
- 当前证据是什么。

- 风险是什么。
- 什么时候重新评估。

写文档不是为了显得正式。

它的价值是让技术判断可传播、可讨论、可追溯。

70.6 方案评审要评审什么

很多方案评审会变成架构图展示。

真正有价值的评审应该关注风险和证据。

对于大模型项目，评审至少要问：

- 问题定义是否清楚？
- baseline 是什么？
- 方案解决的是哪类失败？
- 有没有更简单方案？
- 评测集是否可信？
- 指标是否覆盖质量、延迟、成本和安全？
- 是否会引入新失败模式？
- 是否有灰度计划？
- 是否有回滚方案？
- 是否能观察线上效果？

例如评审一个 Agent 工具调用方案时，不应该只看工具列表。

还要看：

- 工具 schema 是否清晰。
- 参数是否可校验。
- 权限在哪里检查。
- 工具失败时模型如何回答。
- 高风险操作是否二次确认。
- 调用日志是否完整。
- 是否有 prompt injection 防护。
- 是否有人工接管路径。

好的评审不是挑刺。

好的评审是在上线前把风险显性化。

70.7 把争论变成实验

跨团队争论经常会卡住。

例如：

- 产品觉得模型已经够好。
- 算法觉得需要换模型。
- 平台觉得成本太高。
- 安全觉得风险不可控。

这时不要停留在观点对观点。

应该把争论转成实验。

例如对于“是否换模型”，可以定义：

样本：200 条真实用户问题，覆盖高频场景和高风险场景。

方案：当前模型 vs 候选模型。

固定：prompt、上下文、采样参数。

指标：正确率、引用准确率、格式合法率、安全拒答、P95 延迟、单次成本。

结论标准：如果正确率提升超过 8%，且成本增加低于 40%，P95 延迟仍在 SL0 内，则进入灰度。

对于“是否使用 Agent”，可以定义：

样本：50 个真实任务。

方案：人工流程、固定 workflow、Agent。

指标：任务完成率、平均步骤数、工具误调用率、人工接管率、P95 延迟、失败恢复率。

结论标准：Agent 必须显著提升完成率，且高风险误调用为 0，才考虑扩大范围。

实验不会消除所有不确定性。

但它能让讨论从“谁更有道理”转成“证据支持什么”。

70.8 沟通技术风险时要具体

很多工程师说风险时太抽象。

例如：

- “这个方案可能不稳定。”
- “这个有安全风险。”
- “这个成本可能比较高。”
- “这个后面不好维护。”

这些话很难推动决策。

更好的表达是具体化风险。

例如：

如果我们把所有文档都塞进长上下文，短期可以绕过检索问题，但 P95 TTFT 可能从 1.5 秒增加到 8 秒以上。同时无关文档进入上下文。

再例如：

如果工具权限只靠 prompt 约束，模型在 prompt injection 或上下文冲突时仍可能构造越权调用。权限判断必须在后端执行，模型只

具体风险更容易被理解，也更容易设计缓解措施。

70.9 面对不同意见时不要急着防守

跨团队沟通中，不同意见很正常。

产品可能担心上线节奏。

平台可能担心成本。

安全可能担心合规。

业务可能担心效果不稳定。

如果你把所有质疑都当作否定，就会进入防守状态。

更好的做法是先识别对方背后的约束。

可以问：

- 你最担心的是哪类失败？
- 如果我们能监控或回滚，这个风险是否可接受？
- 哪个指标达到什么水平，你会认为可以试点？
- 你希望灰度范围多小？
- 哪些场景必须先排除？

这些问题能把抽象反对变成具体条件。

例如安全团队反对 Agent 自动发邮件。

你可以把方案调整为：

- 模型只能生成草稿。
- 用户必须确认后发送。
- 高风险收件人触发二次确认。
- 所有工具调用写审计日志。
- 试点仅限内部低风险场景。

沟通的目的不是赢辩论，而是找到可控推进路径。

70.10 向上沟通：让管理者理解取舍

向上沟通时，管理者通常不需要所有技术细节。

但他们需要理解：

- 当前问题影响什么业务结果。
- 有哪些方案。
- 推荐方案是什么。
- 需要什么资源。
- 风险是什么。
- 多久能看到结果。
- 如果失败如何止损。

一个清晰的向上沟通模板是：

背景：当前知识库问答在高价值客户场景中引用错误率偏高。

影响：客服团队需要人工复核，试点客户信任度下降。

分析：最近 200 个失败 case 中，主要问题是表格解析和 rerank 排序错误，而不是模型回答能力不足。

方案：优先投入文档解析修复和 rerank 小实验，暂不做微调。

资源：需要 1 名后端、1 名算法、1 名业务专家标注 300 条 query-doc 样本。

风险：rerank 可能增加延迟，灰度时会监控 P95 latency 和成本。

时间：两周内给出实验结论，四周内决定是否扩大。

这种表达比“我们想优化 RAG”更有决策价值。

70.11 横向沟通：让协作团队愿意参与

横向团队没有义务天然支持你的优先级。

他们也有自己的目标和压力。

要推动他们参与，必须说明为什么这件事和他们有关。

对平台团队，可以强调：

- 当前推理成本高在哪里。
- 哪些流量模式导致资源浪费。
- 需要哪些 metrics 支持定位。
- 优化后能减少多少 GPU 压力。

对数据团队，可以强调：

- 哪些数据质量问题导致模型失败。
- 标注优先级是什么。
- 数据改进如何被评测验证。
- 哪些字段会影响后续检索和权限。

对安全团队，可以强调：

- 哪些调用有副作用。
- 哪些数据可能泄露。
- 哪些边界必须在系统层强制。

- 审计日志如何帮助追责和复盘。

横向沟通的核心是互惠。

不要只说“我需要你们做什么”。

要说“我们共同解决什么问题，你们的投入如何降低你们关心的风险”。

70.12 会议不是越多越好

跨团队项目容易开很多会。

但会议不是协作本身。

低质量会议会消耗影响力。

好的会议应该满足：

- 会前有背景材料。
- 会议目标明确。
- 需要决策的问题提前列出。
- 参与者是必要的人。
- 会中记录结论和分歧。
- 会后明确 owner、deadline 和下一步。

如果只是同步信息，文档或异步更新可能更好。

如果需要形成共识，会议应该聚焦少数关键问题。

例如一次 RAG 方案评审会议，不要同时讨论文档解析、embedding、rerank、prompt、权限、UI 和成本所有细节。

可以先聚焦：当前最大失败类型是什么，本阶段优先解决哪两个问题，如何验证。

会议越聚焦，越容易产出行动。

70.13 建立可复用的沟通资产

影响力会随着可复用资产积累而放大。

常见沟通资产包括：

- 项目一页纸。
- 架构图。
- 请求生命周期图。
- 失败 case 分类表。
- 指标 dashboard。
- 实验报告模板。
- 模型切换 checklist。
- RAG 上线 checklist。
- Agent 工具接入 checklist。
- 故障复盘模板。

这些资产能减少重复解释。

例如新业务线想接入 RAG，不需要每次从零讲一遍。

你可以给他们一份接入 checklist：

- 文档来源。
- 权限字段。
- 更新频率。
- 解析格式。
- query 样本。

- 评测样本。
- 引用要求。
- 失败兜底。
- 成本预估。

当别人能基于你的材料自助推进时，你的影响力就从个人沟通变成了组织能力。

70.14 技术影响力来自持续可信

影响力不是一次演讲建立的。

它来自长期可信。

可信通常来自几个行为：

- 不夸大收益。
- 不隐瞒风险。
- 承认不确定性。
- 用数据支持结论。
- 说到做到。
- 出问题主动复盘。
- 不把责任推给模型或其他团队。
- 能为团队减少混乱。

在大模型项目中，不确定性很高。

如果你每次都承诺“这个方案一定能解决”，很快会失去信任。

更成熟的表达是：

基于当前 100 个失败 case，我认为主要瓶颈是 rerank 和上下文组织。这个判断还有不确定性，所以建议先做一个一周实验。若指标
这种表达没有过度承诺，但更可信。

70.15 技术沟通中的常见误区

第一个误区是只讲技术，不讲问题。

别人听完知道你用了什么框架，但不知道为什么要做。

第二个误区是只讲收益，不讲代价。

这会让方案显得不成熟。

第三个误区是用术语压人。

技术术语不能替代解释。

第四个误区是把质疑当阻力。

很多质疑其实是在帮你暴露风险。

第五个误区是没有结论。

长篇分析后没有推荐方案、下一步和 owner，会让沟通失效。

第六个误区是所有人用同一套表达。

业务、平台、安全、算法需要不同层次的信息。

第七个误区是缺少后续追踪。

沟通后没有记录、没有 deadline、没有复盘，影响力会快速衰减。

70.16 一个跨团队推进样例

假设你要推进一个企业 RAG 系统的引用准确率改进。

低影响力做法是：

我觉得现在 RAG 效果不好，我们应该加 rerank，再优化 prompt。

高影响力做法是：

过去两周我们收集了 180 个失败 case，其中 63 个是引用不支持答案，41 个是正确 chunk 被召回但没有进入最终上下文，27 个是表

我建议分两步做。第一步，数据团队和业务专家一起标注 300 条 query-doc 样本，平台侧帮忙补充 retrieval 和 rerank 的日志字段。

两周后如果 rerank 能把 citation accuracy 提升超过 10%，且 P95 延迟增加不超过 300ms，我们灰度给 10% 内部用户。如果不满足，这个表达有几个优点：

- 有失败案例支撑。
- 有问题分类。
- 有跨团队分工。
- 有实验指标。
- 有进入灰度的条件。
- 有失败后的替代路径。

这就是技术影响力的具体体现。

70.17 面试表达

如果面试官问：“你如何推动一个跨团队大模型项目落地？”

可以这样回答：

我会先避免直接推方案，而是先和业务、产品、算法、平台、安全等相关方对齐问题定义。大模型项目里，很多诉求比如效果不好、

然后我会把方案写成设计文档，包含目标、非目标、约束、可选方案、推荐方案、风险、评测方式和回滚计划。对不同团队我会用不同

如果团队有分歧，我会尽量把争论转成小实验。例如是否换模型、是否加 rerank、是否用 Agent，都可以用固定样本、固定指标和明

我理解技术影响力不是让别人听我的，而是让团队基于更清楚的问题、更可靠的证据和更低的协作成本做出更好的决策。

70.18 小结

技术影响力是大模型工程师从“能做事”走向“能带动方向”的关键能力。

它不是职位赋予的控制力，而是通过清楚的问题定义、可信的技术判断、有效的文档、具体的风险表达、跨团队共识和持续可交付结果建立起来的。

大模型项目天然跨越业务、产品、算法、平台、安全和运维。

如果没有沟通能力，技术方案即使正确，也可能无法落地。

如果沟通足够清楚，团队就能把模糊诉求转化成问题，把争论转化成实验，把方案转化成行动，把个人经验转化成组织能力。

长期来看，真正有影响力的大模型工程师，不只是能写代码和做方案的人，而是能让复杂系统中的不同人围绕同一个目标协作的人。

第 71 章：技术战略、路线图与长期规划

上一章讨论了技术影响力与跨团队沟通。

当一个工程师能推动跨团队协作后，下一步会遇到更大的问题：如何规划一个方向的未来。

这时工作不再只是解决眼前需求，也不只是把某个方案讲清楚。

你需要回答：

- 这个方向未来半年应该解决什么问题？
- 哪些能力应该先做，哪些应该延后？
- 哪些技术债必须还，哪些可以暂时接受？
- 哪些平台能力值得沉淀？
- 哪些项目应该停止？
- 如何把大模型技术变化转化为团队可执行路线图？

这就是技术战略和路线图问题。

大模型领域变化很快。

新模型、新框架、新 Agent 形态、新推理优化、新评测方法不断出现。

如果没有技术战略，团队很容易陷入两种状态。

一种是追热点。

看到什么新技术都想试，结果每个方向都浅尝辄止。

另一种是只做需求。

业务提什么做什么，短期交付很多，但长期能力没有沉淀。

成熟的大模型工程师需要在两者之间找到平衡：既能响应业务，又能建设长期技术能力。

这一章讨论如何制定大模型工程方向的技术战略、路线图和长期规划。

71.1 技术战略不是口号

很多团队会把技术战略写成口号。

例如：

- 打造领先的大模型平台。
- 建设智能化业务中台。
- 全面拥抱 Agent。
- 降本增效，提升体验。
- 构建统一 AI 基础设施。

这些话可能方向正确，但不能指导行动。

真正有用的技术战略应该能回答：

- 当前最重要的问题是什么？
- 为什么是现在解决？
- 解决这个问题对业务和技术有什么价值？
- 我们选择哪些路径？
- 我们明确不做什么？
- 需要哪些阶段性里程碑？
- 用什么指标判断战略有效？
- 如果环境变化，什么时候调整？

技术战略不是漂亮文字。

它是一组清晰的选择。

选择做什么，也选择不做什么。

71.2 战略从诊断开始

制定路线图之前，先做诊断。

如果没有诊断，路线图通常只是愿望清单。

大模型方向的诊断可以从几个层面展开。

第一，业务诊断。

要问：

- 当前哪些场景最有价值？
- 哪些场景已经被验证？
- 哪些场景只是想象？
- 用户真正愿意使用什么？
- 哪些问题影响收入、效率、成本或风险？

第二，技术诊断。

要问：

- 当前最大质量瓶颈是什么？
- 最大稳定性瓶颈是什么？
- 最大成本瓶颈是什么？
- 最大安全风险是什么？
- 哪些能力只能靠人工维护？
- 哪些模块已经重复建设？

第三，组织诊断。

要问：

- 团队是否有模型、数据、平台、产品和安全协作机制？
- 是否有评测和反馈闭环？
- 是否有人能维护线上系统？
- 是否有足够的 GPU、预算和工具链？
- 哪些依赖团队是瓶颈？

第四，外部技术诊断。

要问：

- 新模型能力是否改变了方案边界？
- 推理框架是否已经足够成熟？
- 开源生态是否能替代部分自研？
- 合规和安全要求是否变化？
- 竞争对手或行业实践是否改变用户预期？

诊断越真实，路线图越不容易跑偏。

71.3 技术愿景和技术战略的区别

愿景回答“长期希望成为什么”。

战略回答“当前阶段如何走过去”。

例如一个企业 AI 助手方向的技术愿景可以是：

让员工员工可以通过自然语言安全、可靠地获取知识、调用工具并完成跨系统任务。

这是愿景。

它描述长期状态。

但它还不能指导下周做什么。

技术战略需要更具体：

未来 6 个月，我们不优先追求完全自主 Agent，而是先建设可信 RAG、工具调用审计和任务评测体系。原因是当前失败 case 主要来自这就是战略。

它说明了当前阶段的重点和取舍。

愿景可以相对稳定。

战略需要随着约束和证据调整。

71.4 路线图不是任务列表

路线图经常被写成一堆任务。

例如：

- 接入新模型。
- 支持多轮对话。
- 加 rerank。
- 加 prompt 管理。
- 加 Agent 工具。
- 做 dashboard。
- 做成本统计。

这只是 backlog，不是路线图。

真正的路线图应该表达阶段目标。

例如：

阶段一：让系统可评估。

建立核心评测集、日志字段、失败 case 分类和 baseline。

阶段二：让答案可信。

优化文档解析、retrieval、rerank、引用校验和资料不足拒答。

阶段三：让系统可运营。

建立反馈回流、prompt/model 版本管理、灰度、监控和成本归因。

阶段四：让能力可扩展。

抽象工具调用、权限、审计、任务评测和多业务接入规范。

每个阶段都有清楚目标。

任务只是服务于阶段目标。

71.5 大模型路线图的常见维度

一个成熟的大模型技术路线图，通常不只包含模型能力。

可以从这些维度规划。

第一，模型能力。

- 模型选型。
- 多模型路由。
- 领域适配。
- 微调或偏好优化。
- 多模态能力。

第二，数据能力。

- 数据清洗。
- 文档解析。
- 标注体系。
- 反馈回流。
- 合成数据。
- 数据权限和治理。

第三，检索和知识能力。

- chunk 策略。
- embedding 模型。
- hybrid retrieval。
- rerank。
- 引用校验。
- 知识更新和版本。

第四，Agent 和工具能力。

- tool schema。
- 权限校验。
- 工具调用日志。
- 高风险操作确认。
- 多步任务评测。
- 人工接管。

第五，推理和平台能力。

- 模型服务。
- batching。
- KV cache。
- 量化。
- 监报告警。
- 成本归因。
- 多租户隔离。

第六，评测和实验能力。

- 离线评测集。
- LLM-as-a-judge。
- 人工评测。
- A/B 测试。
- bad case 管理。
- 回归门禁。

第七，安全和治理能力。

- prompt injection 防护。
- 权限边界。
- 敏感信息处理。
- 审计。
- red teaming。
- 合规流程。

路线图要根据当前瓶颈选择重点。

不可能所有维度同时深入。

71.6 用北极星指标约束路线图

路线图需要指标约束。

否则很容易变成“我们做了很多能力”，但不知道是否真的变好。

大模型项目的北极星指标不能太单一。

例如只看调用量，可能鼓励无效使用。

只看回答满意度，可能忽略安全和成本。

只看准确率，可能忽略延迟。

更好的方式是设置一组平衡指标。

以企业知识问答为例，可以有：

- 任务成功率。
- 引用准确率。
- 资料不足拒答准确率。
- 人工接管率。
- TTFT / P95 延迟。
- 单次请求成本。
- 越权召回率。

以 Agent 自动化为例，可以有：

- 任务完成率。
- 工具选择准确率。
- 参数准确率。
- 高风险误调用率。
- 平均步骤数。
- 人工接管率。
- 审计完整率。

以推理平台为例，可以有：

- TTFT。
- TPOT。
- tokens/s。
- GPU 利用率。
- KV cache 使用率。
- 错误率。
- 单 token 成本。

指标不只是验收工具。

指标会影响路线图优先级。

如果引用准确率是核心指标，文档解析和 citation eval 可能比换模型更重要。

如果单 token 成本是核心指标，推理优化和路由策略可能比新功能更重要。

71.7 优先级排序：影响、紧急、依赖和可逆性

路线图最难的是排序。

可以用四个维度判断优先级。

第一，影响。

这个问题解决后，能影响多少用户、多少成本、多少风险或多少团队？

第二，紧急。

它是否正在阻塞上线、客户交付、安全合规或系统稳定性？

第三，依赖。

它是否是其他能力的前置条件？

例如没有评测集，就很难判断模型切换和 prompt 改动是否有效。

没有工具调用日志，就很难做 Agent 失败分析。

第四，可逆性。

如果做错了，能否低成本回退？

可逆决策可以快速试。

不可逆或高成本决策要更谨慎。

例如换一个 prompt 是相对可逆的。

重构整个平台、迁移向量数据库、自研推理框架就是更重的决策。

排序时要避免两个极端。

一个极端是只做紧急需求，永远没有基础能力。

另一个极端是只做长期平台，业务短期看不到价值。

好的路线图通常同时包含：

- 近期业务交付。
- 中期技术补强。
- 长期平台建设。

71.8 技术债要翻译成业务风险

技术债如果只用工程语言表达，很难获得资源。

例如：

- prompt 没有版本管理。
- 评测脚本不统一。
- 检索日志字段缺失。
- 工具调用没有 trace。
- 模型服务没有容量规划。

这些听起来像工程内部问题。

需要翻译成业务风险：

- prompt 没有版本管理，会导致线上效果回退无法定位。
- 评测脚本不统一，会导致模型切换决策不可信。
- 检索日志字段缺失，会导致客户投诉时无法解释为什么答错。
- 工具调用没有 trace，会导致误操作无法审计。
- 模型服务没有容量规划，会导致高峰期延迟失控和成本不可预测。

技术债不是“代码不优雅”。

技术债是未来交付速度、稳定性、安全和成本的风险。

当你能把技术债翻译成业务风险，路线图中的技术建设就更容易被理解。

71.9 什么时候应该平台化

平台化是路线图中的常见选择。

但平台化不能太早。

过早平台化会带来：

- 抽象不稳定。
- 需求还没验证就固化。
- 平台团队被大量定制需求拖垮。

- 使用方觉得平台复杂。
- 业务价值迟迟看不到。

适合平台化的信号包括：

- 多个业务线重复接入类似能力。
- 重复问题已经稳定出现。
- 接入流程可以标准化。
- 指标和日志口径已经明确。
- 已有两个以上真实场景验证。
- 手工支持成本开始明显升高。

例如 RAG 平台化之前，最好已经知道常见文档类型、权限模型、chunk 策略、评测方式和失败模式。

Agent 平台化之前，最好已经验证工具 schema、权限、审计、高风险确认和任务评测。

推理平台化之前，最好已经明确模型类型、流量规模、SLO、成本归因和多租户隔离需求。

平台化不是把 demo 包一层 UI。

平台化是把重复成功经验抽象成可复用能力。

71.10 什么时候应该停止一个方向

路线图不只是增加项目。

也要决定停止什么。

大模型项目容易因为新鲜感持续投入。

但有些方向应该及时止损。

可以停止或暂停的信号包括：

- 用户真实使用率很低。
- 评测长期没有改善。
- 成本远高于收益。
- 风险无法被工程机制控制。
- 依赖数据长期无法获得。
- 业务流程本身不适合自动化。
- 需求方无法定义成功标准。
- 团队没有维护能力。

例如一个完全自主 Agent 项目，如果连续几轮评测都显示高风险工具误调用无法降到可接受范围，就应该缩小范围，退回 workflow + human confirmation，而不是继续扩大。

停止项目不是失败。

在不确定领域，及时停止低收益方向，是保护团队资源的一种能力。

71.11 设计 3 个月、6 个月和 12 个月路线图

路线图可以分时间尺度。

3 个月看交付和验证。

重点是：

- 明确核心场景。
- 建立 baseline。
- 完成关键实验。
- 修复最明显瓶颈。
- 形成最小闭环。

6 个月看能力沉淀。

重点是：

- 建立评测和反馈闭环。
- 抽象重复能力。
- 建立监控、灰度和回滚。
- 支持多个业务场景。
- 降低成本和维护负担。

12 个月看方向演进。

重点是：

- 平台化关键能力。
- 建立多团队协作机制。
- 形成技术资产和标准。
- 支撑更复杂场景。
- 让系统能跟随模型和业务变化持续演进。

例如企业 AI 助手路线图可以写成：

3 个月：完成核心知识问答场景，建立 RAG 评测集、引用准确率指标和失败 case 回流。

6 个月：支持多业务知识库接入，统一权限过滤、文档解析、rerank、prompt 版本和灰度流程。

12 个月：扩展到工具调用和半自动任务流，建立工具审计、任务评测、人工接管和多租户成本治理。

这样比“做 RAG、做 Agent、做平台”清楚得多。

71.12 路线图要保留调整空间

大模型技术变化太快，路线图不能写死。

可能影响路线图的变化包括：

- 新模型显著提升能力。
- API 成本大幅下降。
- 开源模型满足私有化需求。
- 推理框架支持新的优化。
- 监管要求变化。
- 业务场景优先级变化。
- 线上事故暴露新风险。
- 用户反馈推翻原假设。

因此路线图应该有复盘节奏。

例如每月检查：

- 当前假设是否仍然成立？
- 指标是否朝目标变化？
- 资源投入是否匹配收益？
- 新技术是否改变了原取舍？
- 哪些任务应该提前、延后或取消？

路线图不是承诺永远不变。

路线图是当前证据下的最佳计划。

成熟团队会定期更新路线图，而不是假装环境不会变化。

71.13 把路线图变成团队行动

路线图如果不能落到 owner 和节奏，就只是文档。

要把路线图变成行动，需要做几件事。

第一，拆成可交付里程碑。

例如“提升 RAG 可信度”太大。

可以拆成：

- 建立 300 条评测样本。
- 补齐 retrieval 日志字段。
- 完成 rerank ablation。
- 上线 citation accuracy dashboard。
- 灰度资料不足拒答策略。

第二，明确 owner。

每个里程碑都要有人负责推进，不只是“算法团队”或“平台团队”。

第三，明确验收指标。

不是“完成优化”，而是“citation accuracy 提升到某个范围，P95 延迟不超过某个阈值”。

第四，明确依赖。

例如业务专家标注、平台日志、后端权限字段、安全评审。

第五，明确复盘时间。

路线图执行中必须定期检查偏差。

71.14 技术战略的面试表达

如果面试官问：“你如何为一个大模型工程方向制定技术路线图？”

可以这样回答：

我不会一开始就列功能清单，而是先做诊断。诊断包括业务价值、当前失败 case、技术瓶颈、团队能力、成本和安全约束。比如在企

然后我会区分技术愿景和当前阶段战略。愿景可能是构建可信企业 AI 助手，但 6 个月战略可能不是做完全自主 Agent，而是先把 RAG

路线图上我会按阶段目标组织，而不是按零散任务组织。第一阶段让系统可评估，建立 baseline、评测集和日志。第二阶段让答案可

优先级排序会看影响、紧急程度、依赖关系和可逆性。技术债也会翻译成业务风险，比如没有 prompt 版本管理会导致线上回退不可逆

最后我会把路线图落到 owner、里程碑、验收指标和复盘节奏，否则路线图只是文档，不会变成团队行动。

71.15 小结

技术战略和路线图，是大模型工程师从项目负责人走向方向负责人的关键能力。

技术战略不是口号，而是基于诊断后做出的清晰选择。

路线图不是任务列表，而是围绕阶段目标组织的行动计划。

大模型方向的路线图要同时考虑模型、数据、RAG、Agent、推理平台、评测、安全、成本和组织能力。

优先级排序不能只看谁声音大，也不能只追热点。

要看影响、紧急、依赖和可逆性。

技术债要翻译成业务风险，平台化要等重复成功经验出现，低收益方向要及时停止。

一个好的技术路线图应该既能指导未来 3 个月的交付，也能支撑 6 个月的能力沉淀和 12 个月的方向演进。

最终，成熟的大模型工程师不仅能解决问题，还能帮助团队决定哪些问题最值得解决，以及如何持续朝正确方向前进。

第 72 章：指标、复盘与持续改进

上一章讨论了技术战略、路线图与长期规划。

路线图回答“接下来要往哪里走”。

但还有一个更现实的问题：怎么知道自己真的走对了？

大模型工程最容易出现一种假进步。

团队做了很多事：换模型、改 prompt、加 rerank、接 Agent、做 dashboard、接入新推理框架。

每个人都很忙。

但过一段时间后发现：用户满意度没有明显提升，成本却上去了；线上失败 case 仍然反复出现；团队很难说清楚到底哪个改动有效。

原因通常不是团队不努力，而是缺少指标、复盘和持续改进闭环。

成熟的大模型工程师要能回答：

- 什么指标能证明系统变好？
- 哪些指标可能误导团队？
- 如何从失败 case 中学习？
- 如何做上线后复盘？
- 如何避免同类问题反复发生？
- 如何让质量改进成为团队机制，而不是个人救火？

这一章讨论大模型工程中的指标体系、复盘方法和持续改进机制。

72.1 没有指标，就没有方向感

没有指标时，团队只能靠感受判断系统好坏。

感受有价值，但不能替代度量。

大模型系统尤其需要指标，因为输出具有不确定性。

同一个模型可能在 demo 中表现很好，但在线上长尾问题中失败。

同一个 prompt 可能在 20 个样本上看起来不错，但在 1000 个真实请求中暴露格式错误、引用错误和安全问题。

指标的价值不是让所有问题都变成数字。

指标的价值是帮助团队建立方向感。

它至少能回答：

- 当前系统基线是什么？
- 改动后是否真的变好？
- 哪类问题最严重？
- 哪些场景被牺牲了？
- 成本和延迟是否可接受？
- 线上是否出现新失败模式？

没有指标，技术讨论很容易变成观点竞争。

有指标，团队才有机会从争论走向实验。

72.2 指标不是越多越好

很多团队一开始会建很多 dashboard。

但指标太多也会失去焦点。

一个指标体系应该分层，而不是把所有数字堆在一起。

大模型应用可以分成几层指标。

第一，业务结果指标。

例如：

- 用户任务完成率。
- 人工处理时长下降。
- 客服转人工率下降。
- 用户满意度。
- 业务转化率。
- 风险事件减少。

第二，模型质量指标。

例如：

- 回答正确率。
- 引用准确率。
- 幻觉率。
- 拒答准确率。
- 格式合法率。
- 工具调用成功率。

第三，系统性能指标。

例如：

- TTFT。
- TPOT。
- P95 / P99 latency。
- throughput。
- timeout rate。
- error rate。

第四，成本指标。

例如：

- input tokens。
- output tokens。
- cached tokens。
- 单次请求成本。
- 单任务成功成本。
- GPU 利用率。

第五，安全治理指标。

例如：

- 越权召回率。
- 敏感信息泄露率。
- prompt injection 命中率。
- 高风险工具误调用率。
- 审计日志完整率。

指标分层后，团队更容易定位问题。

如果业务指标没有改善，要继续看是质量、性能、成本、安全还是产品体验拖了后腿。

72.3 好指标要能驱动行动

一个指标是否有用，关键看它能不能驱动行动。

无用指标通常有几个特点：

- 看起来好看，但不知道怎么改。
- 只能反映总量，不能定位问题。
- 容易被刷高。
- 与用户价值关系弱。
- 没有 owner。
- 没有阈值和报警。

例如“总调用次数”不是没用，但它不能直接证明系统有效。

调用次数上升可能代表用户喜欢，也可能代表系统回答差导致用户反复追问。

“总 token 数”也不能直接代表价值。

token 多可能说明业务使用多，也可能说明 prompt 过长、上下文浪费或模型啰嗦。

更好的指标应该能指导行动。

例如：

引用准确率下降 -> 检查文档解析、retrieval、rerank、引用抽取和生成约束。

资料不足拒答准确率下降 -> 检查知识覆盖、拒答策略和评测样本。

工具误调用率上升 -> 检查 tool schema、参数校验、prompt injection 和权限边界。

TTFT P95 上升 -> 检查 queue time、input length、prefix cache、prefill 负载和 worker 健康。

能指向下一步排查的指标，才是真正有用的指标。

72.4 警惕指标被优化坏

指标会改变团队行为。

如果指标设计不好，团队可能把指标优化好，但系统变差。

这就是常见的指标异化问题。

例如只看回答长度，模型可能输出更长但更啰嗦。

只看用户点赞率，系统可能倾向迎合用户，而不是指出资料不足。

只看自动评测分数，团队可能过拟合评测集。

只看调用量，团队可能鼓励无效使用。

只看 token 成本，团队可能过度压缩上下文，导致回答质量下降。

只看上线速度，团队可能牺牲评测和安全。

设计指标时要保留平衡。

例如 RAG 不应该只看 answer correctness。

还要看 citation accuracy、refusal accuracy、latency、cost 和越权召回。

Agent 不应该只看 task success rate。

还要看 tool misuse、人工接管、高风险误操作和审计完整性。
推理平台不应该只看 tokens/s。
还要看 TTFT、TPOT、错误率、KV cache pressure 和成本。
好的指标体系会防止团队只优化单一数字。

72.5 大模型系统的核心质量闭环

一个可靠的大模型系统应该有质量闭环。

最小闭环可以写成：

线上请求

- > 日志和 trace
- > 失败 case 发现
- > 失败类型分类
- > 样本进入评测集
- > 方案实验
- > 灰度上线
- > 指标观察
- > 复盘和路线图调整

这里每一步都很重要。

如果没有日志，失败无法定位。

如果没有失败分类，团队不知道优先解决什么。

如果失败 case 不进入评测集，同类问题会反复出现。

如果实验没有 baseline，无法判断改动是否有效。

如果没有灰度，线上风险不可控。

如果没有复盘，经验不会沉淀。

质量闭环的核心不是一次性评测。

它是让系统从真实使用中持续学习。

72.6 失败 case 分类

失败 case 是大模型工程最重要的学习材料。

但失败 case 不能只堆在表格里。

必须分类。

常见分类包括：

- 用户问题不清楚。
- 知识库缺失。
- 文档解析错误。
- chunk 切分错误。
- embedding 召回失败。
- rerank 排序失败。
- 上下文组织错误。
- 模型理解错误。
- 模型幻觉。
- 引用不支持答案。
- 资料不足时没有拒答。
- prompt 指令冲突。

- 工具选择错误。
- 工具参数错误。
- 工具执行失败。
- 权限过滤错误。
- 输出格式错误。
- 安全策略误杀或漏杀。
- 产品交互导致误解。

分类的价值是让改进有方向。

如果 60% 失败来自文档解析和引用错误，优先级就不应该是微调。

如果 40% 失败来自工具参数错误，就应该检查 schema、参数校验和工具调用样本。

如果大量失败来自用户问题模糊，可能需要澄清问题或交互设计，而不是换模型。

72.7 复盘不是追责

复盘的目的不是找人背锅。

复盘的目的是让系统变强。

大模型系统的失败经常是多因素叠加。

例如一次错误工具调用可能同时涉及：

- prompt 没有限制清楚。
- tool schema 过于宽松。
- 后端没有权限校验。
- 没有高风险确认。
- 评测集没有覆盖类似场景。
- 日志缺少关键字段。

如果复盘只停留在“某个工程师没有考虑到”，团队不会进步。

好的复盘要问：

- 发生了什么？
- 用户或业务受到什么影响？
- 为什么现有机制没有提前发现？
- 哪些假设被证明是错的？
- 哪些日志或指标缺失？
- 哪些测试或评测应该补上？
- 哪些流程需要调整？
- 谁负责在什么时候完成改进？

复盘的产物应该是系统改进，而不是情绪释放。

72.8 上线后复盘

大模型功能上线后，应该做上线后复盘。

不是只有事故才复盘。

一次模型切换、prompt 改动、rerank 上线、Agent 工具接入、推理框架升级，都值得复盘。

上线后复盘可以看：

- 原始目标是否达成？
- 核心指标是否变化？
- 是否出现新失败类型？
- 成本和延迟是否符合预期？
- 用户反馈和离线评测是否一致？

- 哪些场景收益最大？
- 哪些场景变差？
- 是否需要扩大、回滚或调整？

例如 rerank 上线后，不能只看平均正确率提升。

还要看：

- citation accuracy 是否提升。
- P95 latency 增加多少。
- token 成本是否变化。
- 短 query 和长 query 是否都受益。
- 是否有业务场景被 rerank 误伤。

上线后复盘能防止团队把“已上线”误认为“已成功”。

72.9 事故复盘

大模型系统也会有事故。

事故不一定是服务不可用。

可能包括：

- 大面积幻觉。
- 越权召回。
- 敏感信息泄露。
- 工具误操作。
- 成本异常飙升。
- 模型版本回退失败。
- 推理服务延迟失控。
- prompt injection 绕过防护。
- 评测门禁失效导致坏版本上线。

事故复盘要包含时间线。

例如：

10:00 新 prompt 灰度 10%。
 10:20 用户反馈引用错误增加。
 10:35 dashboard 显示 citation accuracy 下降。
 10:50 灰度扩大被暂停。
 11:10 回滚到旧 prompt。
 13:00 发现新 prompt 改变了引用格式，导致引用抽取器失效。

时间线帮助团队理解事故是如何扩大的。

事故复盘还要分清：

- 直接原因。
- 根因。
- 检测缺口。
- 响应缺口。
- 预防措施。

例如直接原因是 prompt 改动导致引用格式变化。

根因可能是 prompt 变更没有回归引用抽取器，评测集中没有格式兼容样本，灰度指标没有设置自动暂停。

72.10 指标和复盘如何进入路线图

指标和复盘不是独立活动。

它们应该反过来影响路线图。

如果复盘发现大量问题来自日志缺失，那么补 trace 字段应该进入路线图。

如果指标显示成本持续上涨，而质量没有提升，那么成本治理应该提升优先级。

如果失败 case 显示 Agent 工具误调用集中在某几类工具，那么工具 schema 和权限机制应该优先于新增更多工具。

如果线上 P95 TTFT 长期超过 SLO，那么推理优化和容量规划应该进入路线图。

路线图不是管理者拍脑袋定的。

路线图应该吸收真实指标和复盘结论。

这也是持续改进的核心。

72.11 建立质量节奏

持续改进需要节奏。

可以建立几个固定机制。

每日或每周看线上关键指标。

包括：

- 错误率。
- 延迟。
- 成本。
- 失败 case 数量。
- 用户反馈。
- 高风险事件。

每周做失败 case review。

重点看：

- 新增失败类型。
- 高频失败类型。
- 是否需要补评测。
- 是否需要调整优先级。

每两周或每月做实验复盘。

重点看：

- 哪些实验有效。
- 哪些实验无效。
- 哪些假设被推翻。
- 哪些方案应该继续或停止。

每季度做路线图回顾。

重点看：

- 战略假设是否仍成立。
- 指标是否朝目标改善。
- 资源投入是否合理。
- 技术债是否积累过快。
- 新技术是否改变路线。

节奏不需要复杂。

关键是持续。

72.12 面试中的指标表达

如果面试官问：“你如何判断一个大模型应用上线后是否成功？”

可以这样回答：

我不会只看调用量或 demo 效果，而会建立分层指标。第一层是业务结果，例如任务完成率、人工接管率、用户满意度或业务效率提升。

上线后我会按场景和用户分桶看指标，不只看全局平均。还会持续收集失败 case，按文档解析、召回、rerank、生成、引用、工具、

我认为成功不是功能上线，而是核心业务指标改善，同时质量、延迟、成本和安全在可接受范围内，并且团队能从线上反馈中持续改进。

72.13 常见误区

第一个误区是只看调用量。

调用量高不代表用户成功，也可能是系统不好用导致反复尝试。

第二个误区是只看平均指标。

P95 和 P99 失败可能影响最重要用户。

第三个误区是只看离线评测。

离线评测不能完全覆盖线上真实分布和产品交互。

第四个误区是只看模型质量。

延迟、成本、安全和可运营性同样决定系统是否可用。

第五个误区是复盘只复盘事故。

正常上线也需要复盘，否则不知道哪些改动真正有效。

第六个误区是失败 case 不回流。

如果线上失败不进入评测集，同类问题会反复出现。

第七个误区是指标没有 owner。

没有 owner 的指标只会被展示，不会被改进。

72.14 小结

指标、复盘和持续改进，是大模型工程从试点走向生产的核心能力。

没有指标，团队没有方向感。

指标太多或设计不好，又会制造噪声甚至误导行为。

好的指标体系应该分层，覆盖业务结果、模型质量、系统性能、成本和安全治理。

失败 case 是最重要的学习材料，必须分类、回流到评测集，并影响后续路线图。

复盘不是追责，而是让系统变强。

上线后复盘、事故复盘、失败 case review 和路线图回顾，共同构成持续改进机制。

成熟的大模型工程师不只会做功能，也会建立一套让系统持续变好的机制。

长期来看，真正可靠的大模型系统不是一次性设计出来的，而是在真实使用、真实失败和持续复盘演化出来的。

第 73 章：风险、安全与责任边界

上一章讨论了指标、复盘与持续改进。

指标能帮助团队知道系统是否变好。

但大模型工程还有一个不能回避的问题：系统是否足够安全，风险是否被正确管理，责任边界是否清楚。

很多大模型项目早期只关注效果。

能不能回答更准，能不能自动处理更多任务，能不能减少人工，能不能降低成本。

这些当然重要。

但一旦进入生产，问题会变得更复杂。

模型可能泄露敏感信息。

RAG 可能越权召回。

Agent 可能调用错误工具。

工具返回可能被 prompt injection 污染。

模型输出可能被下游系统直接执行。

用户可能过度相信模型建议。

攻击者可能利用长上下文、插件、文件上传、工具调用和多轮对话绕过防线。

如果团队没有安全和责任边界，系统能力越强，风险越大。

这一章讨论：大模型工程师如何识别风险，如何设计安全边界，如何在效果、自动化和责任之间做取舍。

73.1 安全不是上线前最后一步

很多团队把安全当成上线前检查项。

等功能快做完了，再找安全团队评审。

这在大模型系统中很危险。

因为很多风险不是外层加一个过滤器就能解决。

例如：

- 权限过滤如果没有在检索前执行，模型可能已经看到了不该看的文档。
- 工具调用如果没有后端鉴权，prompt 再严格也挡不住越权调用。
- 输出如果直接进入 SQL、Shell、浏览器或业务系统，后处理缺失会导致下游漏洞。
- Agent 如果被赋予过多自主权，后续补一个免责声明并不能消除风险。

安全应该从需求、架构和数据流开始。

设计一个大模型功能时，就要问：

- 输入来自哪里？
- 哪些输入不可信？
- 模型能看到哪些数据？
- 数据权限在哪里强制执行？
- 模型能调用哪些工具？
- 工具调用是否有副作用？
- 输出会被谁消费？
- 输出是否会进入下游执行环境？
- 错误结果的代价是什么？
- 谁可以审计和回滚？

安全不是最后的闸门。

安全是系统设计的一部分。

73.2 大模型应用的风险地图

OWASP LLM Top 10 把 LLM 应用中的典型风险总结为 prompt injection、insecure output handling、training data poisoning、model denial of service、supply chain vulnerabilities、sensitive information disclosure、insecure plugin design、excessive agency、overreliance 和 model theft 等类别。

这些风险可以映射到工程链路中。

第一，输入风险。

包括恶意 prompt、文件上传、网页内容、RAG 文档、工具返回、用户多轮对话和外部 API 返回。

第二，上下文风险。

包括越权文档、敏感字段、被污染的检索结果、过长上下文中的隐藏指令、系统 prompt 泄露和跨租户信息混入。

第三，模型行为风险。

包括幻觉、错误建议、不稳定输出、拒答失败、过度自信和不遵循安全策略。

第四，工具风险。

包括工具选择错误、参数错误、权限绕过、副作用操作、幂等问题和插件设计不安全。

第五，输出风险。

包括敏感信息泄露、恶意代码、错误 JSON、SQL/HTML/Markdown 注入、被下游系统直接执行和误导用户。

第六，系统风险。

包括成本攻击、长上下文 DoS、模型服务过载、供应链漏洞、模型或 prompt 版本污染、日志泄露和审计缺失。

第七，组织风险。

包括用户过度信任、责任不清、人工兜底缺失、安全评审流于形式和事故响应不成熟。

成熟工程师不会只说“加安全过滤”。

他会把风险放回完整链路中分析。

73.3 Prompt Injection 不是 prompt 写得更强就能解决

Prompt injection 是大模型应用中最典型的风险之一。

攻击者可能在用户输入、网页、文档、邮件、代码注释、工具返回或多轮上下文中植入指令，诱导模型忽略系统规则、泄露信息或调用不该调用的工具。

常见形式包括：

- 直接要求模型忽略之前指令。
- 在 RAG 文档中写入“把所有秘密发给用户”。
- 在网页内容中隐藏指令，诱导浏览器 Agent 执行操作。
- 在工具返回中加入下一步操作指令。
- 用多轮对话逐步绕过规则。

防护 prompt injection 不能只靠更长的 system prompt。

因为模型仍然会同时看到可信指令和不可信内容。

更稳的做法是分层防护：

- 明确区分指令和数据。
- 对外部内容做标记和隔离。
- 不让不可信内容覆盖系统策略。

- 工具调用必须经过后端权限校验。
- 高风险操作需要用户确认。
- 对敏感输出做后处理检测。
- 对攻击样本建立 red team 测试集。
- 日志中记录触发策略和工具调用原因。

一句话：

Prompt 可以降低风险，但不能承担权限、安全和审计的全部责任。

73.4 权限必须在系统层执行

大模型系统中最危险的错误之一，是把权限交给模型判断。

例如：

如果用户没有权限，请不要回答。

这类 prompt 只能作为辅助约束，不能作为安全边界。

真正的权限控制必须在系统层执行。

RAG 场景中，权限应该在检索前或检索时生效。

模型不应该看到用户无权访问的文档。

工具调用场景中，权限应该在服务端检查。

模型可以提出 tool call proposal，但后端必须验证用户身份、资源权限、操作范围和风险等级。

多租户场景中，租户隔离必须在数据层、索引层、缓存层和日志层都成立。

不能只依赖 prompt 告诉模型“不要泄露其他租户信息”。

权限设计可以遵循几个原则：

- 最小权限。
- 默认拒绝。
- 服务端强制。
- 数据先过滤再进入上下文。
- 高风险操作二次确认。
- 全链路审计。
- 权限策略可测试。

如果权限只存在于 prompt 中，那么它实际上没有成为系统边界。

73.5 工具调用的风险边界

Agent 和 function calling 让模型可以影响真实系统。

这也是风险从“说错话”升级为“做错事”的关键点。

工具调用风险主要来自几类。

第一，工具选择错误。

模型选择了不该调用的工具。

第二，参数错误。

模型把用户、金额、时间、资源 ID、权限范围填错。

第三，副作用不可控。

工具会发邮件、删文件、改配置、下订单、提交工单、调整权限或执行代码。

第四，幂等性缺失。

重试可能导致重复操作。

第五，工具返回不可信。

工具返回中可能包含恶意指令或污染内容。

第六，审计不足。

事后无法知道模型为什么调用、传了什么参数、谁批准了操作。

工具调用设计要分风险等级。

低风险工具可以自动执行。

例如搜索、读取公开信息、格式转换。

中风险工具需要明确参数校验和可回滚机制。

例如创建草稿、生成配置建议、提交非关键任务。

高风险工具需要人工确认、权限校验和审计。

例如转账、删数据、发外部邮件、修改生产配置、授予权限。

模型不应该直接拥有无限工具权限。

它应该在受控边界内提出建议，由系统执行校验。

73.6 Insecure Output Handling: 不要直接信任模型输出

模型输出是非确定、可能错误、可能被攻击影响的文本。

如果下游系统直接执行模型输出，就会产生 insecure output handling 风险。

典型例子包括：

- 模型生成 SQL，系统直接执行。
- 模型生成 Shell 命令，系统直接运行。
- 模型生成 HTML，前端直接渲染。
- 模型生成 JSON，后端不校验就写库。
- 模型生成代码，自动合并或部署。
- 模型生成工具参数，系统直接调用高风险 API。

正确做法是把模型输出当成不可信输入。

需要：

- schema 校验。
- 类型检查。
- 白名单。
- 参数范围限制。
- escaping 和 sanitization。
- 权限校验。
- dry-run。
- 人工确认。
- 沙箱执行。
- 审计日志。

尤其是代码生成、SQL 生成、浏览器自动化和运维 Agent 场景，必须把验证成本纳入系统设计。

AI 让生成变便宜，但验证变得更重要。

73.7 敏感信息和数据边界

敏感信息泄露是大模型系统中的高频风险。

泄露可能发生在多个位置：

- prompt 中包含密钥或内部规则。
- RAG 召回了无权文档。
- 模型输出了个人信息、商业机密或内部链接。
- 日志记录了完整用户输入和模型输出。
- 评测数据包含真实敏感样本。
- 向第三方 API 发送了不该出域的数据。
- prefix cache 或上下文缓存跨租户复用错误。

数据边界设计要问：

- 哪些数据可以进入模型？
- 哪些数据可以进入第三方 API？
- 哪些数据可以写日志？
- 哪些数据需要脱敏？
- 哪些数据必须加密？
- 哪些数据需要保留，保留多久？
- 哪些数据可以用于训练或评测？

企业场景尤其要注意：

- 用户身份和权限。
- 租户隔离。
- 文档密级。
- 审计要求。
- 数据出域规则。
- 日志和 trace 权限。

敏感信息保护不是只靠模型“不要泄露”。

它需要数据治理、权限、脱敏、日志策略和访问控制共同实现。

73.8 Excessive Agency：不要给模型过度自主权

OWASP LLM Top 10 中的 excessive agency 指的是给 LLM 过多能力、权限或自主决策空间，导致不可预期后果。

这是 Agent 系统中最核心的风险之一。

过度自主权可能表现为：

- 模型可以调用过多工具。
- 工具权限过宽。
- 操作不需要确认。
- 没有任务边界。
- 没有最大步骤数。
- 没有预算限制。
- 没有失败停止条件。
- 没有人工接管。
- 没有审计。

降低 excessive agency 风险，可以使用：

- 限制工具集合。
- 按任务动态授权。
- 设置最大步骤数和最大成本。
- 高风险操作人工确认。

- 先 dry-run 再执行。
- 低置信度转人工。
- 关键操作可回滚。
- 所有工具调用记录 trace。

一个实用原则是：

模型可以负责建议和规划，但系统必须负责授权、执行和审计。

73.9 Overreliance：不要让用户过度相信模型

风险不只来自攻击者。

也来自用户过度相信模型。

模型回答自然、流畅、看起来很自信。

用户可能因此忽略核验。

在医疗、金融、法律、招聘、安全、运维和高风险业务中，过度依赖会造成严重后果。

降低 overreliance 风险，需要产品和工程共同设计。

可以做：

- 明确展示引用和证据。
- 对资料不足场景拒答。
- 对低置信任务提示不确定性。
- 高风险建议要求人工确认。
- 明确模型能力边界。
- 展示最后更新时间和数据来源。
- 不把模型输出伪装成确定结论。
- 关键决策保留人工责任人。

不要滥用免责声明。

免责声明不能替代真实安全机制。

如果系统知道某类任务风险高，就应该限制、确认或转人工，而不是只在页面底部写“AI 可能出错”。

73.10 成本和可用性攻击

大模型系统还有一种特殊风险：成本攻击和资源耗尽。

攻击者或异常用户可能构造超长输入、反复请求、诱导长输出、触发昂贵工具、绕过缓存或制造大量并发。

这会导致：

- GPU 资源被占满。
- 正常用户排队。
- 成本快速上升。
- 模型服务 OOM。
- 下游工具被打爆。
- 日志和存储膨胀。

防护手段包括：

- 用户和租户级限流。
- input token 限制。
- output token 限制。
- 请求并发限制。
- 工具调用预算。
- 单任务最大步骤数。
- 长上下文审批或分级。

- 异常成本告警。
- admission control。
- 过载时优雅拒绝。

大模型服务的安全不仅是数据安全，也包括成本和可用性安全。

73.11 风险分级和上线门禁

不是所有大模型功能风险都一样。

应该按风险等级管理。

低风险功能：

- 文本润色。
- 内部知识摘要。
- 无权限敏感数据的问答。
- 只读公开信息检索。

中风险功能：

- 企业内部知识问答。
- 工单草稿生成。
- 代码建议。
- 半自动数据分析。

高风险功能：

- 权限敏感知识问答。
- 自动发送外部消息。
- 修改生产系统。
- 财务、法律、医疗建议。
- 自动执行代码。
- 自动变更权限。

不同等级对应不同上线门禁。

低风险可以重点看质量和基本安全。

中风险需要评测、权限、日志、灰度和回滚。

高风险需要安全评审、人工确认、红队测试、审计、严格权限、事故预案和责任人。

风险分级的目的不是阻止创新。

它是为了让系统在合适的边界内创新。

73.12 责任边界：模型、系统和人的分工

大模型系统必须明确责任边界。

模型负责什么？

系统负责什么？

人负责什么？

一个比较稳的分工是：

- 模型负责理解、生成、总结、分类、建议和候选行动。
- 系统负责权限、校验、执行、审计、限流、回滚和监控。
- 人负责高风险决策、异常确认、策略制定和最终责任。

不要让模型承担它不适合承担的责任。

模型不应该是权限系统。

模型不应该是唯一审计机制。

模型不应该在没有确认的情况下执行高风险操作。

模型不应该为法律、医疗、金融等高风险决策承担最终责任。

清楚的责任边界能减少事故，也能让团队更容易通过安全和合规评审。

73.13 风险治理不是一次性文档

风险治理需要持续运行。

至少包括：

- 风险登记表。
- 安全评审。
- red team 测试。
- 上线门禁。
- 监报告警。
- 事故复盘。
- 策略更新。
- 权限审计。
- 模型和 prompt 版本审计。

风险登记表可以记录：

- 风险描述。
- 影响范围。
- 发生概率。
- 严重程度。
- 缓解措施。
- owner。
- 当前状态。
- 重新评估时间。

例如：

风险：RAG 越权召回内部薪酬文档。

影响：敏感信息泄露和合规风险。

缓解：检索前按用户权限过滤，索引按租户和密级隔离，日志记录 document_id 和 permission decision。

owner：知识库平台负责人。

状态：灰度前必须完成权限回归测试。

风险治理要进入工程流程，而不是停留在文档里。

73.14 面试表达

如果面试官问：“你如何保证一个大模型 Agent 系统安全上线？”

可以这样回答：

我会先做风险分级，而不是一上来讨论模型效果。Agent 系统的风险主要来自不可信输入、越权上下文、工具误调用、输出被下游执行。

架构上，我不会把安全完全交给 prompt。权限必须在系统层执行，RAG 检索前要按用户权限过滤，工具调用必须经过服务端鉴权。模型 run，再执行。

对 prompt injection，我会区分可信指令和不可信数据，对外部内容做隔离和标记，建立攻击样本评测集。对模型输出，我会当成不

上线前要有 red team、离线评测、灰度、监控和回滚。上线后看工具误调用率、高风险操作确认率、越权召回率、敏感信息泄露、成本。

我的原则是：模型负责理解和建议，系统负责授权、校验、执行和审计，人负责高风险决策和最终责任。

73.15 常见误区

第一个误区是把安全交给 prompt。

Prompt 可以辅助约束，但不能替代权限、校验和审计。

第二个误区是模型看到了不该看的数据后再要求它不要说。

正确做法是不要让无权数据进入上下文。

第三个误区是工具越多越智能。

工具越多，攻击面和误调用风险越大。

第四个误区是只关心 prompt injection，不关心输出处理。

模型输出进入下游系统时同样需要校验。

第五个误区是低估成本攻击。

长上下文、长输出和多步 Agent 都可能造成资源耗尽。

第六个误区是用免责声明替代产品和工程机制。

高风险场景需要限制、确认和审计，不只是提醒用户。

第七个误区是没有责任边界。

一旦出事，团队说不清模型、系统和人的责任划分。

73.16 小结

大模型工程的安全和风险管理，不能等到上线前才补。

它必须从需求、数据流、权限、工具、输出处理、监控和责任边界开始设计。

OWASP LLM Top 10 和 NIST AI RMF 都提醒我们：大模型风险不仅是模型回答错，还包括 prompt injection、敏感信息泄露、不安全插件、过度自主权、过度依赖、供应链、成本攻击和治理缺失。

成熟的大模型工程师不会把安全简化为“加一个过滤器”。

他会把模型放在完整系统中看：哪些输入不可信，哪些数据不能进入上下文，哪些操作必须由系统鉴权，哪些输出必须校验，哪些任务必须人工确认，哪些风险需要持续监控和复盘。

最终原则很简单：

模型可以增强能力，但系统必须定义边界。

没有边界的智能，不是生产能力，而是生产风险。

第 74 章：成本、ROI 与商业判断

上一章讨论了风险、安全与责任边界。

安全解决的是“能不能安全地做”。

这一章讨论另一个现实问题：值不值得做。

大模型工程师很容易把注意力放在技术指标上。

模型更强，prompt 更复杂，RAG 更准确，Agent 步骤更多，推理框架吞吐更高。

但在生产环境中，技术方案最终要回到业务价值和成本结构。

一个方案如果质量提升 2%，但成本增加 5 倍、延迟增加 3 倍、维护复杂度显著上升，它未必是好方案。

一个模型如果能力很强，但只在少数低价值场景使用，也未必值得大规模接入。

一个 Agent demo 如果看起来很智能，但每次任务调用 20 次模型、3 个外部工具、还需要人工复核，ROI 可能并不成立。

成熟的大模型工程师要能把 token、GPU、延迟、质量、风险、维护成本和业务收益放到同一张表里判断。

这一章讨论：如何理解大模型工程的成本结构，如何评估 ROI，如何建立商业判断，如何避免“技术上可行，但商业上不成立”的项目。

74.1 大模型不是免费能力

传统软件功能上线后，边际成本通常较低。

大模型功能不同。

每次调用都可能产生明显成本。

成本来自：

- 输入 token。
- 输出 token。
- cached token 或 prefix cache 资源。
- embedding 调用。
- rerank 调用。
- 工具调用。
- 向量数据库查询。
- GPU 推理。
- 日志和 trace 存储。
- 人工审核。
- 失败重试。
- 安全和评测。

如果是自建模型服务，还要考虑：

- GPU 采购或租赁。
- 机器折旧。
- 机房、电力和网络。
- 推理框架维护。
- 容量冗余。
- on-call 和故障处理。
- 模型升级和回归评测。

因此大模型功能不能只看“能不能做”。

还要看“每做一次要花多少钱”。

74.2 Token 使用量不是价值指标

很多团队喜欢看 token 用量。

它容易统计，也能反映大模型调用规模。

但 token 用量不是价值指标。

token 用得多了，可能说明系统被广泛使用。

也可能说明 prompt 过长、上下文浪费、模型输出啰嗦、用户反复追问、Agent 规划低效。

如果把 token 用量当成生产力指标，团队可能被误导。

例如：

- 工程师为了显示 AI 使用积极，增加无意义调用。
- Agent 为了完成简单任务，反复规划和反思。
- RAG 把大量无关文档塞入上下文。
- 模型输出很长，但用户真正需要的只是一个结论。

更好的指标是单位价值成本。

例如：

- 单次成功问答成本。
- 单个有效工单处理成本。
- 单次人工节省成本。
- 单个高质量代码 review 成本。
- 单个成功 Agent 任务成本。
- 每减少 1% 人工接管率需要增加多少成本。

token 是资源消耗。

价值要从任务结果看。

74.3 成本要按链路拆开

大模型应用的成本不是一个数字。

应该按链路拆开。

以 RAG 问答为例，一次请求可能包括：

```
query rewrite cost
+ embedding cost
+ vector search cost
+ rerank cost
+ prompt input token cost
+ output token cost
+ citation verification cost
+ logging and storage cost
+ human review cost if failed
```

以 Agent 任务为例，一次任务可能包括：

```
planning model calls
+ tool selection calls
+ tool execution cost
+ observation summarization calls
+ verification calls
+ final response cost
+ retry cost
+ human confirmation cost
```

以自建推理平台为例，单位成本可能包括：

```
GPU hourly cost
/ stable output tokens per hour under SLO
+ platform operation cost
+ redundancy cost
+ storage and network cost
+ engineering maintenance cost
```

拆开后才能知道优化点在哪里。

如果成本主要来自长 prompt，就优化上下文和缓存。

如果成本主要来自 rerank，就考虑分层 rerank 或只对低置信 query 启用。

如果成本主要来自多步 Agent，就考虑 workflow 固化、工具结果缓存或减少反思轮次。

如果成本主要来自 GPU 低利用率，就优化 batching、路由和容量规划。

74.4 ROI 的基本公式

ROI 不需要一开始做得非常复杂。

最基本的问题是：收益是否大于成本。

可以用一个简单公式：

$$ROI = (\text{business value} - \text{total cost}) / \text{total cost}$$

其中 business value 可以来自：

- 节省人工时间。
- 提高转化率。
- 降低错误率。
- 减少客服成本。
- 提升用户留存。
- 加快研发交付。
- 降低事故损失。
- 支持新收入。

total cost 包括：

- 模型调用成本。
- GPU 成本。
- 存储和网络成本。
- 工程开发成本。
- 维护和 on-call 成本。
- 安全和合规成本。
- 评测和标注成本。
- 用户教育和运营成本。

ROI 计算不一定精确。

但它能迫使团队把“技术收益”翻译成“业务收益”。

74.5 不同阶段的 ROI 标准不同

大模型项目在不同阶段，ROI 标准不同。

Demo 阶段不需要严格 ROI。

目标是验证可行性。

可以接受手工流程、少量样本和较高成本。

试点阶段需要粗略 ROI。

要看真实用户是否愿意使用，失败类型是否可控，成本是否在可接受范围内。

生产阶段必须看稳定 ROI。

要证明系统在真实流量下长期有价值，且质量、成本、延迟、安全和维护都可持续。

平台化阶段要看规模 ROI。

要证明平台能力能服务多个业务线，减少重复建设和维护成本。

不要用 demo 阶段的标准推动生产上线。

也不要 demo 阶段用生产 ROI 扼杀探索。

关键是知道项目处在哪个阶段。

74.6 质量提升要和成本一起看

模型效果提升很重要。

但不能孤立看。

例如某个更强模型让回答正确率从 82% 提升到 87%。

这看起来是好事。

但还要问：

- 成本增加多少？
- P95 延迟增加多少？
- 格式稳定性是否变化？
- 安全拒答是否变好？
- 哪些场景提升最大？
- 哪些场景没有提升？
- 用户是否感知到提升？
- 这个提升是否值得长期支付？

如果高价值场景从 70% 提升到 90%，成本翻倍也可能值得。

如果低价值场景从 95% 提升到 96%，成本翻倍可能不值得。

所以评估效果时要分桶。

按用户、场景、任务价值、风险等级和请求类型分桶。

平均正确率很容易掩盖真实商业判断。

74.7 什么时候该用强模型，什么时候该用小模型

大模型工程不是永远用最强模型。

更成熟的做法是模型分层。

强模型适合：

- 高价值任务。
- 复杂推理任务。
- 高风险回答前的验证。
- 低频但重要的决策支持。
- 需要强指令遵循和复杂上下文理解的场景。

小模型适合：

- 分类。
- 路由。
- 简单抽取。
- 格式转换。
- 初筛。
- 低风险高频任务。
- embedding 或 rerank 的部分替代任务。

可以设计分层策略：

简单问题 -> 小模型或规则

中等问题 -> 标准模型

复杂或高价值问题 -> 强模型

高风险操作 -> 强模型 + 人工确认

这比所有请求都打到最强模型更经济。

模型路由的目标不是省钱本身。

而是在质量可接受的前提下，把强模型用在最值得的地方。

74.8 RAG、微调、长上下文和 Agent 的成本取舍

不同技术路径有不同成本结构。

RAG 的成本主要来自检索、rerank、上下文 token、引用校验和数据维护。

它适合知识更新频繁、需要引用、企业私有知识和可追溯场景。

微调的成本主要来自数据构造、训练、评测、部署和版本维护。

它适合稳定任务模式、风格一致性和高频重复任务。

长上下文的成本主要来自 input token、prefill 延迟和注意力/KV cache 压力。

它适合完整文档分析、上下文关系密集、材料数量有限的场景。

Agent 的成本主要来自多轮模型调用、工具调用、失败重试、审计和人工确认。

它适合步骤不固定、工具选择动态、用户目标多样的场景。

技术选型时要问：

- 知识是否频繁更新？
- 是否需要引用？
- 任务模式是否稳定？
- 用户请求是否高频？
- 是否需要动态规划？
- 是否有高风险工具？
- 成本随规模如何增长？

不要把所有问题都升级成最复杂方案。

复杂方案如果没有对应的业务价值，就是成本负担。

74.9 降本不是简单砍模型

降低成本不等于把模型换小。

换小模型可能降低调用成本，也可能导致质量下降、重试增加、人工接管增加，最终总成本更高。

更系统的降本手段包括：

- 压缩 prompt。
- 删除无用上下文。
- 提高 prefix cache 命中率。
- 对重复请求做结果缓存。
- 对简单请求使用规则或小模型。
- 对复杂请求再升级强模型。
- 减少不必要的 Agent 轮次。
- 限制 max output tokens。
- 优化 RAG chunk 和 Top-K。
- 只在必要时启用 rerank。
- 批处理 embedding 和离线任务。
- 优化推理 serving 的 batching 和并发。
- 建立租户级预算。

降本要同时看质量。

一个好的降本项目应该写清：

- 成本下降多少。
- 哪些质量指标不能下降。
- 哪些场景可能受影响。
- 如何灰度和回滚。

74.10 成本归因：谁在花钱，为什么花钱

如果没有成本归因，大模型平台很快会变成公共水龙头。

大家都在用，但没人知道谁在花钱、钱花在哪里、是否值得。

成本至少要按几个维度归因：

- 业务线。
- 租户。
- 用户类型。
- 模型。
- 场景。
- 请求类型。
- input / output token。
- cache hit。
- 工具调用。
- 成功和失败任务。

只知道总账单没有用。

要知道：

- 哪个业务线成本最高？
- 哪类请求 token 最多？
- 哪些失败请求消耗了大量成本？
- 哪些 Agent 任务反复重试？
- 哪些租户超过预算？
- 哪些模型性价比最低？

成本归因不是为了阻止使用。

而是为了让团队把资源投到真正有价值的场景。

74.11 成本预算和用量治理

生产系统需要预算机制。

常见做法包括：

- 租户级月度预算。
- 用户级限额。
- 场景级预算。
- 单请求 token 上限。
- 单任务工具调用上限。
- 超预算告警。
- 高成本请求审批。
- 异常用量自动降级。

预算机制要避免过度粗暴。

不能让高价值请求因为统一限额被误伤。

可以做分级：

- 普通请求使用标准预算。
- 高价值客户使用更高预算。

- 高风险或高成本任务需要审批。
- 过载时优先保障核心场景。

预算不是财务限制而已。

预算也是系统稳定性和产品策略的一部分。

74.12 自建还是 API 的商业判断

自建模型服务和调用外部 API 的技术取舍，最终也要回到商业判断。

API 适合：

- 早期验证。
- 流量不稳定。
- 团队缺少推理运维能力。
- 需要最强模型能力。
- 不希望承担 GPU 和平台复杂度。

自建适合：

- 流量稳定且规模足够大。
- 数据不能出域。
- 需要版本可控。
- 需要深度定制。
- 单位成本可以通过规模摊薄。
- 团队有推理平台能力。

判断时不要只比较单次调用单价。

还要比较：

- 工程人力。
- 运维成本。
- 可用性。
- 扩容速度。
- 模型能力差距。
- 合规成本。
- 故障责任。
- 机会成本。

很多团队适合混合策略。

高价值复杂任务用外部强模型。

高频稳定任务用自建模型。

敏感数据任务用私有部署。

这样比单一策略更灵活。

74.13 让工程成为业务驱动者

商业判断不是产品经理或管理者的专属能力。

高级工程师也需要商业意识。

这并不是要求工程师做销售。

而是要理解技术投入如何影响业务结果。

例如：

- 一个评测平台能减少错误上线，降低事故和回滚成本。
- 一个 prompt 版本系统能减少线上回退时间。

- 一个推理成本 dashboard 能让业务线优化高成本场景。
- 一个 RAG 引用校验机制能提升客户信任。
- 一个 Agent 审计系统能让高风险自动化可上线。

工程师如果能把这些技术能力翻译成业务价值，就更容易获得资源和影响力。

工程不是成本中心的同义词。

好的工程能让业务更快、更稳、更可控地使用 AI。

74.14 面试表达

如果面试官问：“你如何评估一个大模型项目是否值得投入？”

可以这样回答：

我会先明确项目阶段。如果是 demo，重点是验证技术可行性；如果是试点，要看真实用户是否愿意使用、失败类型是否可控、成本是

评估 ROI 时，我不会只看模型效果，也不会只看 token 用量。收益侧要看业务价值，比如节省人工时间、提高任务完成率、降低错误

技术方案上，我会按场景分桶评估。高价值复杂任务可以用强模型，低风险高频任务可以用小模型、规则或缓存。RAG、微调、长上下

最终我会看单位价值成本，比如单次成功任务成本、单次有效问答成本、每减少一次人工接管的成本，而不是只看总 token 或总调用。

74.15 常见误区

第一个误区是只看 token 用量。

token 用量是消耗指标，不是价值指标。

第二个误区是只看模型单价。

完整成本还包括工程、评测、运维、安全和失败重试。

第三个误区是默认强模型一定更划算。

强模型在高价值任务上可能值得，在低价值高频任务上可能浪费。

第四个误区是降本只靠换小模型。

如果质量下降导致重试和人工接管增加，总成本可能更高。

第五个误区是只看平均 ROI。

不同场景价值不同，必须分桶分析。

第六个误区是忽略维护成本。

复杂 Agent、私有推理平台和自研框架上线后都需要长期维护。

第七个误区是没有成本归因。

没有归因，团队无法知道谁在花钱、为什么花钱、是否值得。

74.16 小结

大模型工程不是免费能力。

每一次 token、每一次模型调用、每一次工具调用、每一次 GPU 推理，最终都会进入成本结构。

成熟的大模型工程师不能只看技术效果。

他要能把质量、延迟、成本、安全、维护复杂度和业务收益放在一起判断。

token 用量不是价值指标。

真正重要的是单位价值成本和业务结果。

强模型、小模型、RAG、微调、长上下文、Agent、自建服务和外部 API 都有不同成本结构，没有一种方案永远正确。

好的工程判断，是把最贵的能力用在最值得的地方，把简单任务用简单方案解决，把复杂度留给真正有回报的场景。

最终，大模型工程师的商业判断不是让技术妥协，而是让技术投入产生可持续价值。

第 75 章：长期成长与职业护城河

上一章讨论了成本、ROI 与商业判断。

到这里，大模型工程师面试补充章节已经从项目包装、面试表达、入职成长、技术判断、跨团队沟通、路线图、指标复盘、风险安全和商业判断，逐步走到了职业长期发展。

很多人学习大模型，是为了找到工作。

这是一个合理目标。

但真正进入行业之后，你会发现找工作只是开始。

更长期的问题是：如何在快速变化的大模型时代持续成长，如何不被工具替代，如何形成自己的职业护城河。

大模型领域变化很快。

今天流行的模型、框架、Agent 形态、推理优化和工具链，几年后可能都会变化。

如果你的竞争力只建立在某个具体框架、某个 prompt 技巧或某个热门概念上，护城河会很浅。

真正长期有效的能力，往往更底层、更可迁移。

这一章讨论：大模型工程师如何建立长期职业护城河，如何从短期技能学习走向长期能力建设。

75.1 职业护城河不是会更多工具

很多人以为护城河来自掌握更多工具。

会 LangChain，会 vLLM，会 SGLang，会 RAG，会 Agent，会 MCP，会微调，会评测。

这些当然有价值。

但工具本身不是护城河。

工具变化太快。

今天的最佳实践，明天可能被新模型、新 API、新框架重写。

真正的护城河通常来自：

- 能定义问题。
- 能判断技术取舍。
- 能设计可靠系统。
- 能从失败中学习。
- 能跨团队推动落地。
- 能把技术转化为业务价值。
- 能持续学习新工具并识别其边界。
- 能让团队因为你而变强。

工具能力是入场券。

系统能力才是长期竞争力。

75.2 大模型时代更需要基本功

AI 工具让写代码、查资料、生成样例、写文档都变快了。

但这并不意味着基本功不重要。

相反，基本功更重要。

因为生成变便宜，判断变更贵。

代码更容易生成，但你要判断它是否正确、是否安全、是否可维护。

方案更容易生成，但你要判断它是否适合当前约束。

文档更容易生成，但你要判断它是否表达了真实问题。

大模型工程师的基本功包括：

- 计算机系统理解。
- Python 和工程实践。
- 分布式系统基础。
- 数据工程基础。
- 机器学习和深度学习基础。
- Transformer 和 LLM 原理。
- 评测和实验方法。
- 安全和权限意识。
- 产品和业务理解。
- 写作和沟通。

AI 可以帮助你补速度。

但它不能替代你对问题的判断。

长期来看，基本功越扎实，越能用好 AI 工具。

75.3 从执行任务到拥有问题

职业成长的一个关键变化，是从执行任务到拥有问题。

执行任务时，你关心：

- 需求怎么做？
- 代码怎么写？
- 接口怎么接？
- 测试怎么过？

拥有问题时，你关心：

- 这个问题是否值得解决？
- 真实根因是什么？
- 成功标准是什么？
- 哪些方案可选？
- 代价和风险是什么？
- 上线后怎么验证？
- 后续谁维护？

大模型工程尤其需要拥有问题。

因为很多需求本身就是模糊的。

“做一个智能助手” “提升回答质量” “做一个 Agent” “降低成本” “提升自动化率” 都不是清晰任务。

你需要把它们转化成可分析、可验证、可交付的问题。

能拥有问题的人，才会逐渐拥有方向。

75.4 横向影响力是高级工程师的分水岭

Staff 或 Principal 级别工程师的工作，通常不是纵向管理更多人，而是横向影响更多团队。

这和第 70 章讲的技术影响力一致。

你要能跨越团队边界：

- 让业务理解技术风险。
- 让平台理解业务压力。
- 让安全理解产品场景。
- 让算法理解线上约束。
- 让后端理解模型不确定性。
- 让产品理解评测和反馈闭环。

横向影响力来自长期可信。

它不是靠头衔，也不是靠强势。

它来自你持续做对几件事：

- 写清楚问题。
- 提供可靠判断。
- 尊重别人的约束。
- 主动暴露风险。
- 交付结果。
- 出问题时复盘而不是甩锅。

大模型项目天然跨团队。

如果你只能在自己模块内工作，成长会很快遇到瓶颈。

75.5 写作是放大器

写作是工程师最容易低估的能力之一。

在大模型工程中，写作尤其重要。

你需要写：

- 技术设计文档。
- 实验报告。
- 评测说明。
- Prompt 变更记录。
- 模型切换决策。
- 事故复盘。
- 上线 checklist。
- 失败 case 分析。
- 项目总结。
- 面试项目表达。

写作的价值不只是记录。

写作迫使你澄清思考。

如果一个方案写不清，通常说明你还没想清。

如果一个实验报告说不清样本、指标、baseline 和结论，它就很难支撑决策。

如果一个项目总结说不清问题、方案、指标和收益，它就很难成为职业资产。

写作还能放大影响力。

一个清楚的设计文档，可以让多个团队异步协作。

一个好的复盘，可以让团队避免重复事故。

一个好的技术总结，可以成为你的职业名片。

75.6 构建可迁移能力

大模型领域的具体技术会变。

但有些能力可以跨项目、跨公司、跨技术周期迁移。

第一，问题诊断能力。

能区分模型能力问题、数据问题、检索问题、工具问题、权限问题、产品问题和评测问题。

第二，系统设计能力。

能把模型放进完整系统里，设计数据流、调用链路、监控、回滚、权限和成本治理。

第三，实验能力。

能设计 baseline、对照实验、ablation、评测集和上线灰度。

第四，风险意识。

能提前识别 prompt injection、越权、敏感信息、工具副作用和过度自动化风险。

第五，商业判断。

能判断一个方案是否值得投入，是否能产生可持续价值。

第六，组织协作能力。

能推动跨团队形成共识。

这些能力不会因为某个框架过时而失效。

它们才是长期护城河。

75.7 学会和 AI 工具协作

大模型工程师不能排斥 AI 工具。

但也不能盲目信任 AI 工具。

更成熟的方式是把 AI 当作协作者。

AI 适合帮助你：

- 搜集资料。
- 生成初稿。
- 补测试样例。
- 解释陌生代码。
- 生成脚本。
- 比较方案。
- 做代码 review 辅助。
- 总结日志。
- 起草文档。

但你要负责：

- 定义问题。
- 判断输出是否正确。
- 验证代码和实验。
- 识别安全风险。
- 保护敏感信息。
- 做最终取舍。
- 承担结果责任。

AI-native 工程师不是把所有工作交给 AI。
而是知道哪些认知工作可以委托，哪些判断必须自己保留。
长期来看，AI 会放大强工程师，也会暴露弱工程师。
因为弱工程师更容易被错误输出带偏。

75.8 建立个人技术资产

职业成长需要资产沉淀。
资产不是简历上的项目名。
资产是可以反复复用、展示和迁移的材料。
例如：

- 一个完整 RAG 项目复盘。
- 一个 Agent 安全设计文档。
- 一个推理服务容量规划案例。
- 一个评测集构建方法。
- 一个失败 case 分类体系。
- 一个模型切换实验报告。
- 一个成本治理 dashboard 设计。
- 一个线上事故复盘。
- 一篇技术博客。
- 一个开源 demo 或工具。

个人技术资产应该包含：

- 背景。
- 问题。
- 约束。
- 方案。
- 指标。
- 结果。
- 取舍。
- 失败和改进。

只写“我做了 RAG 系统”不够。
要能讲清楚：你解决了什么问题，为什么这样设计，指标如何变化，遇到什么风险，后续如何演进。
这也是面试中最有价值的材料。

75.9 选择能让你成长的项目

不是所有项目都同样有成长价值。
高成长项目通常有几个特征：

- 问题真实。
- 有用户反馈。
- 有线上约束。
- 需要跨团队协作。
- 有质量、成本、安全和稳定性压力。
- 有机会做复盘和沉淀。
- 能让你接触完整链路。

低成长项目通常是：

- 只做 demo。

- 没有真实用户。
- 没有评测。
- 没有上线。
- 没有复盘。
- 只是套框架。
- 做完即丢。

当然，早期可以从 demo 开始。

但如果长期只做 demo，很难形成生产级判断。

要主动争取接触真实系统：数据、评测、上线、监控、失败、用户反馈、成本和安全。

真实约束会迫使你成长。

75.10 不要把职业押注在单一热点上

大模型领域热点很多。

RAG、Agent、MCP、SFT、RLHF、DPO、推理框架、多模态、AI coding、workflow、LLM observability。

这些方向都值得学习。

但不要把职业完全押注在单一热点上。

热点会变化。

你应该建立“T 型能力”。

横向理解大模型工程全链路：

- 数据。
- 训练。
- 推理。
- RAG。
- Agent。
- 评测。
- 安全。
- 产品化。
- 成本。

纵向选择一两个方向深入。

例如：

- RAG + 评测。
- Agent + 安全。
- 推理 + 成本。
- 数据 + 训练。
- AI Infra + 平台化。

这样既有专业深度，又不容易被单一技术周期淘汰。

75.11 职业节奏：探索、聚焦、放大

长期成长可以分成三个阶段。

第一阶段，探索。

目标是建立知识面。

多做项目，多读代码，多看论文，多理解真实系统。

不要过早给自己贴标签。

第二阶段，聚焦。

找到自己更擅长、更有机会的方向。

例如 RAG 质量、推理系统、Agent 工程、AI Infra、安全治理或评测体系。

在一个方向上做出可证明结果。

第三阶段，放大。

把个人经验变成团队能力。

写规范，做平台，带新人，推动跨团队协作，建立机制。

这个阶段的核心不是你一个人更快，而是团队因为你整体更强。

很多人卡在第二阶段。

能做事，但不能放大影响。

从高级到资深，关键就是从个人产出走向组织影响。

75.12 面试表达

如果面试官问：“你如何规划自己在大模型方向的长期成长？”

可以这样回答：

我不会把长期成长押在某个单一框架或热点上，因为大模型技术变化很快。我的思路是先建立扎实的底层能力，包括机器学习和 Tran

在此基础上，我会形成 T 型能力。横向理解大模型应用全链路，比如 RAG、Agent、推理、评测、数据、安全和成本；纵向选择一两个

我也会刻意积累个人技术资产。比如完整项目复盘、实验报告、失败 case 分类、评测集构建、线上事故复盘、成本治理方案和技术设

长期来看，我希望从执行任务成长为拥有问题，再进一步拥有方向。也就是不仅能写代码和接需求，还能定义问题、做技术取舍、推

75.13 常见误区

第一个误区是追逐所有热点。

什么都学一点，但没有任何方向形成深度。

第二个误区是只学工具，不学原理和系统。

工具会变，底层能力更持久。

第三个误区是只做任务，不拥有问题。

长期停留在执行层，很难成长为方向负责人。

第四个误区是忽视写作和表达。

不会表达，影响力很难放大。

第五个误区是只做 demo，不接触生产约束。

没有上线、监控、失败和复盘，很难形成真实判断。

第六个误区是盲目信任 AI 工具。

AI 可以加速，但最终判断和责任仍然在工程师身上。

第七个误区是没有资产沉淀。

项目做了很多，但没有总结、指标和复盘，最后很难证明成长。

75.14 小结

大模型工程师的长期护城河，不是掌握某个固定工具，而是拥有可迁移的系统能力。

工具、框架和模型会变化。

但问题诊断、系统设计、实验方法、风险意识、商业判断、跨团队协作和写作表达，会长期有效。

AI 工具会改变工程师的工作方式。

但越是 AI 能生成更多内容，越需要人类工程师做判断、验证、取舍和负责。

长期成长的路径，是从执行任务到拥有问题，从拥有问题到拥有方向，从个人产出到组织影响。

如果你能持续沉淀技术资产，选择真实约束下的项目，建立 T 型能力，并把个人经验转化为团队机制，你就不会只是在追逐大模型浪潮，而是在浪潮中建立自己的位置。

第 76 章：30-60-90 天成长计划与行动清单

上一章讨论了长期成长与职业护城河。

长期成长很重要，但真正落地时，很多人需要的是更具体的行动计划。

例如：

- 刚入职一家大模型团队，前三个月应该做什么？
- 从传统后端或算法岗位转到大模型工程，如何快速进入状态？
- 晋升为技术负责人后，如何建立影响力？
- 接手一个混乱的 RAG、Agent 或推理项目，如何不被细节淹没？
- 面试通过后，如何在试用期内证明自己？

这类问题适合用 30-60-90 天计划来回答。

30-60-90 天计划不是死板日程表。

它是一种帮助你建立节奏、对齐预期、识别风险、持续交付的工具。

这一章给出大模型工程师的 30-60-90 天成长计划。

你可以用于新入职、转岗、接手新项目、晋升技术负责人，或准备面试中的“入职后你会怎么做”问题。

76.1 30-60-90 天计划的目的

30-60-90 天计划有三个目的。

第一，减少混乱。

新环境中信息很多。

模型、数据、业务、代码、平台、评测、安全、组织关系都需要了解。

如果没有计划，很容易每天都很忙，但没有方向。

第二，对齐预期。

你需要和 manager、mentor、项目 owner、产品、安全、平台等相关方确认：前三个月最重要的目标是什么。

第三，建立可信度。

新加入团队时，不需要一开始就做惊天动地的大项目。

更重要的是快速学习、可靠交付、清楚沟通、暴露风险、形成闭环。

30-60-90 天计划的核心不是“我要证明我很强”。

而是：

我会先理解系统，再找到可控切入点，逐步交付结果，最后形成可持续影响。

76.2 前 30 天：理解系统和建立关系

前 30 天不要急着重构。

也不要急着证明原系统很差。

你的目标是理解上下文。

大模型项目的上下文包括：

- 业务目标。
- 用户场景。
- 当前系统架构。
- 模型和数据链路。
- 评测方法。
- 线上指标。
- 已知问题。
- 历史决策。
- 团队分工。
- 安全和合规约束。

这一阶段最重要的是听、读、问、记录。

76.3 前 30 天的具体行动

第一，读文档。

优先读：

- 项目 PRD。
- 技术设计文档。
- 架构图。
- 模型选型记录。
- 评测报告。
- 事故复盘。
- prompt 变更记录。
- 成本和容量文档。
- 安全评审记录。

第二，跑通系统。

至少要跑通：

- 本地开发环境。
- 一个核心 API。
- 一个核心离线评测脚本。
- 一个线上日志或 trace 查询。
- 一个完整请求链路。

第三，画出请求生命周期。

例如 RAG 系统可以画：

```
user query
-> auth
-> query rewrite
-> retrieval
-> rerank
-> context assembly
-> LLM generation
-> citation verification
-> response
```

-> logging / feedback

Agent 系统可以画：

user goal

- > planner
- > tool selection
- > permission check
- > tool execution
- > observation
- > reflection / next step
- > final response
- > audit

推理服务可以画：

request

- > API gateway
- > tokenizer
- > scheduler
- > prefill
- > KV cache
- > decode
- > sampler
- > streaming
- > metrics

第四，建立关系。

至少约 1:1:

- manager。
- mentor。
- 产品负责人。
- 项目技术 owner。
- 平台或 SRE。
- 数据负责人。
- 安全或合规负责人。
- 主要业务方。

每次 1:1 可以问：

- 当前项目最重要目标是什么？
- 最大风险是什么？
- 哪些问题反复出现？
- 哪些历史决策需要了解？
- 如果我前三个月只能做好一件事，你希望是什么？

第五，整理学习笔记。

建议形成一份 Onboarding Notes：

- 系统架构。
- 核心术语。
- 服务列表。
- 数据表或索引。
- 评测集。
- 常见故障。
- 常用命令。
- 待确认问题。

这份笔记未来可以变成团队 onboarding 文档。

76.4 前 30 天的产出

前 30 天不一定要交付大功能。

但应该有可见产出。

建议至少完成：

- 一张系统请求生命周期图。
- 一份 onboarding notes。
- 一次小 bug 修复或小功能改动。
- 一次评测脚本或日志查询跑通记录。
- 一份当前问题清单。
- 一份 60 天计划草案。

这些产出证明你不是只在“熟悉业务”。

你已经开始把混乱信息整理成可复用材料。

76.5 第 31-60 天：找到切入点并交付闭环

第 31-60 天，目标从理解转向交付。

但仍然不要急着做大重构。

你应该选择一个可控切入点。

好的切入点通常满足：

- 问题真实。
- 范围可控。
- 有明确 owner。
- 可以量化效果。
- 风险可回滚。
- 能帮助你理解更大系统。

例如：

- 补一个关键日志字段。
- 修复一类 RAG 引用错误。
- 为 Agent 工具调用增加参数校验。
- 给 prompt 增加版本记录。
- 建立一个小型失败 case 分类表。
- 优化一个高频请求的 token 成本。
- 给推理服务增加 TTFT/TPOT 分桶 dashboard。

这些事情看起来不大。

但它们能建立可信度。

可信度来自闭环，而不是规模。

76.6 第 31-60 天的具体行动

第一，选择一个小闭环项目。

写清楚：

- 背景。
- 问题。
- 目标。
- 非目标。
- 方案。

- 指标。
- 风险。
- 回滚方式。

第二，建立 baseline。

任何改动前先知道当前状态。

例如：

- 当前 citation accuracy 是多少？
- 当前 P95 TTFT 是多少？
- 当前工具参数错误率是多少？
- 当前单次请求平均 token 成本是多少？
- 当前失败 case 中哪一类最多？

第三，做小实验。

不要一开始全量上线。

可以先做离线评测、小流量灰度、内部用户试点或只覆盖某类场景。

第四，交付并复盘。

交付后写一份短总结：

- 做了什么。
- 指标如何变化。
- 哪些预期成立。
- 哪些预期不成立。
- 后续建议是什么。

第五，继续建立关系。

第 31-60 天要从“认识人”进入“协作做事”。

主动让相关方看到你的思考和交付。

76.7 第 31-60 天的产出

建议完成：

- 一个小闭环项目。
- 一份 baseline 和实验结果。
- 一份失败 case 或指标分析。
- 一次设计评审或方案文档。
- 一次上线后复盘。
- 一份下一阶段优先级建议。

如果能做到这些，你在团队中的定位会从“新人”变成“可靠协作者”。

76.8 第 61-90 天：扩大影响并建立机制

第 61-90 天，目标是从单点交付走向机制化影响。

这时你已经理解了系统，也完成了一个小闭环。

下一步要问：

- 哪些问题不是一次性修复，而需要机制？
- 哪些经验可以沉淀成文档、工具或流程？
- 哪些指标应该持续监控？
- 哪些技术债需要进入路线图？
- 哪些协作关系需要固定下来？

例如你修复了一类引用错误。

第 90 天前，你不只是修 bug。

你应该推动：

- 引用错误进入评测集。
- citation accuracy 进入 dashboard。
- prompt 或 parser 变更进入回归测试。
- 失败 case review 建立固定节奏。

这就是从交付功能走向建立机制。

76.9 第 61-90 天的具体行动

第一，总结前三个月观察。

写一份 First 90 Days Review。

包括：

- 我理解的业务目标。
- 当前系统架构。
- 核心指标。
- 最大风险。
- 已完成工作。
- 发现的问题。
- 推荐的下一步路线图。

第二，提出一个中期改进方向。

不要提出十个方向。

选择一个最重要的方向。

例如：

- 建立 RAG 质量闭环。
- 建立 Agent 工具调用安全门禁。
- 建立推理成本归因。
- 建立模型切换评测门禁。
- 建立线上失败 case 回流机制。

第三，把个人经验转成团队资产。

可以沉淀：

- onboarding 文档。
- debug checklist。
- 评测模板。
- 上线 checklist。
- prompt 变更规范。
- 工具接入规范。
- 成本分析模板。

第四，和 manager 对齐下一阶段目标。

问题包括：

- 我前三个月哪些地方做得有效？
- 哪些地方需要调整？
- 接下来我应该承担什么范围？
- 我的成长目标是什么？
- 哪些项目最能体现我的价值？

第五，明确自己的角色定位。

你可能定位为：

- RAG 质量 owner。
- Agent 工具链 owner。
- 推理服务性能 owner。
- 评测平台 owner。
- 数据质量 owner。
- 安全治理推动者。

角色定位不一定是正式头衔。

但它能帮助团队知道什么问题可以找你。

76.10 第 61-90 天的产出

建议完成：

- 一份 90 天复盘文档。
- 一份下一阶段路线图建议。
- 一个可复用团队资产。
- 一个机制化改进提案。
- 一次 manager 对齐会。
- 一个明确的后续 owner 方向。

这会让你从“可靠协作者”进一步变成“可以负责一个问题的人”。

76.11 不同角色的 30-60-90 重点

如果你是初级大模型工程师，重点是：

- 跑通环境。
- 理解链路。
- 修小问题。
- 写清楚笔记。
- 建立工程基本可信度。

如果你是中级工程师，重点是：

- 独立负责小闭环。
- 建立 baseline。
- 做实验。
- 上线和复盘。
- 理解系统 trade-off。

如果你是高级工程师，重点是：

- 识别关键瓶颈。
- 推动跨团队协作。
- 设计中期方案。
- 建立机制。
- 帮团队减少混乱。

如果你是技术负责人，重点是：

- 对齐业务目标。
- 诊断系统和组织瓶颈。
- 明确优先级。
- 建立指标和节奏。
- 形成路线图。

同样是 90 天，不同级别的目标不同。

不要用高级工程师的标准要求新人。

也不要用人标准要求自己停留在熟悉环境。

76.12 面试表达

如果面试官问：“如果你加入我们团队，前三个月会怎么做？”

可以这样回答：

我会按 30-60-90 天拆。前 30 天重点是理解系统和建立关系，不急着做大改动。我会阅读 PRD、设计文档、评测报告、事故复盘和安

第 31-60 天，我会选择一个可控的小闭环项目，例如补关键日志字段、修复一类 RAG 引用错误、增加工具参数校验、优化 token 成本

第 61-90 天，我会把前面的学习和交付沉淀成机制。比如写 90 天复盘，提出下一阶段路线图建议，把 debug checklist、评测模板、

我的原则是先理解上下文，再做可控交付，最后把个人经验变成团队机制。

76.13 常见误区

第一个误区是前 30 天急着重构。

没有理解历史约束就重构，容易破坏已有系统。

第二个误区是只学习不产出。

熟悉业务也要有可见产出，比如图、笔记、小修复或问题清单。

第三个误区是接太大的项目。

新人前 60 天更适合可控闭环，而不是高风险大改造。

第四个误区是不建立 baseline。

没有 baseline，就无法证明改动有效。

第五个误区是只和直属团队沟通。

大模型项目通常跨产品、平台、数据、安全和业务。

第六个误区是交付后不复盘。

没有复盘，经验不会沉淀。

第七个误区是没有和 manager 对齐预期。

你以为重要的事情，未必是团队当前最需要的。

76.14 小结

30-60-90 天计划，是把长期成长落到短期行动的工具。

前 30 天，重点是理解系统、建立关系、跑通链路、整理上下文。

第 31-60 天，重点是选择一个可控切入点，建立 baseline，交付小闭环。

第 61-90 天，重点是扩大影响，把个人经验沉淀成团队机制，并明确后续 owner 方向。

大模型工程师的前三个月，不需要靠激进重构证明自己。

更可靠的路径是：理解上下文，尊重已有系统，快速形成可见产出，持续对齐预期，再逐步扩大影响。

真正的成长不是前三个月做了多少事，而是前三个月之后，团队是否更清楚问题，系统是否更可观察，风险是否更透明，下一步是否更有方向。

第 77 章：面试复盘、Offer 选择与谈薪

上一章讨论了 30-60-90 天成长计划。

如果你已经拿到面试机会，甚至已经拿到 offer，接下来还有一组非常现实的问题：

- 面试后如何复盘？
- 怎么判断自己哪里答得好、哪里需要补？
- 反向提问应该问什么？
- 拿到多个 offer 怎么选？
- 如何判断团队是否真的适合大模型工程师成长？
- 谈薪时应该注意什么？
- 如何降低入职后的风险？

很多候选人把全部精力放在“通过面试”。

但对职业发展来说，选对团队和角色同样重要。

一个看起来光鲜的大模型岗位，可能只是包装出来的 AI 项目。

一个薪资更高的 offer，可能对应更差的成长空间、更混乱的组织、更不可控的风险。

一个名气较小的团队，反而可能给你完整负责 RAG、Agent、评测、推理或 AI Infra 的机会。

这一章讨论：面试后如何复盘，如何做 offer 选择，如何谈薪，如何在加入前识别风险。

77.1 面试结束不是结束

每一场面试结束后，都应该立即复盘。

原因很简单：面试记忆衰减很快。

如果当天不记录，第二天你就会忘记很多细节。

面试复盘至少记录：

- 公司和岗位。
- 面试轮次。
- 面试官角色。
- 被问到的问题。
- 自己回答得好的地方。
- 回答卡住的地方。
- 面试官追问方向。
- 暴露出的知识短板。
- 反向提问得到的信息。
- 对岗位和团队的初步判断。

建议建立一个表格。

每场面试后用 15 分钟补全。

不要等整个流程结束再回忆。

复盘的价值不只是准备下一场。

它还能帮你判断哪些公司真的理解大模型工程，哪些只是跟风招聘。

77.2 如何分析自己的面试表现

复盘时不要只写“答得不好”。

要具体拆分。

大模型工程面试可以按几个维度分析。

第一，原理表达。

例如 Transformer、attention、KV cache、RAG、Agent、训练、评测、推理优化是否能讲清楚。

第二，工程经验。

是否能讲清楚项目架构、数据流、线上问题、监控、回滚、成本和安全。

第三，系统设计。

是否能从需求、约束、模块、指标、故障和演进讲方案。

第四，实验思维。

是否能讲 baseline、ablation、评测集、指标、误差分析和上线验证。

第五，沟通表达。

是否能结构化回答，是否能承认不确定性，是否能和面试官形成讨论。

第六，商业和风险判断。

是否能谈质量、成本、延迟、安全、ROI 和责任边界。

如果某一轮挂了，不要简单归因为“面试官不喜欢我”。

要问：

- 是知识不会？
- 是项目讲不清？
- 是缺少指标？
- 是没有真实上线经验？
- 是系统设计太散？
- 是表达没有结构？
- 是反问暴露了兴趣不匹配？

找到原因，下一轮才会变强。

77.3 面试后如何补短板

面试暴露的短板要分类补。

如果是概念短板，例如 KV cache、LoRA、DPO、speculative decoding 不清楚，要回到原理材料和代码样例。

如果是项目短板，例如讲不清指标和结果，要补项目复盘文档。

如果是系统设计短板，要练习固定模板：需求、约束、模块、数据流、指标、风险、演进。

如果是表达短板，要把高频问题写成 2 分钟、5 分钟、10 分钟三个版本。

如果是经验短板，要承认边界，同时用实验、demo、开源项目 and 设计文档补证据。

补短板时不要无限学习。

面试准备最容易陷入“我再学一点就好了”。

更有效的是面向问题补：

这次被问到 RAG 评测，我答得散。

下一步我要准备一个固定回答：评测集构建、指标、bad case 分类、线上反馈回流和灰度验证。

这样补才有效。

77.4 反向提问是双向面试

很多候选人把反向提问当成礼貌环节。

这是浪费机会。

反向提问能帮你判断团队质量。

尤其是大模型岗位，外部描述和真实工作可能差异很大。

你要问出：

- 这个团队是真的做大模型核心工程，还是只做 API 包装？
- 是否有真实用户和线上流量？
- 是否有评测和反馈闭环？
- 是否有数据、平台、安全和产品协作？
- 是否有资源做长期建设？
- 是否有技术 owner 和清晰路线图？
- 是否能给你成长空间？

反向提问不是质问。

要用专业、具体的问题判断信息。

77.5 大模型岗位的反向提问清单

关于业务和场景，可以问：

- 这个大模型系统服务的核心用户是谁？
- 当前最重要的业务指标是什么？
- 项目处于 demo、试点还是生产阶段？
- 目前最大的用户痛点是什么？
- 成功标准如何定义？

关于技术栈，可以问：

- 当前主要使用外部 API、自建模型，还是混合模式？
- RAG、Agent、微调、长上下文分别在哪些场景使用？
- 是否有统一评测平台或实验流程？
- 推理服务是自建还是使用现成框架？
- 是否有模型、prompt、数据和工具版本管理？

关于线上质量，可以问：

- 现在如何评估回答质量？
- 失败 case 如何收集和回流？
- 是否有灰度、回滚和线上监控？
- 最常见的线上失败类型是什么？
- 有哪些事故或复盘经验？

关于安全和成本，可以问：

- 权限过滤在哪里执行？
- Agent 工具调用如何做审计和确认？
- 是否有 prompt injection 或 red team 测试？
- 成本如何归因到业务线或租户？
- 当前最大的成本瓶颈是什么？

关于团队和成长，可以问：

- 这个岗位前三个月希望我完成什么？
- 团队内谁负责技术路线图？
- 我会负责完整链路，还是只负责某个模块？
- 是否有机会参与设计评审和线上复盘？

- 这个岗位未来 6-12 个月的成长空间是什么？

这些问题会让你看清岗位真实情况。

77.6 Offer 选择不要只看薪资

薪资重要。

但不是唯一因素。

尤其对大模型工程师，offer 选择至少看八个维度。

第一，岗位真实性。

是否真的做大模型核心工程，还是只是调用 API 做包装。

第二，项目阶段。

demo、试点、生产、平台化，不同阶段成长完全不同。

第三，团队能力。

是否有强技术 owner，是否有数据、平台、安全、产品协作。

第四，个人职责。

你是完整负责一个问题，还是只做边缘模块。

第五，资源和授权。

是否有模型、GPU、数据、评测、业务支持。

第六，成长空间。

是否能接触真实用户、线上系统、评测、成本和安全。

第七，组织稳定性。

公司是否持续投入 AI，团队是否稳定，项目是否有明确价值。

第八，薪资和长期收益。

包括 base、bonus、股票、期权、涨薪空间和学习价值。

一个好的 offer，不一定是钱最多的。

而是能让你在未来 1-2 年形成更强职业资产的。

77.7 如何识别高风险团队

面试中要注意危险信号。

例如：

- 团队说不清核心业务指标。
- 所有项目都叫 Agent，但没有真实用户。
- 没有评测集，只靠主观感觉判断效果。
- 没有线上监控和回滚。
- 安全问题只靠 prompt 解决。
- 成本完全没人管。
- 大模型项目只是领导口号，没有明确 owner。
- 团队频繁变方向。
- 面试官无法解释岗位前三个月要做什么。
- 所有问题都要求候选人“来了再说”。

这些不一定意味着不能去。

但意味着你要降低预期，或在谈 offer 前问清楚。

有些高风险团队也有高成长机会。

前提是你有足够授权和资源去改变现状。

如果团队混乱，但你只是执行者，风险很高。

如果团队混乱，但明确希望你建立评测、平台或技术方向，并给你支持，可能是机会。

77.8 谈薪的基本原则

谈薪不是对抗。

谈薪是把你的市场价值、岗位价值和公司预算对齐。

几个原则：

第一，尽量拿到完整 offer 结构。

包括：

- base。
- bonus。
- 股票或期权。
- 签字费。
- 年终或绩效规则。
- 涨薪周期。
- 试用期政策。
- 社保、公积金、福利。

第二，不要只比较月薪。

不同公司股票、bonus、现金流、税务和稳定性差异很大。

第三，表达要基于事实。

例如：

我对团队和方向很感兴趣。结合我的大模型工程经验、目前其他机会和市场水平，我希望 total compensation 能调整到 X 区间。如果

第四，不要编造 offer。

这会损害信任。

第五，提前明确底线。

包括最低可接受薪资、通勤、远程、工作强度、成长空间和风险接受度。

谈薪前如果自己都不知道底线，很容易被动。

77.9 职级比 title 更重要

大模型岗位 title 很乱。

同样叫“大模型工程师”，可能差异很大。

你要看实际职级和职责。

要问：

- 这个岗位对应什么级别？
- 职级期望是什么？
- 晋升标准是什么？
- 是否需要带方向或带人？

- 绩效如何评估？
- 入职后第一个周期看什么结果？

title 听起来高级，但实际只是执行 API 接入，成长有限。

title 普通，但能负责 RAG 质量闭环、Agent 工具平台、推理成本治理，反而更有价值。

职业成长看的是你解决问题的复杂度和影响范围，不是 title 字面。

77.10 入职前要确认的事项

接受 offer 前，尽量确认：

- 汇报对象是谁。
- 所属团队是什么。
- 第一个项目是什么。
- 前 90 天期望是什么。
- 是否有 mentor 或 buddy。
- 是否有明确技术 owner。
- 是否有试用期目标。
- 是否有岗位变动风险。
- 是否有竞业或保密限制。
- 是否有工作地点、远程、出差和加班要求。

这些问题越早确认，入职后越少意外。

如果对方无法回答前 90 天目标，说明岗位可能还不清晰。

如果岗位不清晰，你要判断自己是否愿意承担这种不确定性。

77.11 面试复盘如何变成长期资产

面试复盘不只是为了当前求职。

它也能变成长期学习资产。

你可以把所有面试问题归类：

- 原理题。
- 项目题。
- 系统设计题。
- Debug 题。
- 实验评测题。
- 安全风险题。
- 成本 ROI 题。
- 行为面试题。

每类问题写出标准回答。

经过几轮面试，你会得到一份高度真实的市场题库。

这比网上零散题库更有价值。

因为它来自真实公司、真实岗位、真实追问。

如果你把面试复盘持续沉淀，下一次求职准备会轻松很多。

77.12 面试表达

如果面试官问：“你选择 offer 时最看重什么？”

可以这样回答：

我会综合看岗位真实性、项目阶段、团队能力、个人职责、资源支持、成长空间和薪酬。对大模型岗位来说，我不会只看 title 或是如果有一个岗位能让我接触真实业务场景，参与从数据、模型、RAG 或 Agent 到评测、上线、监控和复盘的完整链路，即使短期 title 薪酬当然重要，但我更关注 total compensation 和未来 1-2 年能否形成职业资产。理想的机会是技术挑战真实、团队目标清晰、我能

77.13 常见误区

第一个误区是面试后不复盘。

错过了最真实的学习材料。

第二个误区是反向提问只问福利。

福利可以问，但更要问项目、团队、评测、上线和成长空间。

第三个误区是只看薪资最高的 offer。

高薪但低成长、低授权、高风险，长期未必划算。

第四个误区是相信所有 AI 岗位包装。

很多岗位只是普通业务加一层大模型 API。

第五个误区是谈薪时没有完整结构。

只看 base，不看 bonus、股票、签字费、试用期和涨薪规则。

第六个误区是入职前不确认前 90 天目标。

入职后才发现岗位和预期不一致。

第七个误区是忽视组织风险。

方向不稳、owner 不清、资源不足，会严重影响成长。

77.14 小结

通过面试不是求职链路的终点。

面试后复盘、反向提问、offer 选择、谈薪和入职前确认，同样决定你的职业收益。

大模型岗位尤其需要谨慎判断。

因为外部包装往往很相似，但真实工作可能差异巨大。

一个成熟候选人，不只是会回答问题，也会判断团队是否值得加入。

你要通过反向提问看清业务目标、项目阶段、技术链路、评测闭环、安全成本和成长空间。

选 offer 时，不只看薪资，也看未来 1-2 年能否形成可迁移的职业资产。

最终，求职不是把自己卖给一个 title，而是选择一个能让你持续成长、持续交付、持续扩大影响的环境。

第 78 章：求职材料包、作品集与公开证据

上一章讨论了面试复盘、offer 选择与谈薪。

这一章继续往前补一环：你如何让别人相信你真的具备大模型工程能力。

很多候选人准备面试时，只准备简历。

但在大模型岗位竞争中，只有简历往往不够。

因为简历很短，无法展示完整项目细节、工程判断、实验过程、失败复盘和真实取舍。

更好的方式是准备一套求职材料包。

它包括：

- 简历。
- 项目一页纸。
- 项目复盘文档。
- GitHub 仓库或代码样例。
- 技术博客。
- 面试题库和标准回答。
- 反向提问清单。
- 30-60-90 天入职计划。

这些材料共同构成你的职业证据链。

你不是只告诉面试官 “我做过 RAG” 或 “我懂 Agent”。

你要让对方看到：你能定义问题、设计系统、做实验、上线验证、处理风险、总结复盘。

这一章讨论如何构建这套材料。

78.1 简历是入口，不是全部

简历的任务不是讲完所有东西。

简历的任务是让对方愿意继续了解你。

大模型工程师简历最常见的问题是堆技术名词。

例如：

熟悉 LangChain、RAG、Agent、Prompt Engineering、LoRA、vLLM、SGLang、MCP、向量数据库。

这类描述信息密度很低。

因为它没有告诉别人你解决了什么问题。

更好的写法是：

负责企业知识库 RAG 问答系统，从文档解析、chunk、hybrid retrieval、rerank、引用校验到线上反馈闭环；构建 500 条业务评测集

简历要突出四件事：

- 问题。
- 方案。
- 指标。
- 影响。

技术栈可以写，但不能替代结果。

78.2 大模型简历的项目 bullet 模板

一个好的项目 bullet 可以按这个模板写：

在什么场景下，为了解决什么问题，采用什么技术方案，做了什么关键设计，最终带来什么指标或业务结果。

例如 RAG 项目：

面向企业内部知识问答场景，设计 RAG 质量优化闭环，覆盖文档解析、chunk 策略、hybrid retrieval、rerank、引用校验和 bad cas

Agent 项目：

负责工单自动化 Agent 的工具调用链路，设计 tool schema、权限校验、参数验证、dry-run 和人工确认机制；将低风险工单自动处理

推理服务项目：

参与 LLM 推理服务性能优化，分析 TTFT、TPOT、KV cache 和 batching 瓶颈，调整 prefix cache、max batched tokens 和路由策略；

评测项目：

搭建模型和 prompt 回归评测流程，沉淀业务样本、LLM-as-a-judge rubric、人工复核和上线门禁；将 prompt 变更导致的线上回退问题控制在 1 次。

注意：指标必须真实。

没有线上指标时，可以写离线指标、样本规模、实验设置和 demo 约束。

不要编造数据。

78.3 项目一页纸

项目一页纸是简历和长文档之间的材料。

它适合面试前自我准备，也适合在面试后发给面试官或内推人。

结构可以是：

项目名称：

场景背景：

核心问题：

我的角色：

系统架构：

关键技术：

指标结果：

主要取舍：

失败和改进：

面试可展开点：

例如：

项目名称：企业知识库 RAG 问答系统

场景背景：客服和售前需要从内部文档中快速回答客户问题。

核心问题：回答经常缺少可靠引用，且文档更新后效果不稳定。

我的角色：负责检索质量、引用校验和评测闭环。

系统架构：文档解析 -> chunk -> embedding/hybrid retrieval -> rerank -> context assembly -> LLM -> citation verification -> feedback。

关键技术：hybrid retrieval、rerank、citation eval、bad case 分类、prompt 版本管理。

指标结果：citation accuracy 从 71% 到 86%，P95 latency 控制在 2.3s。

主要取舍：rerank 提升质量但增加延迟，因此只对低置信 query 启用。

失败和改进：表格文档解析仍是主要失败来源，后续计划引入结构化解析。

面试可展开点：RAG 评测、失败 case 分类、引用校验、延迟成本取舍。

一页纸的价值是帮助你在面试中稳定表达。

它也能帮助你发现项目故事是否完整。

如果一页纸写不出来，说明项目还没有被你真正理解和沉淀。

78.4 项目复盘文档

项目复盘比一页纸更详细。

适合沉淀成个人资产。

结构可以是：

- 背景。
- 问题定义。
- 需求和非需求。
- 系统架构。
- 技术方案。
- 数据和评测。

- 上线过程。
- 指标结果。
- 成本和风险。
- 失败 case。
- 复盘和下一步。

项目复盘要避免只写成功。

面试官往往更关心：

- 你遇到过什么失败？
- 你如何定位问题？
- 你如何选择方案？
- 你如何判断上线是否成功？
- 你如何处理安全、成本和延迟？

真实失败比完美故事更可信。

例如：

最初我们以为 RAG 错误主要来自 embedding 召回不足，但分析 200 个 bad case 后发现，42% 的错误是正确 chunk 已经进入 top-20，但没有进入最终上下文。因此我们没有马上换 embedding 模型，而是先做 rerank 和 context assembly 优化。

这类复盘能体现工程判断。

78.5 GitHub 仓库如何展示

如果你有公开项目，GitHub README 很重要。

很多候选人的仓库只有代码，没有说明。

这会让面试官很难判断价值。

一个好的 README 应该包括：

- 项目解决什么问题。
- 架构图或流程图。
- 快速开始。
- 核心模块说明。
- 示例输入输出。
- 评测方法。
- 已知限制。
- 后续计划。

大模型项目尤其要写清：

- 使用什么模型。
- 数据来源是什么。
- 是否支持本地或 API。
- prompt 在哪里。
- 如何运行评测。
- 如何复现实验。
- 安全和隐私注意事项。

不要只展示一个 demo UI。

要展示工程链路。

例如一个 RAG 仓库可以包含：

```
README.md
docs/architecture.md
docs/evaluation.md
docs/failure_cases.md
```

```
configs/  
src/ingestion/  
src/retrieval/  
src/rerank/  
src/generation/  
src/evaluation/  
examples/
```

项目不一定要很大。

但要可读、可运行、可解释。

78.6 技术博客写什么

技术博客不是为了刷存在感。

它是把你的经验公开沉淀成证据。

适合写的主题包括：

- 一次 RAG bad case 分析。
- 一个 Agent 工具调用安全设计。
- 一次模型切换评测。
- 一次推理延迟优化复盘。
- 一个 prompt 版本管理方案。
- 一个 LLM-as-a-judge 评测模板。
- 一个成本归因 dashboard 设计。
- 一个 30-60-90 天成长复盘。

博客不需要追求“大而全”。

一篇具体、真实、有细节的小文，比泛泛而谈的“万字综述”更有价值。

好的技术博客应该回答：

- 问题是什么？
- 为什么重要？
- 你试过哪些方案？
- 哪些失败了？
- 最后选择了什么？
- 指标如何变化？
- 有什么限制？

这和面试表达是同一套能力。

78.7 面试材料包如何组织

建议建立一个私有目录，结构如下：

```
career-package/  
  resume/  
    resume-llm-engineer.pdf  
    resume-ai-infra.pdf  
  projects/  
    rag-one-pager.md  
    agent-one-pager.md  
    inference-one-pager.md  
    evaluation-retrospective.md  
  interview/  
    project-stories.md  
    system-design-answers.md
```

```
behavior-answers.md
reverse-questions.md
company-notes/
  company-a.md
  company-b.md
offer/
  offer-comparison.md
```

每次面试后更新。

每次项目完成后更新。

每次复盘后更新。

这套材料会越来越值钱。

它能让你在下一求职时不再从零开始。

78.8 不同岗位的材料侧重点

如果你投大模型应用工程师，重点展示：

- RAG。
- Agent。
- 工具调用。
- 评测。
- 上线和监控。
- 用户反馈。

如果你投 AI Infra 或推理工程，重点展示：

- 推理服务。
- batching。
- KV cache。
- 性能分析。
- 容量规划。
- 成本治理。

如果你投算法工程师，重点展示：

- 模型原理。
- 训练和微调。
- 数据构造。
- 实验设计。
- 指标分析。

如果你投资深或负责人岗位，重点展示：

- 技术路线图。
- 跨团队协作。
- 风险治理。
- 机制建设。
- 业务影响。

同一个人可以准备不同版本材料。

不要用同一份简历和同一套项目讲法投所有岗位。

78.9 公开表达要注意边界

公开项目和博客要注意保密。

不要泄露：

- 公司内部数据。
- 客户信息。
- 业务指标的敏感细节。
- 内部 prompt。
- 生产架构细节。
- 安全漏洞细节。
- 未公开模型或项目。

可以做脱敏：

- 改写业务背景。
- 使用模拟数据。
- 使用相对指标。
- 隐去公司和客户名称。
- 只讲方法，不暴露内部实现。

公开表达的目标是展示能力，而不是泄露信息。

如果不确定能不能公开，宁可写成私有面试材料。

78.10 面试表达

如果面试官问：“你如何证明自己具备大模型工程能力？”

可以这样回答：

我会用项目证据而不是只列技术栈来证明。比如在简历中，我会写清楚我解决的具体问题、采用的方案、我的角色、指标变化和业务范围。

如果有公开项目，我会在 [GitHub README](#) 中写清问题、架构、快速开始、评测方法和已知限制，而不是只放代码。技术博客方面，我

我认为大模型工程能力的证据链应该包括：能定义问题，能设计系统，能做实验，能上线验证，能处理风险成本，能总结复盘。简历

78.11 常见误区

第一个误区是简历堆术语。

熟悉很多工具，不等于解决过真实问题。

第二个误区是项目没有指标。

没有指标，面试官很难判断效果。

第三个误区是 [GitHub](#) 只有代码没有 [README](#)。

代码再好，没人看懂也很难产生价值。

第四个误区是博客只写概念综述。

真实项目复盘通常更有说服力。

第五个误区是所有岗位用同一套材料。

应用、算法、推理、[Infra](#)、负责人岗位关注点不同。

第六个误区是公开泄露敏感信息。

展示能力不能以违反保密为代价。

第七个误区是材料不更新。

项目做完不复盘，面试后不沉淀，长期会反复从零准备。

78.12 小结

求职材料包的核心，是把你的能力变成可验证证据。

简历是入口。

项目一页纸帮助你稳定表达。

项目复盘展示你的工程判断。

GitHub README 展示你的工程可读性。

技术博客展示你的思考和总结能力。

面试题库和复盘记录帮助你持续迭代。

大模型岗位竞争激烈，单靠“熟悉某某技术栈”已经不够。

真正有说服力的是：你能讲清楚自己解决过什么问题，如何设计系统，如何验证效果，如何处理失败，如何权衡成本、安全和业务价值。

把这些材料持续沉淀下来，你的每一次项目、每一次面试、每一次复盘，都会变成下一次机会的资产。

第 79 章：模拟面试训练与刻意练习

上一章讨论了求职材料包、作品集与公开证据。

材料准备好之后，还需要把它们转化成稳定的面试表现。

很多候选人不是不会，而是在面试现场讲不出来。

原因通常不是知识完全缺失，而是缺少模拟训练。

大模型工程师面试覆盖范围很广：原理、项目、系统设计、debug、实验评测、安全、成本、行为面和英语表达。

如果只靠临场发挥，很容易出现：

- 项目讲得太散。
- 系统设计没有结构。
- 被追问时慌乱。
- 指标和细节记不清。
- 英语表达卡顿。
- 行为面只讲经历，不讲判断。

这一章给出一套可执行的模拟面试训练方法。

目标不是背答案，而是建立稳定表达能力。

79.1 模拟面试训练什么

模拟面试不是随便找几个题问一问。

它应该覆盖六类能力。

第一，项目表达。

能否在 2 分钟、5 分钟、15 分钟三个长度下讲清一个项目。

第二，技术原理。

能否解释 Transformer、RAG、Agent、KV cache、LoRA、推理优化、评测等核心概念。

第三，系统设计。

能否从需求、约束、架构、数据流、指标、故障和演进讲方案。

第四，debug 和排查。

能否面对“效果下降”“延迟变高”“成本飙升”“工具误调用”给出排查路径。

第五，行为面。

能否讲冲突、失败、协作、推动落地、技术取舍和成长。

第六，反向提问。

能否通过问题判断团队是否适合自己。

这六类能力共同决定面试稳定性。

79.2 建立题库而不是临时刷题

建议建立自己的面试题库。

结构可以是：

```
interview-bank/  
  project.md  
  fundamentals.md  
  system-design.md  
  debugging.md  
  evaluation.md  
  safety-cost.md  
  behavioral.md  
  english.md  
  reverse-questions.md
```

每个问题至少包含：

- 问题。
- 标准回答结构。
- 关键术语。
- 可展开追问。
- 自己的项目例子。
- 常见错误。

不要只收藏题目。

题目没有答案结构，训练价值有限。

79.3 项目表达训练

每个核心项目至少准备三个版本。

第一，30 秒版本。

用于自我介绍或快速概述。

我做过一个企业知识库 RAG 系统，主要负责检索质量和引用可信度。我们从文档解析、hybrid retrieval、rerank、引用校验和 bad

第二，2 分钟版本。

用于项目简介。

要讲清：背景、问题、方案、指标、我的角色。

第三，10 分钟版本。

用于深挖。

要展开：架构、数据、评测、上线、失败、取舍、安全和成本。

训练时可以录音。

听回放检查：

- 是否有背景。
- 是否有明确问题。
- 是否有指标。
- 是否讲清我的贡献。
- 是否过度堆术语。
- 是否能自然进入追问。

79.4 系统设计训练

系统设计题不要一上来画架构。

先问需求和约束。

一个稳定模板是：

1. 明确需求：用户、场景、规模、SLO、安全、成本。
2. 定义成功指标：质量、延迟、吞吐、成本、风险。
3. 画核心链路：输入 -> 数据/工具/模型 -> 输出 -> 日志反馈。
4. 拆模块：API、数据、模型、评测、监控、权限、运维。
5. 讨论 trade-off：RAG vs 长上下文，API vs 自建，强模型 vs 小模型。
6. 设计上线策略：离线评测、灰度、监控、回滚。
7. 说明演进路线：MVP、生产化、平台化。

可以重点练这些题：

- 设计企业知识库 RAG 系统。
- 设计客服 Agent 系统。
- 设计大模型评测平台。
- 设计 OpenAI-compatible serving 平台。
- 设计 prompt 和模型版本管理系统。
- 设计多租户大模型应用平台。

每题至少练一次白板讲解。

79.5 Debug 训练

Debug 题考的是排查顺序。

不要直接猜原因。

例如面试官问：“RAG 系统最近回答质量下降，你怎么排查？”

可以这样答：

我会先确认影响范围，是全局下降还是某个业务、某类文档、某个模型版本、某个时间段下降。然后拆链路看：用户 query 分布是否

常练 debug 题：

- RAG 回答质量下降。
- Agent 工具误调用增加。
- TTFT 变高。
- TPOT 变高。
- token 成本突然上升。
- prompt 改动后格式错误增加。
- 模型切换后用户反馈变差。
- 越权召回事故。

Debug 回答要体现：先分范围，再拆链路，再看变更，再验证假设。

79.6 行为面训练

行为面不要只讲故事。

要讲你的判断。

推荐使用 STAR+R:

Situation: 背景是什么。

Task: 你承担什么责任。

Action: 你做了什么。

Result: 结果是什么。

Reflection: 你学到了什么, 下次如何改进。

大模型工程师常见行为题:

- 讲一次你推动跨团队项目的经历。
- 讲一次技术方案被挑战的经历。
- 讲一次线上失败或事故。
- 讲一次你做技术取舍的经历。
- 讲一次你发现自己判断错了。
- 讲一次你如何处理不确定性。
- 讲一次你如何说服团队不做某个方案。

每个故事都要准备结果和反思。

没有反思的故事, 容易显得不成熟。

79.7 英语面试训练

如果目标是外企或英文面试, 英语表达要单独练。

重点不是口音完美。

重点是结构清楚。

建议准备这些英文模板:

- Self introduction.
- Project overview.
- Technical trade-off.
- System design clarification.
- Debugging approach.
- Behavioral stories.
- Asking reverse questions.

例如项目开场:

One of my most relevant projects was an enterprise RAG system for internal knowledge search. My main responsibility was improving case feedback. As a result, citation accuracy improved from 71% to 86%, while P95 latency stayed within 2.3 seconds.

英语训练要录音。

检查是否:

- 句子太长。
- 术语表达不稳定。
- 逻辑连接词不足。
- 遇到追问就卡住。

79.8 一周模拟面试节奏

如果有一周集中准备, 可以这样安排:

第一天：整理简历和项目一页纸。

第二天：练项目表达和行为面。

第三天：练技术原理。

第四天：练系统设计。

第五天：练 debug、评测、安全和成本题。

第六天：完整 mock interview。

第七天：复盘、补短板、整理反向提问。

每次训练结束必须写复盘：

- 哪个问题答得散。
- 哪个术语不熟。
- 哪个项目指标忘了。
- 哪个追问没有接住。
- 下一次如何改。

没有复盘的模拟面试，提升很有限。

79.9 面试当天状态管理

面试当天不要继续大量学习新内容。

重点是稳定状态。

建议：

- 提前检查网络、摄像头、麦克风。
- 准备简历、项目一页纸和反向问题。
- 复习核心项目 30 秒和 2 分钟版本。
- 准备纸笔记录问题。
- 遇到不会的问题先澄清，不要硬编。
- 回答过长时主动总结。
- 面试结束后立即复盘。

如果不会，可以说：

这个细节我没有在生产中直接实现过，但我会从问题目标和系统约束来分析。我的理解是...

诚实加结构，比硬装更好。

79.10 小结

模拟面试训练的目标，是把知识转化成稳定表达。

大模型工程师面试不是只考背概念。

它考你能不能讲清项目、设计系统、排查问题、解释取舍、处理风险、表达成长。

有效训练要覆盖项目表达、原理、系统设计、debug、行为面、英语表达和反向提问。

每次模拟都要记录、复盘、修正。

真正的面试能力不是临场发挥，而是长期刻意练习形成的稳定输出。

第 80 章：常见失败案例与反面教材

上一章讨论了模拟面试训练与刻意练习。

这一章从反面看问题。

很多大模型工程师候选人不是因为完全不会而失败，而是踩中了非常典型的坑。

这些坑包括：项目包装过度、只会讲框架、没有指标、忽视安全、不会复盘、面试中硬编经验、入职后急于重构等。

反面教材的价值在于：它能帮助你提前避坑。

80.1 失败一：把 demo 当生产项目讲

很多候选人做过 demo，但在简历和面试中讲成生产项目。

面试官一追问：

- 有多少真实用户？
- 评测集怎么构建？
- 如何灰度？
- 如何回滚？
- 线上失败怎么处理？
- 成本如何控制？

候选人就答不上来。

demo 可以讲，但要讲清边界。

正确表达是：

这是一个个人项目或原型验证，重点验证 RAG 链路和评测方法。它没有真实线上流量，所以我主要用公开数据和自建评测集验证 retrieval accuracy。

诚实边界比夸大更可信。

80.2 失败二：简历堆满技术名词

“熟悉 RAG、Agent、LangChain、vLLM、LoRA、MCP、向量数据库”不是项目能力。

面试官关心的是：

- 你解决了什么问题？
- 你负责哪一块？
- 你如何验证效果？
- 你遇到什么失败？
- 你做了什么取舍？

堆技术名词会让候选人看起来像在追热点。

简历要从工具清单改成结果清单。

80.3 失败三：没有指标

大模型项目没有指标，很难证明有效。

常见空泛表达：

显著提升回答质量。

大幅降低延迟。

优化用户体验。

提升系统稳定性。

更好的表达是：

citation accuracy 从 71% 提升到 86%。

P95 TTFT 从 3.4s 降到 1.9s。

格式合法率从 92% 提升到 98%。

人工接管率下降 18%。

单次成功任务成本下降 27%。

如果没有精确指标，也要给出样本规模、评测方式和相对变化。
没有指标的项目，很难形成说服力。

80.4 失败四：只会讲框架，不会讲原理

候选人说用了 LangChain、LlamaIndex、vLLM、SGLang。

面试官继续问：

- chunk size 为什么这样选？
- rerank 解决什么问题？
- PagedAttention 不是 sparse attention 吗？
- continuous batching 和 dynamic batching 有什么区别？
- Agent 工具调用失败怎么恢复？

如果答不上来，框架名会变成扣分项。

框架是工具。

面试要讲清工具背后的机制和取舍。

80.5 失败五：忽视安全和权限

很多候选人讲 Agent 时只讲智能。

但不讲权限、审计、确认和回滚。

这在生产面试中很危险。

如果模型能调用工具，就必须问：

- 工具是否有副作用？
- 参数如何校验？
- 权限在哪里执行？
- 高风险操作是否确认？
- 调用日志如何审计？
- prompt injection 怎么防？

只讲“模型会自动调用工具”是不够的。

生产系统里，模型可以建议，系统必须校验。

80.6 失败六：不会讲失败

完美项目故事通常不真实。

成熟候选人应该能讲失败。

例如：

- 最初判断错了瓶颈。
- 某次 prompt 改动导致格式回退。
- rerank 提升质量但增加延迟。
- 长上下文方案成本不可接受。
- Agent 自主性过高导致误调用风险。

关键不是失败本身。

关键是你如何发现、定位、修正和复盘。

不会讲失败的人，面试官会怀疑你没有真实生产经验。

80.7 失败七：面试中硬编经验

大模型面试追问很深。

如果硬编经验，很容易露馅。

例如你说做过 vLLM 生产优化，面试官可能问：

- TTFT 和 TPOT 如何拆？
- KV cache block 如何估算？
- prefix cache 命中率是多少？
- prefill 和 decode 如何调度？
- OOM 时看哪些指标？

如果没有真实经验，很难持续回答。

正确做法是承认边界。

可以说：

我没有在生产中维护过 vLLM 集群，但我做过本地 benchmark，并系统学习过 serving engine 的请求生命周期、KV cache、batching 等。

诚实边界不会一定扣分。

硬编通常会严重扣分。

80.8 失败八：入职后急于证明自己

有些人刚入职就想重构系统。

但没有理解历史约束，就容易破坏已有系统。

正确做法是：

- 先理解业务和历史决策。
- 跑通链路。
- 看线上指标。
- 和相关方沟通。
- 选择小闭环建立可信用。
- 再提出中期改进。

前 30 天最重要的不是证明自己比前人聪明。

而是证明自己可靠、能学习、能协作、能形成闭环。

80.9 失败九：只追热点，不沉淀资产

今天学 RAG，明天学 Agent，后天学 MCP，大后天学推理框架。

但每个方向都没有项目复盘、指标、代码、文档和面试表达。

这会导致知识很散。

正确做法是每学一个方向，都沉淀一个资产：

- 一篇项目复盘。
- 一个可运行 demo。
- 一个评测集。
- 一个 debug checklist。
- 一个标准回答。

职业成长靠资产累积，不靠热点堆积。

80.10 小结

常见失败并不复杂。

它们大多来自不真实、不结构化、不复盘、不看指标、不讲边界。

大模型工程师要避免：

- 把 demo 当生产。
- 堆术语。
- 没指标。
- 只讲框架。
- 忽视安全。
- 不讲失败。
- 硬编经验。
- 入职急于重构。
- 只追热点不沉淀。

真正可靠的候选人，不是永远不犯错，而是能清楚知道边界、能用证据说话、能从失败中学习。

第 81 章：完整求职作战路线图

前面几章分别讨论了材料、模拟面试、offer 选择和失败案例。

这一章把所有内容整合成一条完整求职路线图。

目标是让你知道：从准备到投递，从面试到 offer，从入职到成长，每一步应该做什么。

81.1 第一阶段：定位目标岗位

先不要急着投简历。

先确定目标岗位。

大模型岗位至少分几类：

- 大模型应用工程师。
- RAG 工程师。
- Agent 工程师。
- 大模型算法工程师。
- 推理部署工程师。
- AI Infra 工程师。
- 评测平台工程师。
- 大模型产品工程师。

不同岗位准备重点不同。

应用工程师重 RAG、Agent、工具调用、上线和反馈。

算法工程师重模型原理、训练、微调、数据和实验。

推理工程师重 serving、KV cache、batching、并行、延迟和成本。

负责人岗位重路线图、跨团队协作、风险、成本和业务价值。

定位不清，简历和面试都会散。

81.2 第二阶段：补齐核心能力

根据目标岗位补能力。

最低公共能力包括：

- Transformer 和 LLM 基础。
- Prompt、RAG、Agent 基础。
- Python 和工程能力。
- 评测和实验方法。
- API、服务化和日志。
- 安全、权限和成本意识。

不要无限学习。

围绕目标岗位补最短路径。

例如投 RAG 岗位，就优先补：文档解析、chunk、embedding、hybrid retrieval、rerank、citation、评测、bad case。

例如投推理岗位，就优先补：prefill、decode、KV cache、PagedAttention、continuous batching、TTFT、TPOT、vLLM/SGLang。

81.3 第三阶段：做一个可讲项目

至少准备一个主项目。

主项目要能讲 30 分钟。

它应该包括：

- 背景。
- 问题。
- 架构。
- 数据。
- 模型。
- 评测。
- 指标。
- 风险。
- 成本。
- 失败和改进。

如果没有公司项目，可以做个人项目。

但个人项目要尽量生产化：

- 有 README。
- 有评测。
- 有错误分析。
- 有日志。
- 有安全边界说明。
- 有后续计划。

一个完整项目胜过五个浅 demo。

81.4 第四阶段：准备求职材料包

材料包包括：

- 一份目标岗位简历。
- 一个项目一页纸。
- 一个项目复盘。
- 一个 GitHub 或代码样例。
- 一个面试题库。
- 一个反向提问清单。

如果目标岗位不同，准备不同版本。

不要一份简历投所有岗位。

81.5 第五阶段：投递和内推

投递时要记录。

表格字段包括：

- 公司。
- 岗位。
- JD 要点。
- 投递渠道。
- 简历版本。
- 当前状态。
- 面试轮次。
- 面试问题。
- 复盘。
- 结果。

内推时不要只发简历。

最好附上：

- 目标岗位。
- 你和岗位匹配的三点。
- 一页纸项目摘要。
- GitHub 或作品链接。

让内推人更容易帮你。

81.6 第六阶段：面试准备

按轮次准备。

HR 面：讲动机、背景、期望、稳定性。

技术一面：讲原理和项目细节。

技术二面：讲系统设计、debug、实验和工程深度。

交叉面：讲协作、安全、成本、业务理解。

主管面：讲成长、owner 意识、路线图和影响力。

每轮都要准备反向问题。

面试是双向选择。

81.7 第七阶段：面试复盘和迭代

每场面试后立刻复盘。

记录：

- 问了什么。
- 答得如何。
- 追问在哪里。
- 暴露了什么短板。
- 团队有什么信息。
- 下一场如何改。

复盘后更新题库和标准回答。

求职过程本身就是训练过程。

越复盘，越稳定。

81.8 第八阶段：Offer 选择和谈薪

offer 选择看：

- 岗位真实性。
- 项目阶段。
- 团队能力。
- 个人职责。
- 资源支持。
- 成长空间。
- 组织稳定性。
- total compensation。

谈薪前明确底线。

拿到完整结构后再比较。

不要只看 base。

也不要只看短期薪资，忽略长期职业资产。

81.9 第九阶段：入职前准备

接受 offer 后，准备：

- 复习岗位相关技术。
- 整理 30-60-90 天计划。
- 确认汇报对象和第一个项目。
- 了解团队技术栈。
- 准备 onboarding 问题。

入职不是求职结束。

真正的职业收益要靠入职后兑现。

81.10 小结

完整求职路线不是“投简历、刷题、面试”。

它应该是：

定位岗位 -> 补能力 -> 做项目 -> 准备材料 -> 投递内推 -> 模拟面试 -> 复盘迭代 -> offer 选择 -> 入职计划
每一步都要形成资产。

当你把求职当成系统工程，而不是临时冲刺，成功率和长期收益都会更高。

第 82 章：最终自检清单与补充篇总结

这一章是大模型工程师面试补充篇的收束章。

从第 60 章到第 82 章，我们讨论了从项目包装、面试表达、入职成长、技术判断、跨团队影响、路线图、指标复盘、安全风险、成本 ROI、长期成长，到求职材料、模拟面试、offer 选择和完整求职路线。

这套内容的目标不是制造焦虑，而是帮助你把“大模型工程师”从一个模糊岗位，拆成一套可训练、可证明、可成长的能力系统。

82.1 项目能力自检

你应该能回答：

- 我最核心的 1-2 个项目是什么？
- 每个项目解决了什么真实问题？
- 我的具体贡献是什么？
- 系统架构是什么？
- 数据链路是什么？
- 使用了哪些模型和工具？
- 如何评测？
- 指标如何变化？
- 失败 case 是什么？
- 安全和成本如何处理？
- 如果重做一次，我会怎么改？

如果这些问题答不清，先补项目复盘。

82.2 技术能力自检

你应该能解释：

- Transformer 和 attention。
- tokenization 和 position encoding。
- pretraining、SFT、RLHF、DPO。
- LoRA 和参数高效微调。
- RAG 的完整链路。
- embedding、hybrid retrieval、rerank。
- Agent、function calling、tool use。
- LLM eval 和 LLM-as-a-judge。
- KV cache、prefill、decode。
- vLLM、SGLang、PagedAttention、continuous batching。
- prompt injection、权限、审计和安全边界。

不要求每个方向都源码级深入。

但至少要把核心机制、适用场景和 trade-off。

82.3 工程能力自检

你应该能讲清：

- 如何设计一个生产级 RAG 系统。
- 如何设计一个安全 Agent 系统。
- 如何设计一个推理服务。
- 如何设计评测平台。
- 如何做灰度和回滚。
- 如何做日志和 trace。
- 如何排查质量下降。
- 如何排查延迟上升。
- 如何做成本归因。
- 如何处理线上事故。

大模型工程师不是只调 prompt。

你要能把模型放进真实系统。

82.4 面试表达自检

你应该准备：

- 30 秒自我介绍。
- 2 分钟自我介绍。
- 30 秒项目介绍。
- 2 分钟项目介绍。
- 10 分钟项目深挖版本。
- 5 个行为面故事。
- 3 个系统设计模板。
- 5 个 debug 排查模板。
- 1 份反向提问清单。
- 1 份英语项目介绍。

所有回答都要结构化。

不要临场组织。

82.5 求职材料自检

你应该有：

- 一份目标岗位简历。
- 一个项目一页纸。
- 一个项目复盘文档。
- 一个 GitHub README 或代码样例。
- 一个面试题库。
- 一个公司投递跟踪表。
- 一个 offer 对比表。
- 一个 30-60-90 天计划。

材料不是一次性准备。

每次面试和项目后都要更新。

82.6 职业判断自检

你应该能判断：

- 这个岗位是真大模型工程，还是 API 包装？
- 这个团队是否有真实用户？
- 是否有评测闭环？
- 是否有安全和成本意识？
- 是否有技术 owner？
- 是否有资源支持？
- 我能否负责完整问题？
- 未来 1-2 年能否形成职业资产？

好的职业选择，不只是拿到 offer。

而是进入一个能持续成长和交付的环境。

82.7 最后一版求职前检查

投递前，检查：

- 简历是否匹配目标岗位。
- 项目是否有指标。
- 是否有一页纸。

- 是否能讲失败和取舍。
- 是否准备反向问题。
- 是否知道自己的薪资底线。
- 是否准备好面试复盘表。

面试前，检查：

- 核心项目是否能 2 分钟讲完。
- 高频原理题是否能讲清。
- 系统设计是否有模板。
- debug 是否有排查顺序。
- 行为故事是否有结果和反思。
- 英语表达是否练过。

拿 offer 前，检查：

- 岗位职责是否清楚。
- 前 90 天目标是否清楚。
- 汇报对象是否清楚。
- 薪酬结构是否完整。
- 成长空间是否真实。
- 风险是否可接受。

82.8 补充篇总结

大模型工程师的竞争力，不是一组孤立技能。

它是一套系统能力：

- 懂模型原理。
- 会工程落地。
- 能设计评测。
- 能处理线上问题。
- 有安全和成本意识。
- 能跨团队沟通。
- 能做技术判断。
- 能沉淀资产。
- 能持续成长。

面试只是这套能力的一次展示。

求职只是职业路径的一段。

真正长期有效的，是你能否不断把真实问题转化成项目，把项目转化成复盘，把复盘转化成能力，把能力转化成更大的影响力。

如果你已经读完这套补充篇，下一步不是继续无限读材料。

下一步应该是行动：

- 整理简历。
- 复盘项目。
- 写一页纸。
- 做模拟面试。
- 补一个真实 demo。
- 写一篇技术复盘。
- 投递几个目标岗位。
- 每次面试后更新题库。

学习最终要落到行动。

大模型工程师的成长，也只能在真实问题、真实反馈和真实交付中完成。